

Echo



שם בית הספר: תיכון אלון

שם העבודה: Echo

שם התלמידה: תמר בז'רנו

ת.ז. התלמידה: 216988774

שמות המנחים: עידן פלדברג, רום רפסון, עודי רז,

קובי שוצמן

שם החלופה: הגנת סייבר

תאריך הגשה: 30.4

תוכן

4	מבוא:
4	ייזום:
4	תיאור ראשוני של המערכת:
4	הגדרת הלקוח:
4	הגדרת יעדים:
4	בעיות תועלות וחסכונות:
5	סקירת פתרונות קיימים:
5	סקירת טכנולוגיית הפרויקט:
6	תיחום הפרויקט:
7	פירוט תיאור המערכת:
7	תיאור מפורט של המערכת-
7	יכולות לפי סוגי משתמש:
8	פירוט הבדיקות- קופסא שחורה שמתוכננות לפרויקט:
10	תכנון וניהול לו"ז זמנים לפיתוח המערכת:
10	ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם.
11	תיאור תחום הידע - פרק מילולי:
11	פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר:
11	יכולות צד השרת:
13	יכולות בצד לקוח-
15	מבנה / ארכיטקטורה של הפרויקט
15	תיאור הארכיטקטורה הכללית:
16	תיאור הטכנולוגיה הרלוונטית:
17	תיאור זרימת המידע במערכת:
22	תיאור האלגוריתמים מרכזיים בפרויקט:
22	ניסוח וניתוח של הבעיה האלגוריתמית:
27	תיאור סביבת הפיתוח:
27	פרוט כלי הפיתוח הדרושים לפיתוח:
28	תיאור פרוטוקול התקשורת:
28	תיאור מילולי של פרוטוקול התקשורת:
29	פירוט כלל ההודעות הזורמות במערכת:
30	תיאור מסכי המערכת:
39	תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם:
41	תיאור מבני הנתונים:
43	סקירת חולשות והאיומים:

43	שכבת האפליקציה:
44	שכבת התעבורה:
44	הפעלת המערכת
45	מימוש הפרויקט:
45	חלק א'-
60	חלק ב'-
67	חלק ג'
71	מדריך למשתמש:
71	עץ קבצי המערכת:
71	התקנת המערכת:
72	משתמשי המערכת:
72	מדריך למשתמש Echo Client:
84	סיכום אישי / רפלקציה:
84	תיאור תהליך העבודה על הפרויקט:
84	תיאור תהליך הלמידה:
84	כלים להמשך:
84	תובנות מהתהליך:
85	בראייה לאחור:
85	משאבים נוספים:
85	שאלות חקר עצמי:
85	תודות:
86	ביבליוגרפיה:
87	נספחים
87	צירוף קוד הפרויקט:

מבוא:

י"ז:

תיאור ראשוני של המערכת: Echo היא אפליקציה שתהפוך למשתמשים שלה קבצי אודיו לספר דיגיטלי אותו הם יכולים לשמור ולדפדף בו. הספרים שאותו משתמש מכין ישמרו לו במין ספרייה דיגיטלית.

המוצר המוגמר תהיה אפליקציה בה המשתמש מכניס קובץ אודיו או מקליט בעצמו באפליקציה, זה יהפוך לו ספר דיגיטלי או לקובץ pdf שאותו משתמש יכול להוריד.

Echo נוצרה מתוך הצורך האישי שלי לפתרון בעיה אליה נחשפתי במהלך המסע לפולין. במהלך המסע כל יום היו נסיעות ארוכות מאוד באוטובוס. אז כשחשבתי בבית לפני הטיסה מה אני אעשה בהן הגעתי למסקנה שאני אוריד אודיובוקס וספרים דיגיטליים שאקשיב ואקרא בדרך. אבל הופתעתי לגלות שכשאני קונה אודיובוק אני לא מקבלת עותק דיגיטלי של הספר כתוב אלא רק קובץ קולי. כתוצאה מכך חשבתי על הרעיון והגעתי למסקנה שאם אני ארחיב אותו יכולים להיות לו שימושים רבים ומגוונים להרבה קבוצות אנשים שונות.

האתגרים שאני צופה לי בפרוייקט הם: עבודה עם בינה מלאכותית, הפיכת אודיו לטקסט ואז יצירה ועיצוב של flip-book המלא בטקסט הזה, התחברות דרך שם משתמש וסיסמא שמספקת למשתמש גישה לספריו הדיגיטליים, כפתור שמאפשר למשתמש להקליט את עצמו ואז להעלות את ההקלטה לאפליקציה ולנגן את ההקלטה בעת הצורך.

הגדרת הלקוח: האנשים להם האפליקציה עשויה להועיל הינם: הורים שמקליטים ספר לילדים שלהם. מורים שמקליטים שיעורים לתלמידים. כל אחד שרוצה להוציא ספר. כל אחד שרוצה לכתוב יומן אבל שמתקשה לכתוב בעצמו ומעדיף להקליט. כל אחד שרוצה לקרוא יותר, תלמידים של מנטורים שרוצים להקליט את העצות ותובנות שלהם על מנת לקרוא אותם אחר כך.

הגדרת יעדים: המטרה של אקו: לאפשר לכל משתמש להפוך את קבצי הקול שלו לספר דיגיטלי. ספר שהוא יוכל לשמור, לדפדף, לקרוא ולהקשיב לקובץ הקול תוך כדי הקריאה שלו.

בעיות תועלות וחסכונות: בעיה אפשרית יכולה להיות שיש לי קובץ פודקאסט שאני מעדיפה לקרוא כי הוא מרעיש ואני אל יכולה להקשיב אליו/ אני קולטת מידע כתוב יותר טוב... הפתרון שהאפליקציה תספק יהיה להתחבר למשתמש באפליקציה, להדביק בפנים את הקובץ ולקרוא. המערכת תספק ספר דיגיטלי של המוקלט ותאפשר למשתמש לקרוא כרצונו ואף להקשיב לפודקאסט תוך כדי הקריאה.

סקירת פתרונות קיימים: לא מצאתי אתר שעושה את ההמרה של אודיו לספר דיגיטלי אבל מצאתי כמה אתרים שעושים משהו דומה:

-FLIPHTML5
https://fliphtml5.com/?_gl=1*gztbak*_up*MQ..*_gs*MQ..&gclid=Cj0KCQIA4rK8BhD7ARIsAFe5LXlKofodlKOAiC9muD-PZN_XITP1SfSOQwXP4GERCMc529tz-D9L4QaAtabEALw_wcB

האתר הופך קבצים של טקסט לספרים דיגיטליים

-COCKATOO
https://www.cockatoo.com/?gad_source=1&gclid=Cj0KCQIA4rK8BhD7ARIsAFe5LXlh8FPYt_1LUJr0CbWGAWSOymiQhRSaled4Fpu_hv16HxPIWQIG2z4aAqvYEALw_wcB

האתר הופך קבצי אודיו לטקסט

סקירת טכנולוגיית הפרויקט: באפליקציה לא נעשה שימוש בטכנולוגיה חדשנית או בלתי מוכרת, וכל הספריות הנבחרות פופולריות, מתועדות ונמצאות בשימוש רחב. Echo משתמשת בטכנולוגיה של תמלול עם AI שקיימת, אך היא נעזרת בה כדי לעשות משהו שלא קיים כל כך. וזה הפיכתם לספר דיגיטלי שמורכב מהתמלול.

סייגים והגבלות במערכת:

תלות ב Cloudinary לתמלול: אין מנגנון תמלול מקומי, התמלול מתבצע באמצעות שימוש ב-CLOUDINARY. ללא חיבור אינטרנט או חשבון Cloudinary תקין, פונקציונליות התמלול לא תעבוד.

תלות בPortAudio להקלטה: PyAudio דורשת התקנת מנהלי התקן של PortAudio. ללא התקנה זו ההקלטה תיכשל.

עמידות מוגבלת בעומס: השרת מריץ כל חיבור ב-thread נפרד ומתאים לעשרות חיבורים, אך לא מוגדר לאלפים של משתמשים בו-זמנית.

תיחום הפרויקט:

התחומים בהם הפרויקט עוסק:

רשתות ותקשורת:

פרוטוקול TCP בין קליינט (או מספר קליינטים) לשרת.

ערוץ מוצפן: החלפת מפתח RSA ואז הצפנת AES-GCM

מנגנון Fixed-Window Rate-Limiting למניעת מתקפת DDOS

מערכות הפעלה ותשתיות:

תומך בווינדואוס, מאק ולינוקס כל עוד מותקן פייתון, אין תלות במערכת הפעלה ספציפית מעבר לדרישות אלו.

:GUI

ממשק משתמש גרפי מלא בcustomtkinter, חלונות ל-Login, יצירת חשבון, הקלטת שמע, ספרייה דיגיטלית וספר דיגיטלי.

בסיס נתונים:

אחסון מקומי בSQLite

עיבוד אודיו ותמלול:

הקלטה והשמעת WAV באמצעות PyAudio וPydub

תמלול אודיו לטקסט באמצעות Cloudinary

פירוט תיאור המערכת:

תיאור מפורט של המערכת-

מערכת Echo היא אפליקציית לקוח-שרת שמיועדת לאפשר למשתמשים:

הרשמה ולוג-אין מאובטחים באמצעות הצפנות בשילוב RSA/AES.

הקלטת אודיו בממשק גרפי, והעלאתו לענן (Cloudinary) לתמלול אוטומטי- הפיכת האודיו לטקסט באמצעות בינה מלאכותית.

שמירה של תוצאת התמלול כספר דיגיטלי בדאטה בייס.

עריכה, הצגה, הורדה וגלילה של הספר הדיגיטלי.

ניהול ספריה דיגיטלית של המשתמש המכילה את רשימת הספרים הקיימים שיצר עם האפשרות לפתוח כל אחד מהם ושהוא יכול את התוכן ששייך לו.

השרת מבצע: החלפת מפתח AES מוצפן ב-RSA, תיאום תקשורת AES-GCM מוצפנת עם הלקוח, אימות משתמשים בלוג אין מול דאטה בייס, העלאת קבצי אודיו לתמלול ב-Cloudinary ולקיחת התמלול בחזרה, טיפול בריבוי לקוחות באמצעות multithreading וב-DDOS

יכולות לפי סוגי משתמש:

משתמש בלתי רשום: יצירת חשבון חדש

משתמש רשום: התחברות- לוג אין, הקלטת שמע והעלאה לתמלול, יצירת ספר דיגיטלי מהתמלול הנוצר, צפייה ועריכה בספר עם אפשרות להורדתו כקובץ-PDF, גלישה בספריה הדיגיטלית

פירוט הבדיקות- קופסא שחורה שמתוכננות לפרויקט:

מספר	שם הבדיקה	מטרה	אופן הביצוע
1	התחברות תקינה	לוודא שמשתמש קיים מצליח להיכנס עם שם וסיסמה תקינים	במסך ההתחברות להזין שם משתמש וסיסמה שהוזנו מראש, ללחוץ "Login" ולקבל אישור הצלחה
2	התחברות עם פרטי משתמש שגויים	לוודא שהמערכת חוסמת ניסיון כניסה עם שם משתמש או סיסמה שגויים	במסך ההתחברות להזין שם תקין וסיסמה שגויה (או להפך), ללחוץ "Login" ולקבל הודעת שגיאה
3	יצירת חשבון חדש	לוודא שניתן להירשם עם שם משתמש וסיסמה העומדים בקריטריונים	במסך "Create New Account" להזין שם חדש וסיסמה חזקה, ללחוץ "Submit" ולבדוק במסד הנתונים שהמשתמש נוסף
4	יצירת חשבון עם שם קיים	לוודא שלא ניתן לרשום שם משתמש שכבר קיים	לנסות לרשום פעמיים את אותו שם משתמש, בפעם השנייה לקבל הודעת שגיאה "Username already exists"
5	החלפת מפתח AES מוצפן באמצעות RSA	לוודא שהחלפת המפתח מאובטחת והשרת מקבל את המפתח כראוי	להריץ לקוח ושרת יחד, לוודא שהלקוח מקבל את המפתח הציבורי, שולח מפתח AES מוצפן, והשרת מפענח אותו
6	תקשורת-AES GCM מוצפנת	לוודא שההודעות מוצפנות ומתפענחות ללא שיבושים	לשלוח הודעה ארוכה, לוודא שהלקוח והשרת מקבלים ופענחים אותה בהצלחה ללא שגיאות אימות
7	העלאת קובץ אודיו לתמלול	לוודא שהקובץ עולה ל- Cloudinary מתמלל ומוחזר בהצלחה	לשלוח פקודת upload_audio עם קובץ WAV תקין, לבדוק שהמערכת מחזירה status: success ותמלול מלא
8	יצירת ספר דיגיטלי מהתמלול	לוודא שהתמלול נשמר כפריט חדש בטבלת BOOKS	לאחר קבלת תמלול תקין לשלוח create_digital_book, ולבדוק ב־ SQLite שהספר נוצר עם שם וטקסט

9	פתיחת ממשק Flipbook	לוודא שהספר מוצג כהלכה וניתן לדפדף בין הדפים	בממשק הספר ללחוץ על "Open" ולוודא שהתצוגה נטענת וניתן לעבור דפים קדימה ואחורה
10	יצוא ל-PDF	לוודא שנוצר קובץ PDF חוקי שמכיל את תוכן הספר	בממשק Flipbook ללחוץ על "Download" לפתוח את הקובץ ולוודא שהטקסט והפורמט נכונים

תכנון וניהול לו"ז זמנים לפיתוח המערכת:

שלב	משך מתוכנן (בשבועות)	משך זמן בפועל (בשבועות)	תכולה
דרישות ואפיון	1-2	1-2	איסוף דרישות, כתיבת אפיון, תרשימים ראשוניים
עיצוב ארכיטקטורה	3	3	הגדרת הרכיבים, RSA/AES, DB, Cloudinary, GUI
מימוש הליבה	4-5	4-5	מודול AES, לקוח/שרת בסיסי, GUI התחברות
הרחבת הפיצ'רים	6	6	הקלטת אודיו, upload/transcript, ספר דיגיטלי
כתיבת בדיקות	7	7	כתיבת בדיקות קופסה שחורה, תיקון באגים, בדיקת עומסים
כתיבת תיעוד	8-9	8	מדריך משתמש, מדריך התקנה, רפלקציה, ביבליוגרפיה

ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם**למידה והתמודדות עם תחומים חדשים**

הסיכון- ללמוד לעבוד עם API של Cloudinary, ומספר הצפנות שונות AES RSA, שהיוו עבורי תחומים חדשים ולא מוכרים שאני לא ידעתי כיצד לגשת אליהם ולגרום להם לעשות את מה שאני צריכה כדי שיתאימו לפרויקט שלי.

דרך התכנון- לפני שעירבתי את הדברים החדשים בקוד, קראתי בקפידה על כל אחד מהם ולא השתמשתי בו עד שהבנתי איך להוסיף אותו, מה הוא דורש ומה הוא יעשה בדיוק.

בפועל- השתמשתי בדוגמאות קוד מהתיעוד הרשמי כדי לכתוב מודולים קטנים לבדיקה, ולאחר מכן שילבתי אותם בהדרגה במערכת הראשית. כך הפחתתי תקלות רציניות בשלב האינטגרציה.

ניהול זמנים נכון

הסיכון- לא להספיק לסיים את קוד הפרויקט והספר בזמן עם הלחץ של כיתה י"ב

דרך התכנון- לסדר לעצמי ביומן מתי ההגשות של כל חלק ועל מה אני אעבוד בכל יום עד ההגשה כדי

שהכל יהיה מוכן בזמן

בפועל- התקשתי בהתחלה למצוא זמן לעבוד על הפרוייקט אך בסוף כשראיתי שנשאר לי ממש קצת זמן לסיים סופית את הכל, לקחתי את עצמי בידיים ועבדתי מסודר על כל חלק שנותר לי.

תיאור תחום הידע - פרק מילולי:

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר:

יכולות צד השרת:

שם היכולת: ניהול תקשורת עם הלקוחות במקביל והגנה מפני DDOS
מהות היכולת: יצירת שרת TCP המאזין לפנייות מלקוחות, קבלת חיבורים והגבלת קצב הבקשות מכל IP למניעת קריסה.

אוסף יכולות נדרשות:

- פתיחת socket
- הקשבה לכתובת IP ופורט

- קבלת חיבורים חדשים
 - מעקב אחר request_tracker ו IP חסומים
 - חסימת IP שחצה את המגבלה.
 - פתיחת thread לכל לקוח.
- אובייקטים נחוצים: socket, threading.Thread, time

שם היכולת: תמלול קובץ אודיו

מהות היכולת: העלאת קובץ שמע ל Cloudinary המתנה לביצוע תמלול אוטומטי ואחזור הטקסט.

אוסף יכולות נדרשות:

- פתיחת קונפיגורציה עם מפתחות API
 - קריאה ל cloudinary.uploader לעשות העלאה
 - לולאת retry בהמתנה לתמלול עד שה transcript מוכן
 - יצירת URL לתמלול.
 - הוצאת ה JSON ועיבוד הטקסט.
 - טיפול בשגיאות רשת/קבצים
- אובייקטים נחוצים: cloudinary.uploader, cloudinary.api, DigitalBookHandler, urllib.request
- שימוש ב-keyring לשליפת מידע סודי

שם היכולת: טיפול בבקשות משתמש

מהות היכולת: קבלת בקשות להרשמה או התחברות, אימות נתונים, ושליחה חזרה של סטטוס התחברות.

אוסף יכולות נדרשות:

- פענוח JSON שהתקבל מלקוח
 - חיפוש במסד הנתונים לפי מזהה משתמש
 - אימות סיסמה מוצפנת
 - שליחת הודעת תגובה עם תוצאה
- אובייקטים נחוצים:**
- ספריית json
 - מסדי נתונים users
 - קוד תיאום סוג הבקשה מול הפונקציה המתאימה

יכולות בצד לקוח-

שם היכולת: הרשמת משתמש חדש
מהות היכולת: שליחת פרטי משתמש חדשים לשרת לצורך פתיחת חשבון חדש.

אוסף יכולות נדרשות:

- קבלת קלט מהמשתמש
- יצירת אובייקט JSON עם נתוני הרשמה
- שליחת ההודעה לשרת
- קבלת תגובה – הצלחה או שגיאה

אובייקטים נחוצים:

- Echo_Login או מסך SignUp
- שדות טקסט, כפתור אישור
- פונקציית send_json

שם היכולת: התחברות עם משתמש קיים
מהות היכולת: אימות פרטי התחברות מול מסד הנתונים בשרת.

אוסף יכולות נדרשות:

- קבלת שם משתמש וסיסמה
- שליחה לשרת באמצעות JSON
- קבלת תגובת התחברות, פתיחת דף בית

אובייקטים נחוצים:

- Echo_Login
- פונקציית send_login_json
- Label להודעות שגיאה/הצלחה

שם היכולת: צפייה בספרים דיגיטליים
מהות היכולת: בקשת רשימת ספרים מהשרת והצגתם למשתמש.

אוסף יכולות נדרשות:

- שליחת בקשה לקבלת ספרים
- קבלת רשימה בפורמט JSON
- עיבוד הנתונים להצגה גרפית
- לחיצה על ספר לפתיחה

אובייקטים נחוצים:

- Echo_DigitalFlipbook
- מסך גלילה להצגת ספרים
- כפתור פתיחה לכל ספר

שם היכולת: יצירת ספר דיגיטלי חדש

מהות היכולת: הלקוח שולח בקשה לשרת ליצור ספר מאוסף הקלטות שלו.

אוסף יכולות נדרשות:

- בחירת הקלטות קיימות
- שליחת בקשה מסודרת לשרת
- קבלת אישור או שגיאה
- עדכון הספרים ברשימה המקומית

אובייקטים נחוצים:

- Echo_BookCreationWindow
- פונקציית send_create_book_request
- Label לתגובה מהשרת

שם היכולת: הקלטת אודיו והשמעה חוזרת

מהות היכולת: הקלטת קול מהמיקרופון לשמירתו כקובץ WAV ואפשרות להשמעתו לפני שליחה לשרת.

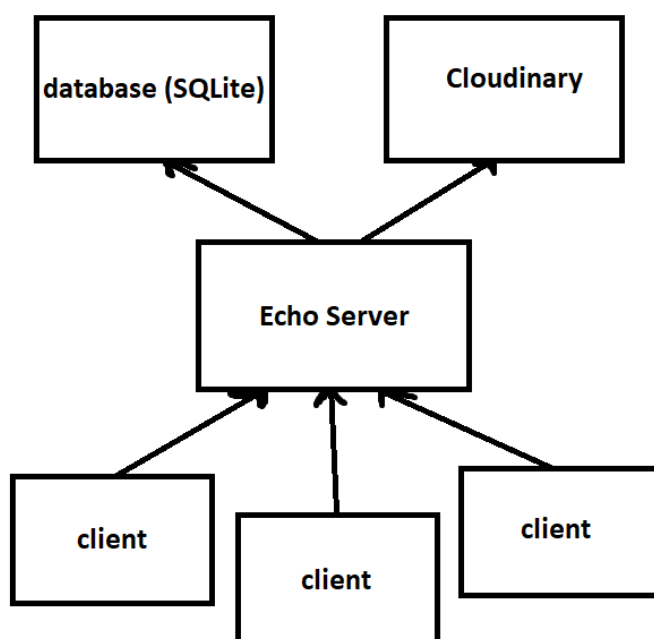
אוסף יכולות נדרשות:

- פתיחת pyaudio.Stream
- איסוף פריימים
- כתיבת פריימים לקובץ WAV
- טעינת הקובץ והשמעה חוזרת

אובייקטים נחוצים: pyaudio, wave, pydub.AudioSegment, RecordAudio

מבנה / ארכיטקטורה של הפרויקט**תיאור הארכיטקטורה הכללית:**

הפרויקט Echo בנוי כשתי ישויות עיקריות שפועלות בניהן בצורה של תקשורת מוצפנת מרובת משתמשים- הלקוח (יכולים להיות גם כמה לקוחות) והשרת. הארכיטקטורה מיושמת בשפת פייתון באמצעות ספריית socket. השרת מקשיב לכל חיבור חדש ומתחיל ת'רד ייעודי לכל לקוח המתחבר. התקשורת בין השרת ללקוחות מתבצעת באמצעות TCP- כשהלקוח פונה לשרת בבקשות מוגדרות: התחברות, שליחת קובץ אודיו, יצירת ספר והשרת מעבד אותן לפי סדר הגעתן. השרת מנהל את הליך ההצפנה, תחילה מתבצעת החלפת מפתח AES מוצפן ב-RSA, ולאחריה כל ההודעות עוברות הצפנה ב-AES-GCM. שרת Echo, כדי לתת מענה לבקשות רבות במקביל, מחובר לרשת אינטרנט אמינה כדי להעביר קבצים ונתונים לתמלול ב-Cloudinary ואל הקליינטים והלקוח (או לקוחות) מסוגל להקליד פרטי התחברות, להצפין אותם ולהעבירם לשרת, להקליט שמע, לשלוח קבצים בצורה מאובטחת.



תיאור הטכנולוגיה הרלוונטית: שפות התכנות-

Python- פייתון היא שפת תכנות קריאה ופשוטה המיועדת לביטוי קצר וברור של אלגוריתמים ועבודה נוחה עם ספריות, היא שפת הקוד העיקרית בפרויקט.

SQL- פעולות יצירה, קריאה, עדכון ומחיקה מתבצעות באמצעות שפת ה SQL שמשומשת בהוצאה ועדכון מידע בבסיס הנתונים, יצירת טבלאות ועדכון.

מערכות ההפעלה- הלקוח נבדק ורץ על Windows 10/11, כל עוד שבהפעלת הפרויקט השרת והלקוחות יהיו באותה רשת תקשורת, ניתן להפעיל גם על Linux או macOS ללא התאמות מיוחדות.

תקשורת- התקשורת בפרויקט היא מרובת משתמשים, בה התעבורה בין הלקוח לשרת מתבצעת על גבי TCP- פרוטוקול התחבורה של מודל השכבות- OSI לשידור נתונים אמין בין לקוח לשרת. לאחר החיבור, החלפת מפתח AES באמצעות RSA יוצרת ערוץ מוצפן ב- AES-GCM לכל שידור JSON.

הצפנת RSA שאחריה AES-GCM. RSA הוא אלגוריתם אסימטרי להצפנת מפתח AES.

ברגע שיש למערכת את המפתח הסימטרי, כל המידע עובר הצפנה באמצעות AES-GCM שמספק הצפנה סימטרית יחד עם TAG, שמוודא שהנתונים לא הושחתו או תוקנו בדרך

חומי עניין- אבטחת מידע: יישום של הצפנה, ניהול מפתחות וסנכרון ערוץ מוצפן.

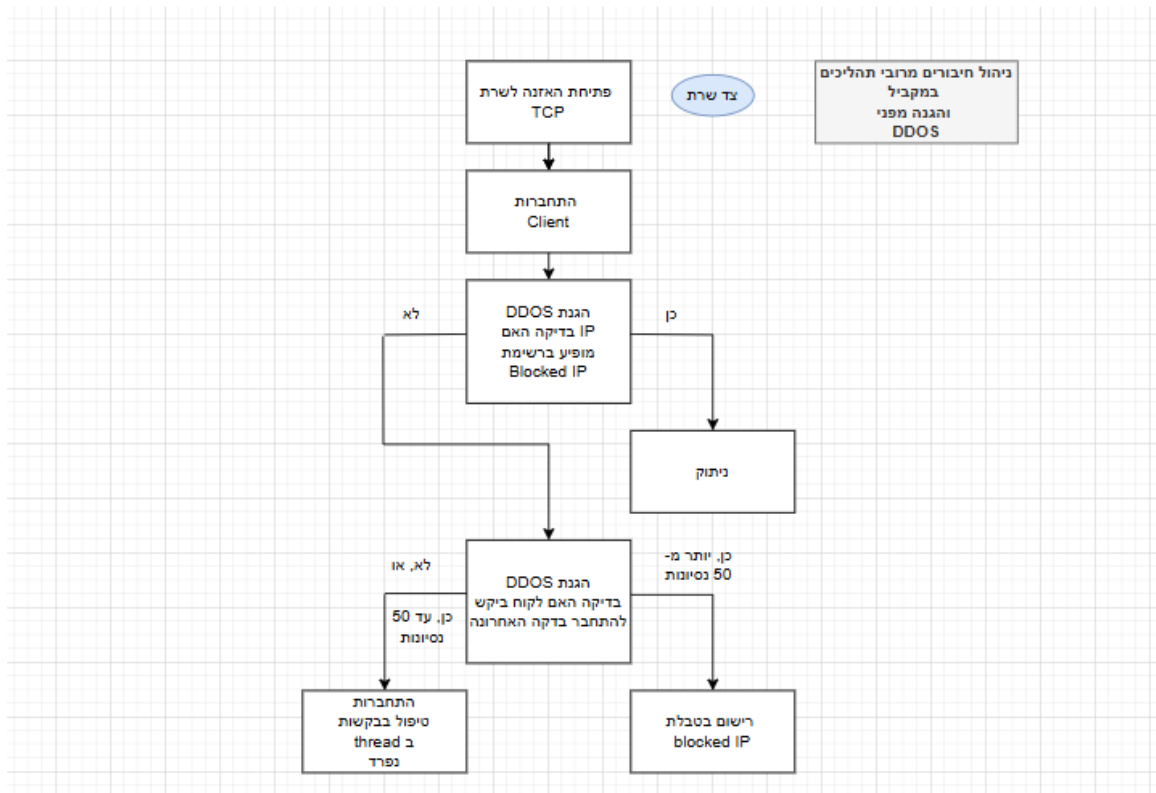
GUI: ניהול מסכי משתמש עם קאסטום טיקינטר.

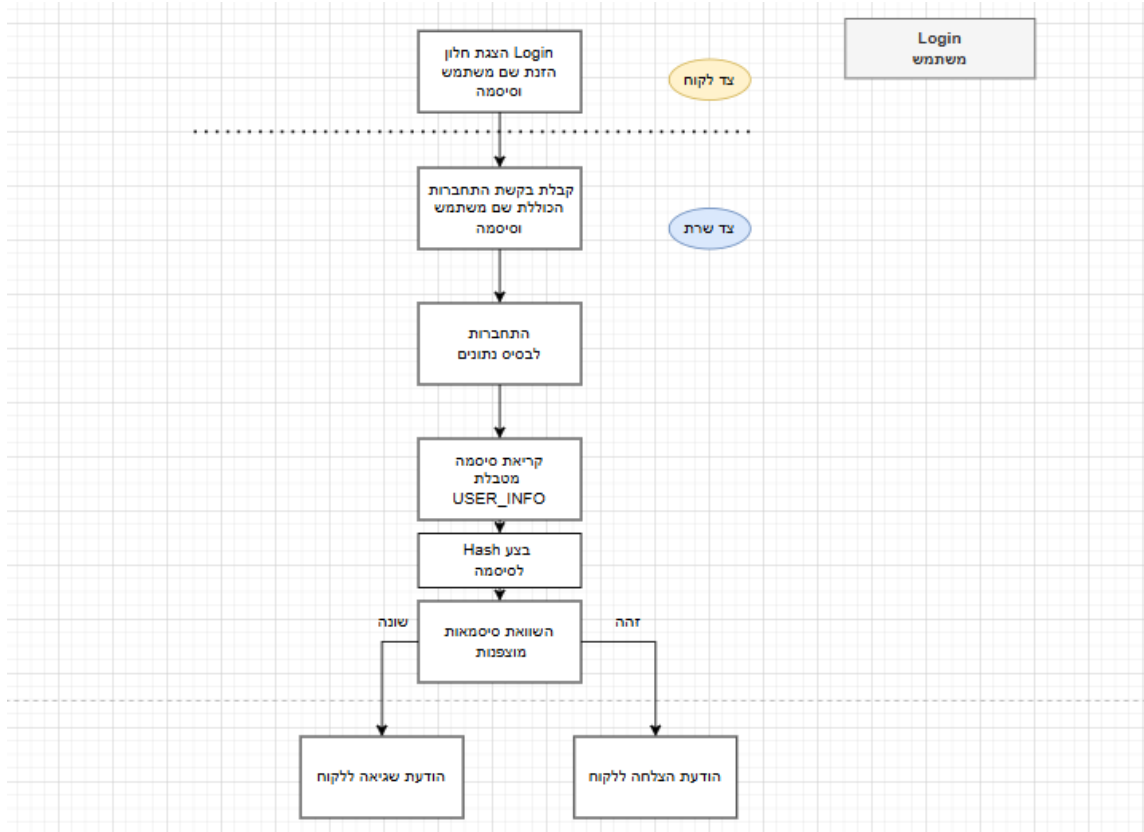
עיבוד קול ושמע: הקלטה ושמירת קבצי WAV עיבוד והשמעה חוזרת והעלאתם.

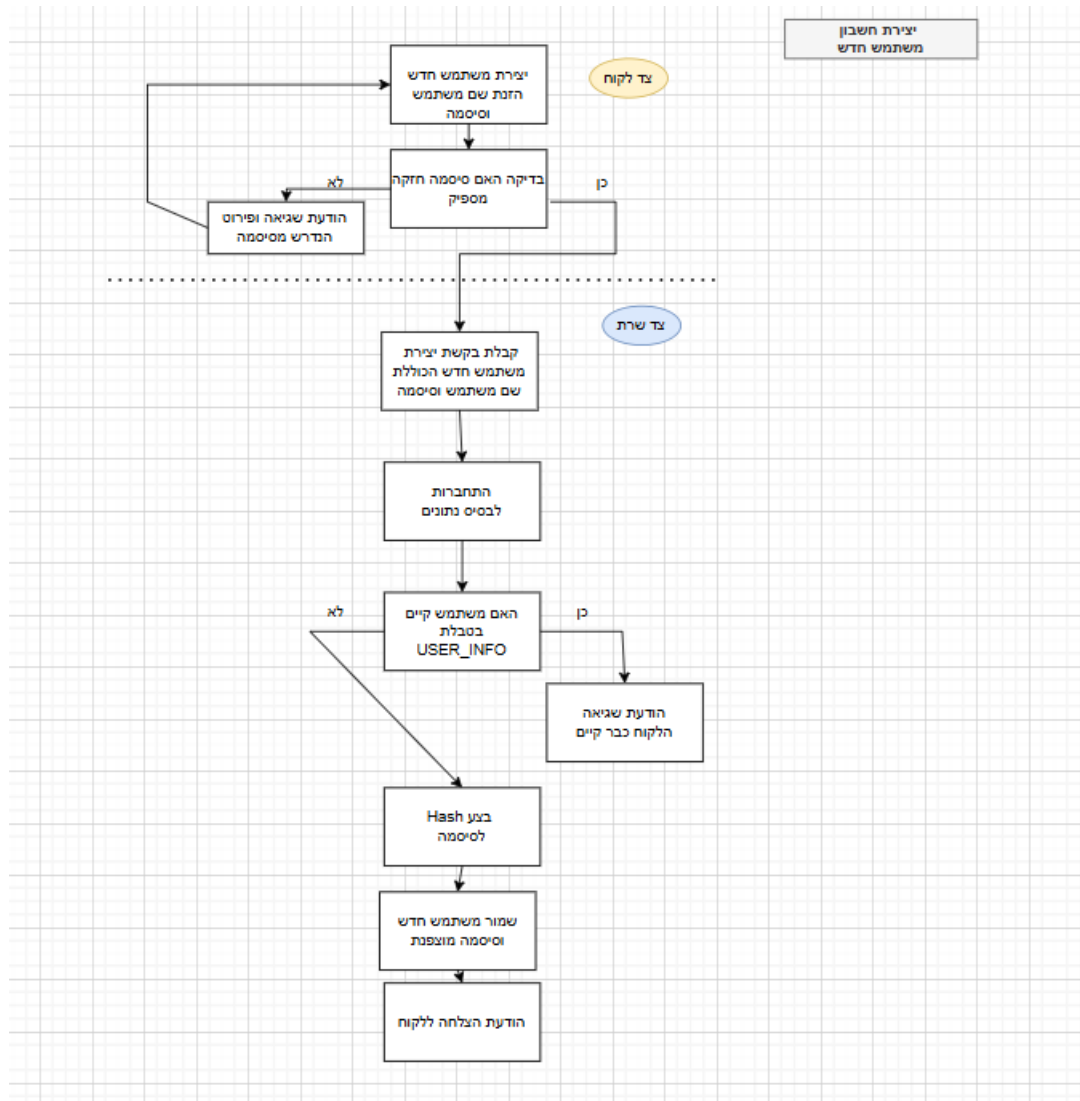
תקשורת רשת: ניהול socket, multithreading וrate limiting להגנה מפני DDOS

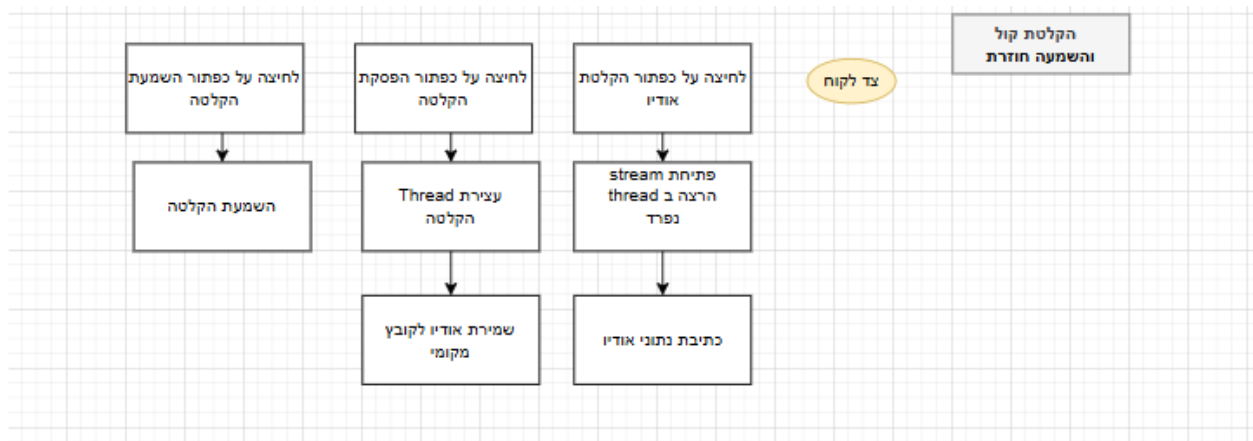
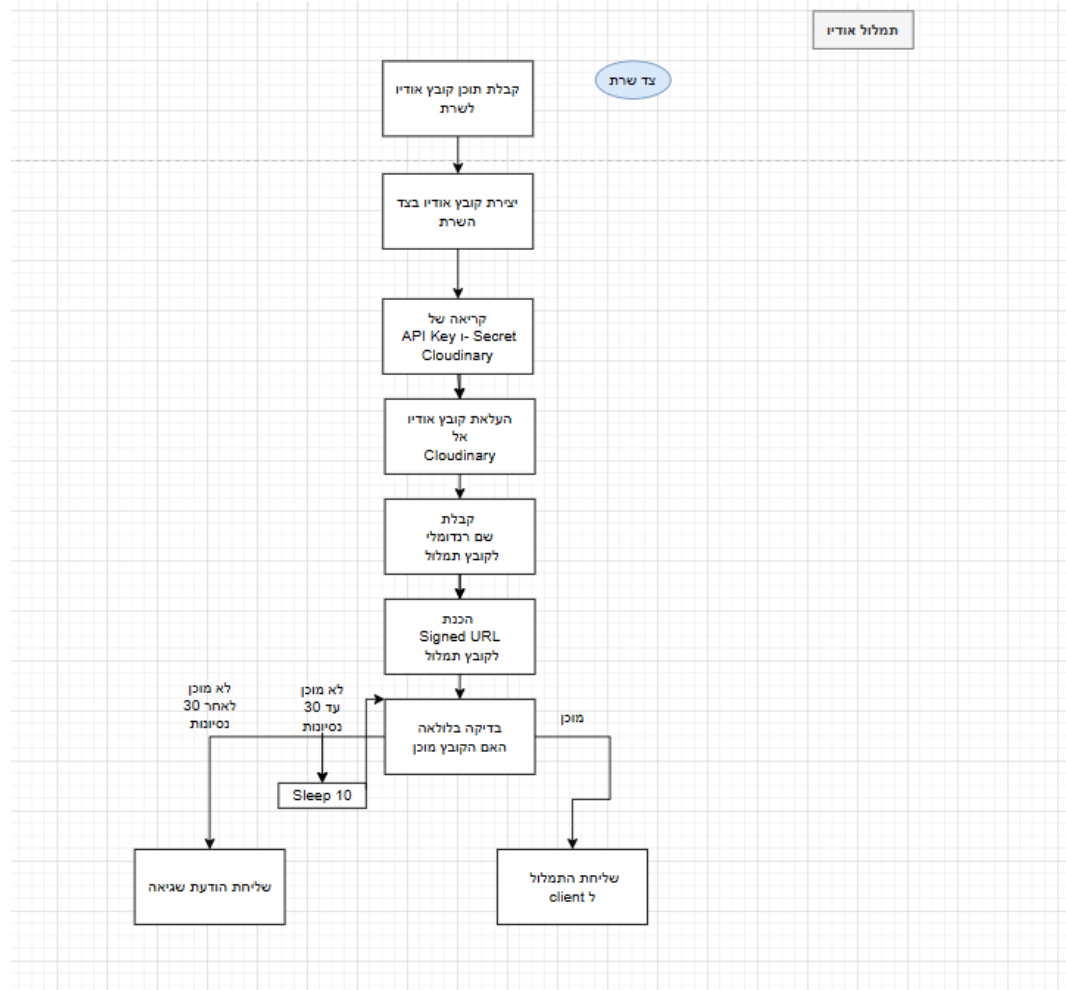
שירותי Cloudinary: להמרת שמע לטקסט עם שימוש בAPI שלהם.

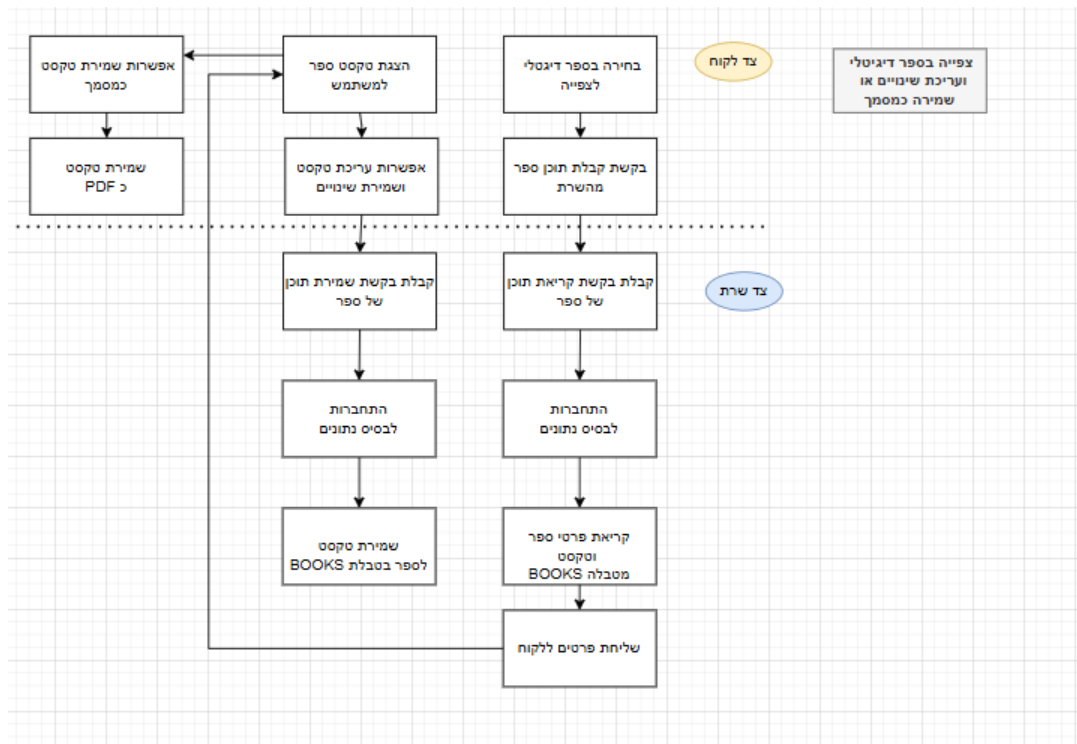
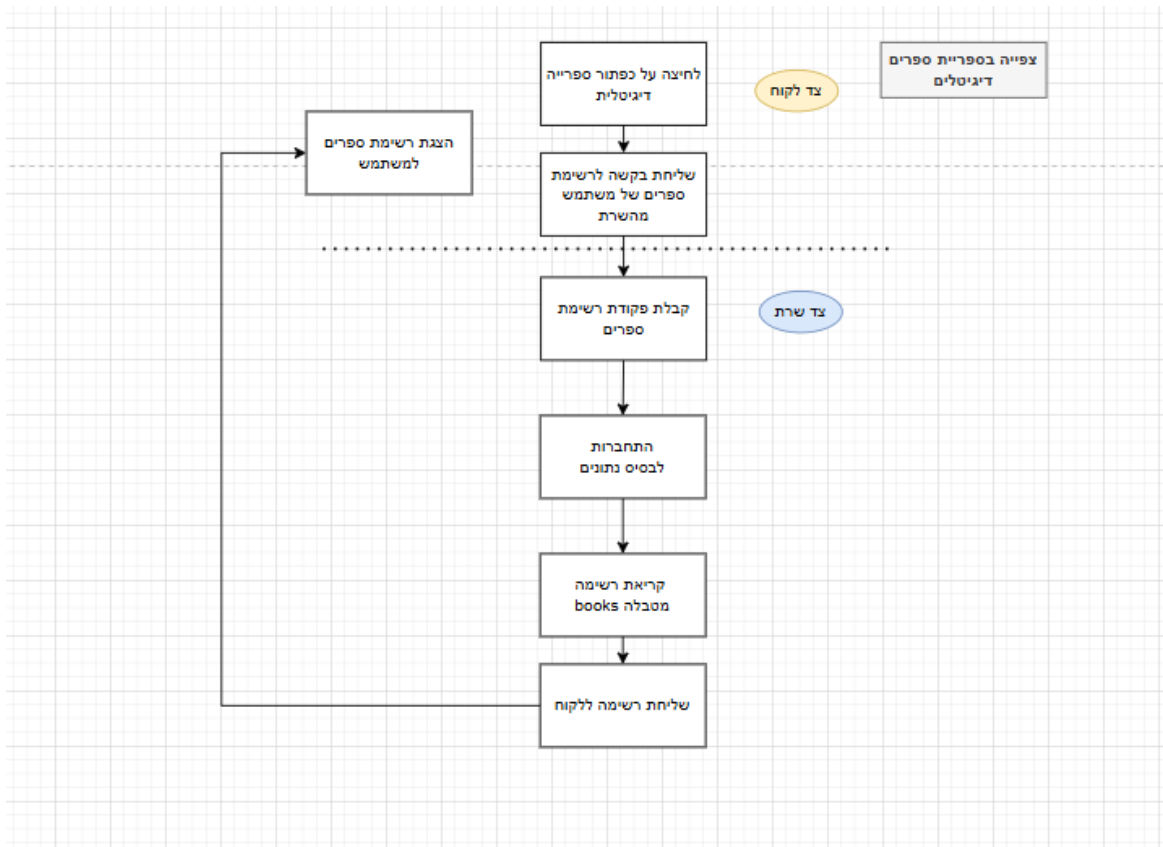
תיאור זרימת המידע במערכת:











תיאור האלגוריתמים מרכזיים בפרויקט:

ניסוח וניתוח של הבעיה האלגוריתמית:

העברת מפתח AES מוצפן בין הלקוח והשרת כדי שיכולו לבצע תקשורת מוצפנת.

ניסוח הבעיה: כיצד להעביר מפתח AES סימטרי לצורך הצפנה מהירה, מבלי לשלוח אותו ישירות ברשת?

אלגוריתמים קיימים:

הפרוטוקול Diffie–Hellman שדורש פרמטרים וחישובים מורכבים שמבצע החלפת מפתח סימטרי ללא שליחתו ישירות.

Elliptic-Curve של Diffie–Hellman שמבוסס על עקומות אליפטיות משתמש בקושי מתמטי דומה אך עם מפתחות קצרים יותר לביצועים טובים יותר.

RSA Key Transport שיטת הצפנת מפתח סימטרי AES על ידי הצפנתו עם מפתח ציבורי של הנמען שיכול לפענח עם המפתח הפרטי המקביל שיש לו.

סקירת הפתרון הנבחר: RSA Key Transport - מצפינים את מפתח ה-AES ב-RSA ושולחים ב-TCP. השרת מפענח ושומר את המפתח, ואז עוברים להמשך עיבוד ב-AES. נשללו הפתרונות האחרים - DH/ECDH עקב רמת המורכבות.

הסבר על RSA: ב Echo אני משתמשת ב RSA Key Transport כדי להעביר בבטחה את מפתח ה AES בין הלקוח לשרת. העקרונות של אלגוריתם ה RSA הם:

בחירת מספרים ראשוניים גדולים: בוחרים שני מספרים ראשוניים p ו q בפועל באורך של מאות ביטים כדי למנוע פיצוח, ככל שהמספרים גדולים יותר, קשה יותר לפרק את המכפלה שלהם.

חישוב המודולו n : מחשבים את המכפלה

$$q \times p = n$$

פונקציית אוילר $\phi(n)$: מכיוון ש p ו q ראשוניים, $\phi(n)$ מייצג את מספר המספרים הקטנים מח שהמכנה המשותף הגדול שלהם עם n הוא 1.

$$\phi(n) = (1 - p) \times (1 - q)$$

בחירת המפתח הציבורי e : בוחרים e כך שיהיה: (בוחרים לרוב ערך קבוע מפני שהוא מייעל את פעולות ההצפנה ועדיין נשאר בטוח)

$$1 < e < \varphi(n) \text{ ו- } \gcd(e, \varphi(n)) = 1.$$

חישוב המפתח הפרטי d : המפתח הפרטי d הוא ההופכי המודולרי של e ביחס ל- $\varphi(n)$ כלומר מספר שמקיים:

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

ניתן לחשב d באמצעות האלגוריתם המורחב של אוקלידס למציאת ההופכי המודולרי.

צורת המפתחות:

מפתח ציבורי (n, e) נמסר לכל השולחים להצפנה.

מפתח פרטי (n, d) נשמר רק אצל המקבל לפענוח.

הצפנה ופענוח: הצפנה של הודעה m - $c = m^e \bmod n$

פענוח של הצופן c - $m = c^d \bmod n$

ב- Echo הלקוח מצפין את מפתח ה- AES הייחודי על ידי העלאתו בחזקה e מודולו n ושולח את התוצאה לשרת. השרת, שמחזיק את d מבצע חזקה ל d על ההצפנה שהתקבלה כדי לשחזר את מפתח ה- AES המשותף לטווח ההצפות הסימטריות הבאות. כך מבטיחים ששליחתו של מפתח ה- AES לאורך הרשת נעשית בצורה בלתי ניתנת לקריאה על ידי מי שאין ברשותו המפתח הפרטי.

מניעת הצפת בקשות - הגנה מ-DDOS

ניסוח הבעיה: כתובת IP שגורמת למספר רב של פניות בתוך זמן קצר עלולה לגרום לשרת לקרוס.

אלגוריתמים קיימים:

ניהול אסימונים Token Bucket מייצר "אסימונים" בקצב קבוע Leaky Bucket, משחרר בקשות בקצב קבוע כדי למנוע הצפה, מורכב לתחזוקה.

Sliding Window Log שמבצעת רישום timestamps לכל בקשה ובדיקה בחלון זמן משתנה, כמות של הזיכרון שנדרשת לצריכה אינה קבועה מראש, אלא משתנה בזמן אמת לפי גודל ועומס הנתונים.

Fixed Window חלוקת זמן לחלונות קבועים וספירת בקשות בכל חלון, פשוט ליישום ויעיל בזיכרון קבוע.

סקירת הפתרון הנבחר Fixed Window Rate Limiting - לכל IP נשמר מילון של timestamps, מנקה ערכים ישנים וסופר פניות. חריגה מהמגבלה חוסמת את ה-IP ל-60 שניות, בחרתי בפתרון זה בזכות הפשטות ויעילות שלו והתחזוקה הקלה, ברור כיצד לשנות את פרמטר החלון והמגבלה במידת הצורך. נשללו הפתרונות האחרים בעקבות המורכבות לניהול אסימונים וצורך בזיכרון "דינמי".

הרשמה וכניסה של משתמשים חדשים או קיימים לאפליקציה

ניסוח הבעיה: לאפשר למשתמש ליצור חשבון חדש עם סיסמה חזקה, ולוודא שהכניסה מתבצעת רק עם פרטי הזדהות תקינים וללא חיבור כפול.

אלגוריתמים קיימים

SQL CRUD לבדיקת ייחודיות וכתיבה.

Hashing לאבטחת סיסמאות.

Session Flag למניעת חיבור כפול.

סקירת הפתרון הנבחר - הרשמה: הלקוח שולח שם וסיסמה מוצפנים ב-AES-GCM השרת מבצע

SELECT לוודא ייחודיות, עושה HASHING לסיסמה ושומר רק את התוצר.

כניסה: הלקוח שולח שוב את הסיסמה, השרת עושה HASHING בשיטה זהה ומשווה ל-hash שבמסד הנתונים, בודק דגל is_logged ולאחר מכן מאשר התחברות, בחרתי לעשות את זה ככה בגלל ששילוב של הצפנת התעבורה עם HASHING חזק מבטיח סודיות ואבטחה כפולה, תוך שימוש בספריות סטנדרטיות בפיתוח. חלופות שנשללו bcrypt מורכב מדי לפרויקט לימודי לבית הספר ואחסון טקסט גולמי לא מאובטח.

הצפנה ואימות של ההודעות AES-GCM

ניסוח הבעיה: לספק גם סודיות של הצפנה וגם אימות שמוודא שלמות בהעברת הודעות.

אלגוריתמים קיימים:

AES-CBC + HMAC שתי פעולות נפרדות, שילוב של מצב CBC להצפנה עם חישוב HMAC נפרד על מנת לעשות אימות לשלמות.

ChaCha20-Poly1305 אלגוריתם הצפנה סימטרית מודרנית, שדורש התקנה חיצונית.

AES-GCM מצב פעולה של AES המשלב CTR להצפנה עם GHASH לאימות (עם TAG) בקריאה אחת.

סקירת הפתרון הנבחר- בחרתי לעשות שימוש ב AES-GCM- מבטיח פענוח ואימות ופשוט להבנה, קריאה אחת קצרה לתפעול, ביצועים גבוהים ומתועד היטב. לעומת הפתרונות האחרים שמסובכים יותר ChaCha20 הוא בכלל חיצוני.

סנכרון GUI ותקשורת עם המולטי קליינטים

ניסוח הבעיה: כיצד אוכל לשמור על ממשק טיקינטר שמוצג לקליינט בצורה חלקה, במקביל לפרוטוקול התקשורת שדורש האזנה רציפה לעוד משתמשים שנכנסים.

אלגוריתמים קיימים:

Asyncio מאפשרת לכתוב פונקציות שכוללות נקודות השהייה וכשהמערכת ממתינה לתשובה היא יכולה לטפל בדברים אחרים עד שהפעולה הסתיימה.

threading + Locks יצירת תהליכים מקבילים בתוך תהליך אחד, עם נעילות.

queue.Queue + Scheduler תור להעברת הודעות בין תהליכים וקריאה מחזורית ללא חסימה.

סקירת הפתרון הנבחר: הרצת ה mainloop-בתהליך הראשי, וקוד התקשורת ב thread-נפרד. העברת אירועים דרך queue.Queue ובדיקת התור, בחרתי בפתרון זה בגלל שהוא פתרון יציב ופשוט שמשלב את האלגוריתמים הקיימים למשהו אידיאלי ושימושי עבור האפליקציה שלי לעומת פתרונות אחרים מסובכים יותר.

תמלול קובץ שמע לטקסט שיוצר אחר כך בספר הדיגיטלי שהמשתמש יצר.

ניסוח הבעיה: לשלוח קובץ WAV ל Cloudinary, לחכות לסיום התמלול האוטומטי ולמשוך בחזרה את קובץ ה JSON עם הטקסט לשימוש בפעולות אחרות באפליקציה.

אלגוריתמים קיימים:

Webhooks מנגנון שבו שירות חיצוני מתקשר בצורה אוטומטית כשהמשימה מוכנה.

Long-Polling החזקת חיבור HTTP פתוח עד שהשרת ישיב בשביל עדכון מיד.

Polling רגיל ביצוע קריאות חוזרות (למשל כל 10 שניות) כדי לבדוק את מצב עדכון.

סקירת הפתרון הנבחר - Polling כל 10 שניות עד 30 ניסיונות, ברגע ש bytes של transcript > 0 מושך את ה JSON בחרתי בפתרון זה בגלל שהוא פשוט ליישום, אינו דורש חיבורים חיצוניים או ממושכים כמו WEBHOOKS ו long polling נשלל גם הוא בגלל שמחזיק חיבור HTTP פתוח לאורך זמן, גורם לעומס ולקושי בניהול חיבורי רשת מרובים כמו שיש לי באפליקציה.

חלוקה לדפים בספר הדיגיטלי שהאפליקציה שלי יוצרת עבור המשתמש עם הקובץ קול שלו

ניסוח הבעיה: הצגת טקסט ארוך בדפי הספר ללא השהיות בגלילה.

אלגוריתמים קיימים:

טעינה מלאה, BATCHLOAD מאפשר טעינת של כל הטקסט בבת אחת לפני ההצגה.

טעינת דף לפי דרישה, טעינת חלק קטן לפי בקשה רק כאשר המשתמש מגיע אליו.

Prefetch Cache טעינת הפריט הנוכחי ואת הפריט הבא מראש בזיכרון, כדי לאפשר תגובה מיידיית בגלילה.

סקירת הפתרון הנבחר - טעינת הדף הנוכחי והדף הבא מראש בזיכרון (Prefetch) כדי לאפשר גלילה חלקה בחרתי בפתרון זה בגלל שהוא מספק לי תגובה מיידיית, שימוש בזיכרון מוגבל מה שיוצר חוויית משתמש טובה. הפתרון של טעינה מלאה עלול לגרום לעיכובים משמעותיים בזיכרון כאשר הטקסט גדול וטעינת דף לפי דרישה יוצרת השהיות גלילה בכל פעם שהמשתמש מדפדף.

תיאור סביבת הפיתוח:
פרוט כלי הפיתוח הדרושים לפיתוח:
שפת תכנות והתקנות-

Python 3.13.3 היא שפת הפיתוח הראשית שהשתמשתי בה, וכתבתי איתה את הקוד בתוך Visual Studio Code.

SQLite Browser כלי לפתיחת קבצי הדאטה בייס ולצפייה במבנה הטבלאות ובתוכן שלהן.

GitHub לניהול גרסאות הקוד וההיסטוריה שלו.

הסביבה והכלים הנדרשים לבדיקות-

Terminal - פקודה למציאת ה IP המקומי ולבדיקת חיבורים בין מכשירים.

Wireshark - תוכנת ניתוח חבילות לבדיקת תעבורת TCP מוצפנת, אימות שאף נתון לא דולף ברשת והרצה בהתאם לפרוטוקול.

Pytest - מסגרת לבדיקות יחידה (unit) של פונקציות קריפטו ועל SQLite.

SQLite Browser - שימוש חוזר כדי לוודא שתוצאות- ההרשמה, כניסה, שמירת ספרים נשמרות ומאוחזרות כמצופה.

תיאור פרוטוקול התקשורת:

תיאור מילולי של פרוטוקול התקשורת:

בפרויקט Echo התקשורת בין הלקוח לשרת מתבצעת על גבי פרוטוקול TCP פרוטוקול Connection Oriented הפועל בשכבת התעבורה אשר מבטיח העברה אמינה באמצעות סדרה של שלושה צעדים: SYN מהלקוח לשרת לפתיחת חיבור.

השרת מקבל את ה-SYN-ACK ושולח בחזרה אישור שהוא קיבל את הבקשה ומוכן לתקשורת.

הלקוח מקבל את ה SYN-ACK ושולח "ACK" אישור שהשרת זמין, וכעת החיבור נפתח רשמית

לאחר מכן, במסגרת הפרויקט מוסיפים לכל חבילת נתונים את TCP HEADER הכותרת מורכבת בין 20 ל-60 בתים – שם נשמרים למשל מספרי פורט ומידע על סדר החבילות, כדי לאפשר ניתוב ולהבטיח הגעה תקינה. כל העברת תוכן של פקודות או תשובות עומדת באבטחה נוספת של AES-GCM

המבנה של ההודעה המוצפנת הוא

12 Nonce בתים אקראיים לאימות בהצפנה יחידים לכל הודעה, למניעת חזרות.

16 Tag בתים, תג אימות שבודק שלא שונו הבתים בזמן ההעברה.

4 Length בתים שקובעים כמה בתים תופס ה ciphertext

Ciphertext גוף ההודעה המוצפן אורכו משתנה לפי גודל ה JSON-הפנימי

כך בכל מקטע שנשלח יש קודם את כותרת ה TCP (20–60 בתים), ואחריה את ה AES-GCM-header (32 בתים) ואז את גוף ההודעה המוצפן. באמצעות שימוש בשילוב הזה מובטח חיבור יציב ונכון מבחינת TCP וגם שמירה על סודיות ושלמות המידע בכל שלב.

פירוט כלל ההודעות הזורמות במערכת:

שם ההודעה	נשלחת מ- אל-	מבנה השדות בהודעה
EncryptedRequest	לקוח אל שרת	Length (4) , Tag (12 בתים) , Nonce (16 בתים) , Ciphertext (JSON) ,
EncryptedResponse	שרת אל לקוח	Length (4) , Tag (12 בתים) , Nonce (16 בתים) , Ciphertext (JSON) ,
AESKeyExchange	לקוח אל שרת	encrypted_aes_key (32 בתים AES מוצפן ב- RSA ciphertext, בגודל 256 בתים)
ServerPublicKey	שרת אל לקוח	pem_key (מחרוזת PEM של המפתח הציבורי RSA, אורכה דינמי לפי גודל המפתח)
SYN	לקוח אל שרת	אין גוף נתונים – מציין התחלת handshake
SYN-ACK	שרת אל לקוח	אין גוף נתונים – אישור קבלת SYN והערכות לפתיחת תקשורת
ACK	לקוח אל שרת	אין גוף נתונים – אישור השלמת תהליך handshake
CreateBookResponse	שרת אל לקוח	JSON: { "status": "...", "book_id": "<מספר>" }
CreateBookRequest	לקוח אל שרת	JSON: { "command": "create_digital_book", "data": { "audio_file" } }
UploadAudioResponse	שרת אל לקוח	JSON: { "status": "...", "transcription": "..." }
UploadAudioRequest	לקוח אל שרת	JSON: { "command": "upload_audio", "data": { "filename" } }
CreateAccountResponse	שרת אל לקוח	JSON: { "status": "...", "message": "..." }

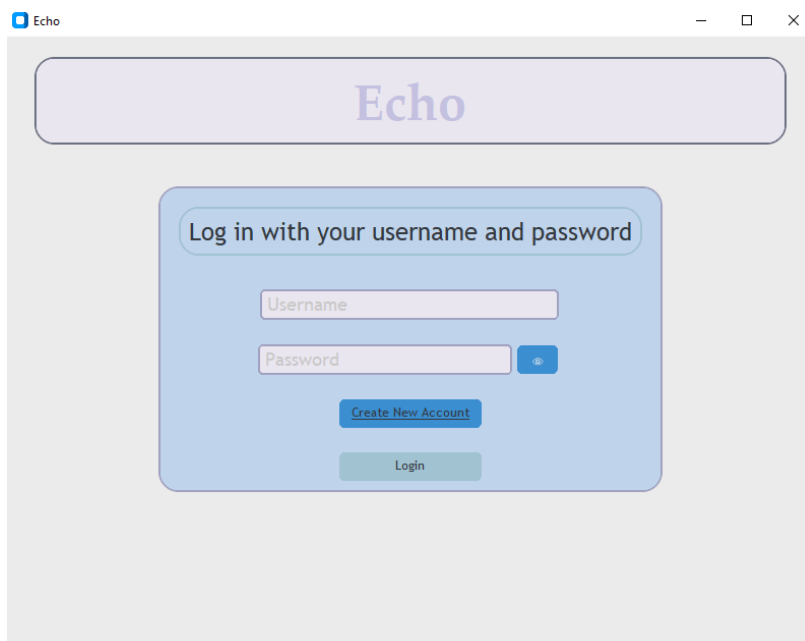
JSON: { "command": "create_account", "data": { "username", "password" } }	לקוח אל שרת	CreateAccountRequest
JSON: { "status": "...", "message": "..." }	שרת אל לקוח	LoginResponse
JSON: { "command": "login", "data": { "username", "password" } }	לקוח אל שרת	LoginRequest

תיאור מסכי המערכת:

מסך הכניסה Login Window

תפקיד: התחברות משתמש קיים למערכת.

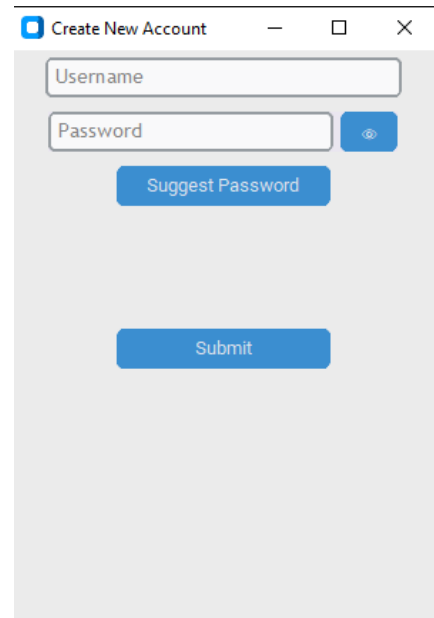
מה מציג ומה ניתן לבצע: מוצגים שני שדות שם משתמש, סיסמה שלידה מוצג אייקון בצורת עין שכאשר הוא נלחץ מאפשר למשתמש לראות את הסיסמא שלו כפי שהיא ולא מוסתרת, כפתור "Login" וכפתור "Create New Account". לחיצה על "Login" שולחת את הפרטים לשרת ומציגה הודעת שגיאה או ממשיכה למסך המבוא.



מסך יצירת החשבון Create Account Window

תפקיד: רישום משתמש חדש.

מה מציג ומה ניתן לבצע: שדה שם משתמש, שדה סיסמה עם אפשרות הצפייה/הסתרה, כפתור "Suggest Password" לגנרטור סיסמה חזקה וכפתור "Submit" - מבצעת בדיקות חוזק, ייחודיות ושמירה ב-SQLite ואז המשתמש החדש נרשם ורשאי להתחבר עם שם המשתמש וסיסמא שבחר.



Create New Account

Username

Password

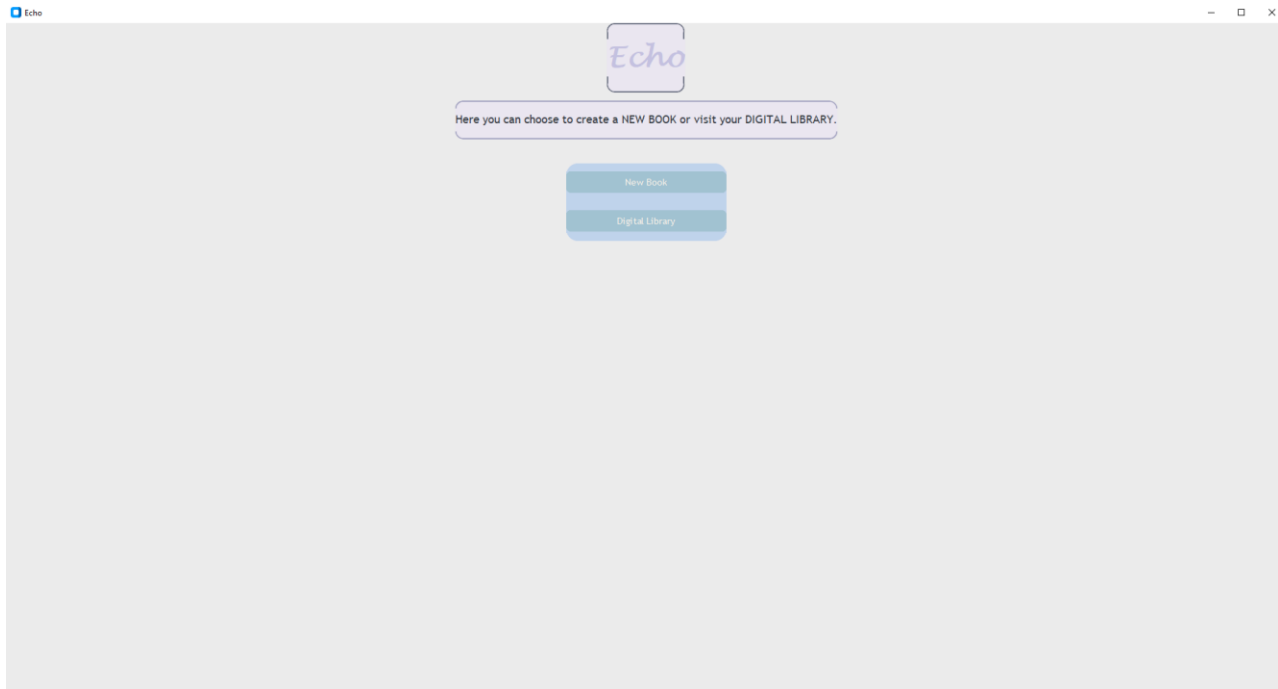
Suggest Password

Submit

מסך הברוכים הבאים Landing Page

תפקיד: בחירת תפקיד ראשי לאחר כניסה.

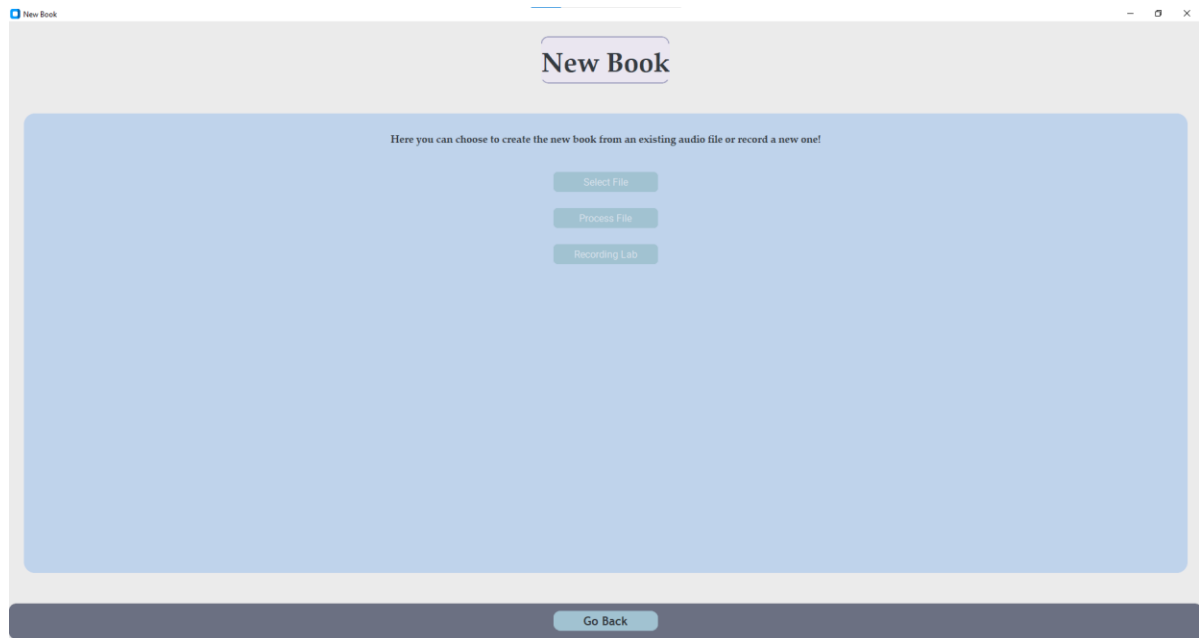
מה מציג ומה ניתן לבצע: מציג הסברים למה ניתן לעשות בחלון זה, שני כפתורים "New Book" שמכוון את המשתמש בעת לחיצה לחלון המותאם ליצירת ספר חדש ו-"Digital Library" כדי להיכנס לספרייה הדיגיטלית שם ניתן לצפות בספרים הקיימים השייכים למשתמש המחובר.



מסך יצירת ספר חדש / New Book / Homepage

תפקיד: התחלת תהליך יצירת ספר דיגיטלי חדש.

מה מציג ומה ניתן לבצע: מציג הסברים למה ניתן לעשות בחלון זה, כפתורים להעלאה לעיבוד של קובץ שמע קיים שיש למשתמש, כפתור למעבר לחלון ההקלטות קבצי קול חדשים, וכפתור חזרה שמחזיר את המשתמש למסך הברוכים הבאים.

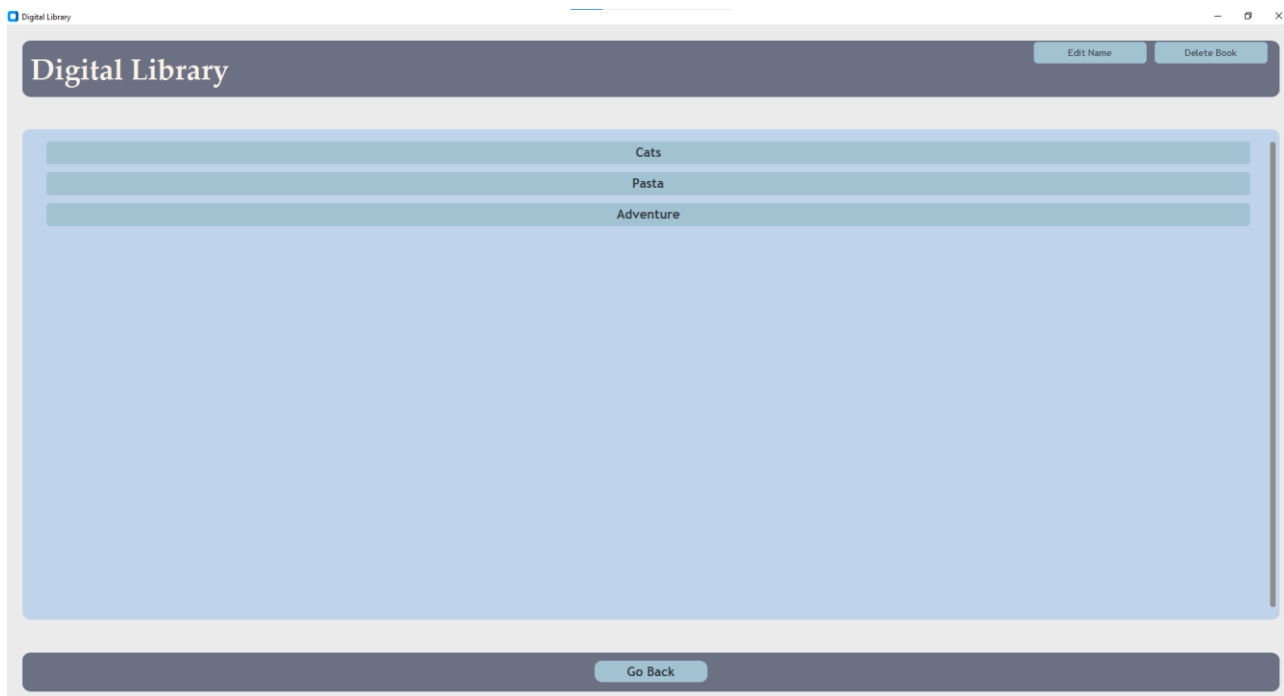


מסך ספריה דיגיטלית Digital Library

תפקיד: עיון וניהול ספרים קיימים שנוצרו על ידי המשתמש עם אפשרות לפתוח ספר נבחר.

מה מציג ומה ניתן לבצע: מציג את רשימת הספרים השייכים למשתמש לפי שמותיהם וכפתורים שונים: לחיצה על שם הספר כדי לפתוח אותו לקריאה, לחיצה על "Edit Name" לשינוי שם הספר הנבחר, לחיצה על "Delete Book" כדי למחוק ספר באופן מוחלט, לחיצה על "Go Back" לחזרה למסך הברוכים הבאים.

*למשתמש שאין לו ספרים תוצג רשימה ריקה עד שיכין ספר



מסך מעבדת הקלטות Recording Lab

תפקיד: לאפשר למשתמש להקליט קבצי קול בלייב על מנת שליחתם לתמלול.

מה מציג ומה ניתן לבצע:

המסך מוקדש לפעולות הקלטה, השמעה ושליחה של קובץ השמע לשרת. ההצגה והכפתורים משתנים בהתאם למצב:

התחלת הקלטה: (Start Recording)

- מוצג כפתור פעיל. "Start Recording"
- לחיצה עליו מפעילה הקלטה במיקרופון ומשנה את מצב הכפתורים כך שרק "Stop Recording" יהיה פעיל.

הקלטה רצה: (Recording...)

- במקום "Start" מוצג תצוגת מצב "Recording..." (בצבע אדום) ללא כפתור נוסף.
- מופעל "Stop Recording" כפתור ירוק.

סיום הקלטה: (Stop Recording)

- לאחר לחיצה על "Stop Recording" הכפתור משתנה ל "Play Recording" לצורך השמעה חוזרת.
- מופעל גם כפתור "Submit" לשליחת הקובץ.

השמעה: (Play Recording)

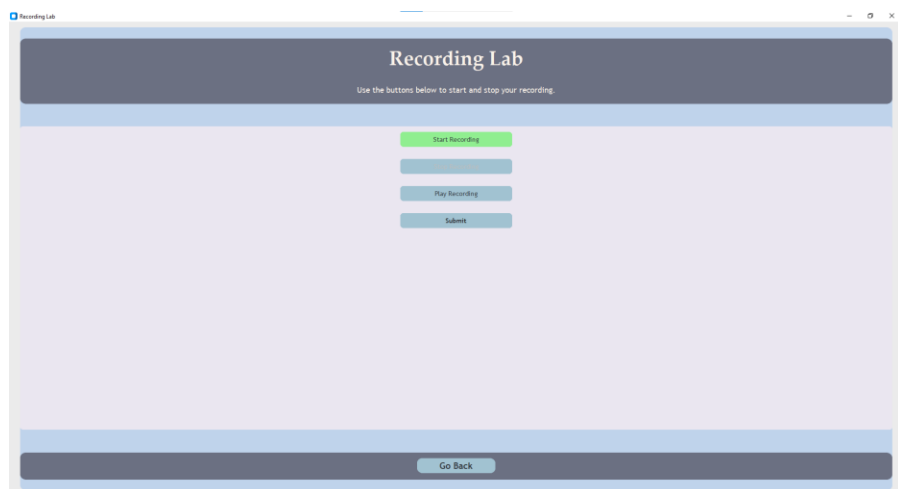
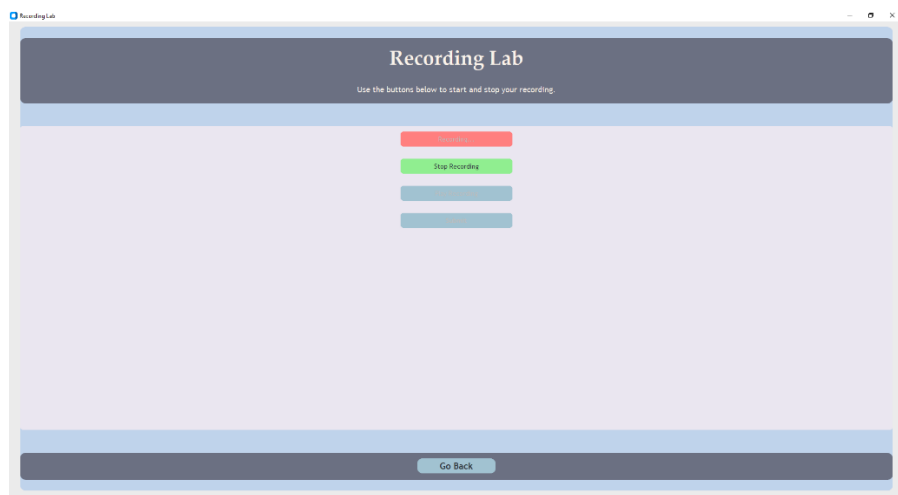
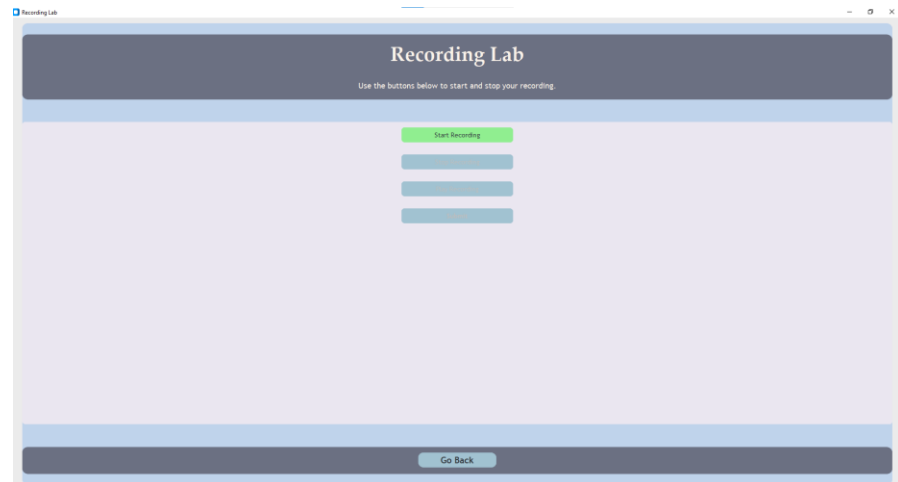
- לחיצה על "Play Recording" משמיעה את הקובץ שהוקלט,
- הכפתור נשאר פעיל עד לסיום ההשמעה.

שליחה: (Submit)

- לחיצה על "Submit" מעבירה את הקובץ לשרת לתמלול.
- בזמן ההעלאה הכפתור נעצר ומופיע סטטוס "Uploading..." או הודעת הצלחה/שגיאה בסיום.
-

כפתור חזרה: (Go Back)

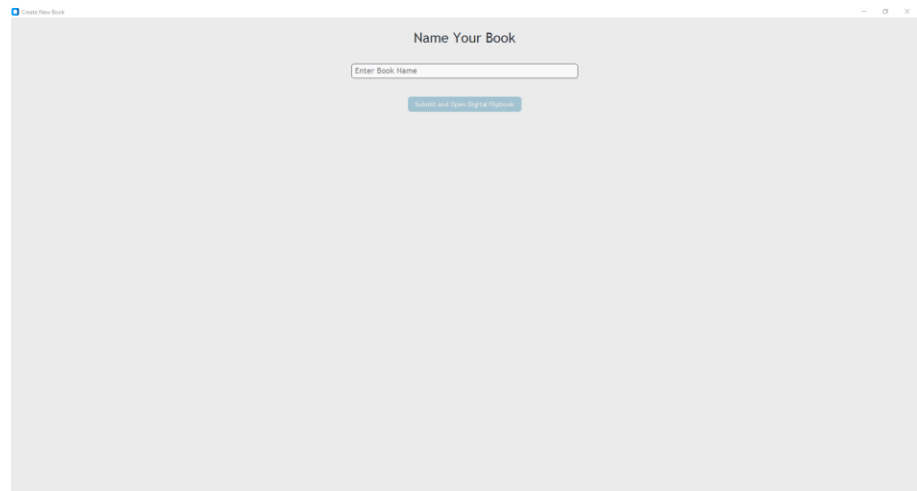
- תמיד זמין בתחתית המסך לחזרה למסך הברוכים הבאים.



מסך מתן שם לספר Book Creation Window

תפקיד: מתן שם סופי לספר לאחר קבלת תמלול.

מה ניתן לבצע: הזנת שם הספר בתיבה, ולחיצה על "Submit and Open Digital Flipbook" לשמירה ופתיחה.



מסך כיסוי הספר Cover Page

תפקיד המסך: הצגת שם הספר ומחברו לפני תחילת הדפדוף.

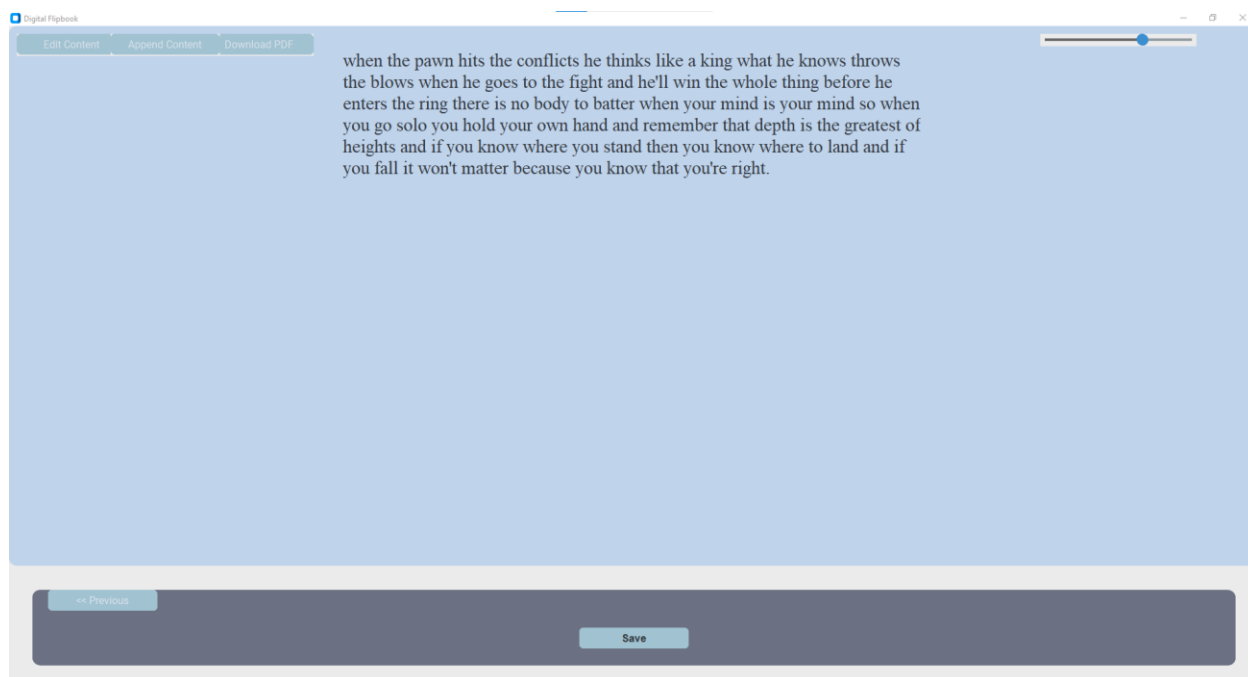
מה מציג ומה ניתן לבצע: כותרת גדולה במרכז המסך שמציגה את שם הספר. שורה מתחת לכותרת: שם הכותב- שהוא שם המשתמש של המשתמש שהספר שלו. לחצן "Save" לשמירת הספר כולל התוכן שלו לספרייה הדיגיטלית של המשתמש, כפתור NEXT למעבר לעמוד הבא של הספר שמכיל את התוכן שתומלל.



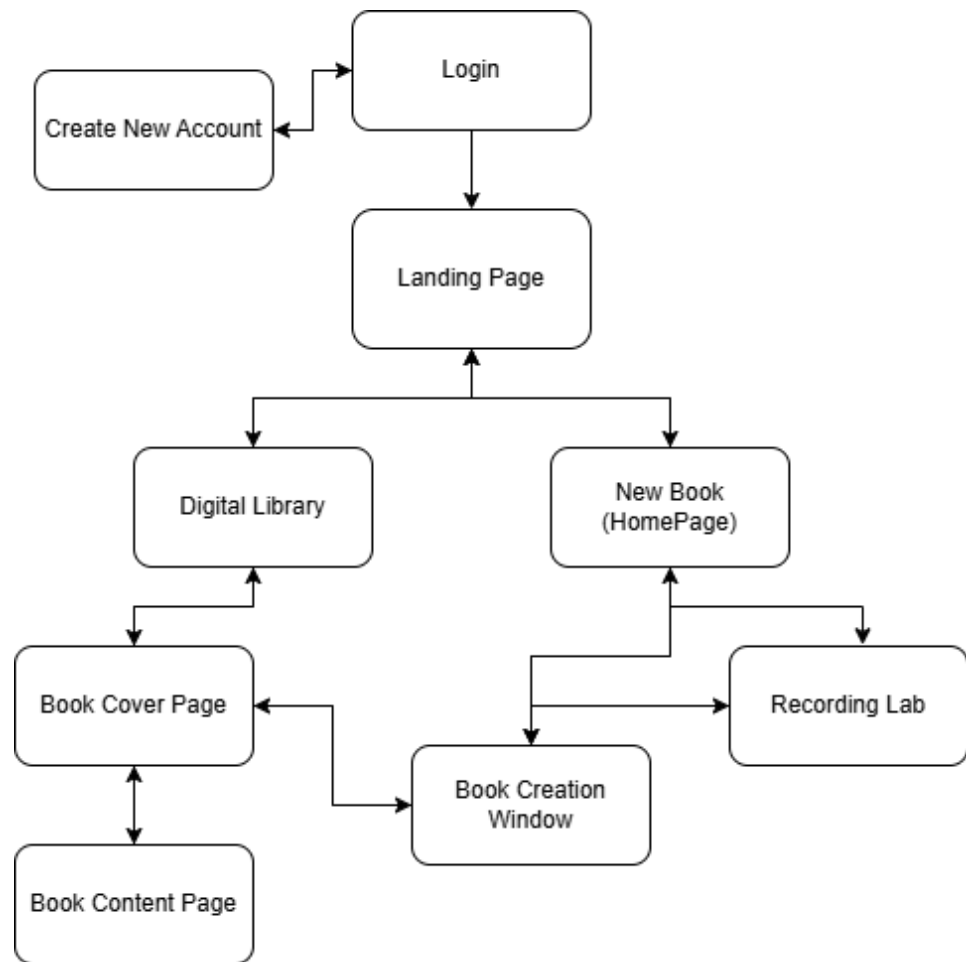
מסך תוכן הספר Content Page

תפקיד: הצגת הטקסט שתומלל מקבצי הקול- תוכן הספר.

מה מציג ומה ניתן לבצע: טקסט התמלול בגוף החלון, עם פס גלילה אנכי לגישה לשאר תוכן הספר- במקרה והספר ארוך ולא ניתן לקרוא את כולו במסך. לחצן "Previous" לחזרה למסך כיסוי הספר, לחצן "Save" לשמירת עדכונים שנעשו בספר לחצן ייצוא הספר המלא כקובץ PDF, לחצני Edit Content Append Content להוספת או עריכת תוכן הספר וגלגלת שמאפשרת למשתמש להגדיל את גודל התוכן בספר לפי רצונו.



תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם:



תיאור מבני הנתונים:

תיאור מבני הנתונים:

מסד הנתונים המקומי SQLite :

הקובץ echo_db.db מאחסן שתי טבלאות עיקריות:

טבלת המשתמשים USER_INFO שבה כל שורה מיוצגת על-ידי מזהה מספרי אוטומטי user_ID, שם משתמש כטקסט user_name וערך הסיסמה כטקסט ארוך לרוב פלט HASHING.

טבלת הספרים BOOKS שבה יש זיהוי מספרי אוטומטי ID שם הספר כמחרוזת טקסט, גוף התמלול כטקסט חופשי ושדה מבני מספרי OWNER שמקשר לשדה user_ID בטבלת המשתמשים.

קבצי שמע מקומיים WAV

כל הקלטה במסך ההקלטות נשמרת בקובץ output.wav גודל הקובץ משתנה בהתאם למשך ההקלטה. אם המשתמש מקליט שוב, קובץ ההקלטה הקודם מתעדכן ומתחלף להקלטה החדשה.

קבצי תמלול עם Cloudinary

ב Cloudinary-נוצרים קבצים עם סיומת transcript. בפורמט JSON המורכב ממערך של עצמים שכל אחד מהם כולל את השדה transcript. גודל הקובץ משתנה לפי אורך ההקלטה אבל המבנה שלהם קבוע.

מבני נתונים בזיכרון RAM

מפתחות קריפטו- מפתח ציבורי בפורמט PEM ומפתח AES ב-RAM

DDoS Tracking - מילוני request_tracker ו-banned_ips המורכבים מטיימסטאמפים וכתובות IP לזיהוי ומניעת הצפות.

פירוט מאגרי המידע של המערכת: בבסיס הנתונים של Echo echo_db שם כל המידע נמצא. יש שתי טבלאות

עיקריות: USER_INFO ו-BOOKS. בטבלת USER_INFO מוגדרים שלושה שדות: user_ID מטיפוס INTEGER

המשמש כמפתח ראשי אוטומטי

user_name מטיפוס לציון שם המשתמש

ו user_password גם הוא מטיפוס TEXT לאחסון הסיסמה.

בטבלת BOOKS יש ארבעה שדות: ID מטיפוס INTEGER NAME מטיפוס TEXT לשם הספר TEXT מטיפוס TEXT

לתמלול המלא OWNER מטיפוס INTEGER המהווה מפתח זר המפנה אל user_ID בטבלת USER_INFO, מה

שמבטיח שכל ספר מקושר למשתמש קיים.

מסד נתונים: שם הקובץ echo_db.db סוג SQLite

טבלה USER_INFO פרטי משתמש

מפתח ראשי user_ID

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
user_ID	מזהה משתמש אוטומטי	INTEGER	1	PRIMARY KEY AUTOINCREMENT, NOT NULL
user_name	שם המשתמש (טקסט)	TEXT	Fiona	UNIQUE, NOT NULL
user_password	סיסמה (HASH)	TEXT	5e884898da280...	NOT NULL

טבלה BOOKS ספרים דיגיטליים

מפתח ראשי ID

שם השדה	תיאור	טיפוס השדה	דוגמאות	מאפיינים
ID	מזהה ייחודי של הספר	INTEGER	17	PRIMARY KEY AUTOINCREMENT, NOT NULL
NAME	שם הספר כפי שהוזן על ידי המשתמש	TEXT	My First Echo Book	NOT NULL
TEXT	תוכן התמלול המלא של הספר	TEXT	This is the transcript...	NOT NULL
OWNER	המשתמש שיצר את הספר, מתייחס לUSER_INFO.user_ID	INTEGER	1	NOT NULL, FOREIGN KEY → USER_INFO.user_ID

סקירת חולשות והאיומים:

שכבת האפליקציה:

עבודה עם בסיס נתונים - SQL INJECTION

האיום: ניסיונות לשבץ קוד SQL בשדות הקלט כדי לשנות או לגנוב נתונים.
התמודדות: כל קריאה ל-DB משתמשת ב-"?" במקום שרשור מחרוזות, כך שהקלט מטופל כערך בלבד.
מבצעים בדיקות פשוטות על המלל לשם וולידציה לפני שליחתו, כדי לאפשר רק תווים מותרים למשל אותיות ומספרים בשמות.

עבודה עם אתרי web

האיום: התקפות מבוססות דפדפן כגון XSS בה קורה ניצול חור באתרים בו ניתן להזריק קוד JavaScript בתוך הדף עצמו או CSRF שהיא התקפת הונאה שבה תוקף גורם לדפדפן של משתמש מחובר לבצע בקשה לאתר היעד בלי ידיעתו.

התמודדות: אין במערכת ממשק HTTP או דפי אינטרנט מכיוון כל התקשורת נעשית דרך סוקט TCP פנימי, כך שהאיומים הללו אינם קיימים.

תהליך ה login אימות ווידוא

האיום: פיצוח סיסמאות ב brute force או שימוש בסיסמאות שנדלפו.
התמודדות: סיסמאות נשמרות ב-DB כערך עם HASHING כך שגם אם יש גישה לטבלה אי אפשר לשחזר סיסמה נקייה.
אחרי מספר מוגבל של ניסיונות כושלים (5), חוסמים את שם המשתמש לכמה עשרות שניות.

ההגנה מפני HITM

האיום: האזנה או שינוי מסרים בין הלקוח לשרת.
התמודדות: בשלב ההתחלתי הלקוח מצפין את מפתח ה-AES בעזרת מפתח ציבורי RSA לאחר מכן כל התקשורת מועברת בהצפנת AES-GCM, שמוודאת גם שלא בוצעו שינויים בתוכן.

מניעת DDOS

האיום: קריסת השרת עקב עומס יתר.
התמודדות: מוציאים מגבלת קצב לכל כתובת IP (למשל עד 50 בקשות בדקה), ומפעילים חסימה קצרה לאחר חריגה, מגדירים טיימאאוט בחיבורי הסוקט, כדי שהחיבורים הלא פעילים ישוחררו במהירות ולא ייתקעו במערכת.

העלאת קבצים

האיום: קבצים זדוניים שעלולים לשבש את השרת.
התמודדות: מאשרים רק קבצים מסוג WAV

שכבת התעבורה:

פרוטוקול TCP ולחיצת היד המשולשת

האיום: תוקף עלול לנסות לבצע SYN FLOOD על ידי שליחת המון הודעות SYN ללא השלמת ה-ACK מה שיסתום את תורי ההמתנה בשרת וימנע מלקוחות לגיטימיים להתחבר.

התמודדות: הגבלת עומס, SYN השרת מקבע גודל לתור קבלת החיבורים ומסיר תורים ישנים בהתאם לגבול. Timeout קצר שקובע שאם לקוח לא משלים את השלבים בתוך שבריר שנייה, החיבור נסגר אוטומטית.

הצפנה

האיום: האזנה sniffing שמהווה תהליך שבו תוקף מצמיד את עצמו לתעבורת השרת כדי לקרוא ולהעתיק מנות נתונים שעוברות בין שני צדדים או tampering של מנות ברשת שהיא פעולה שבה התוקף מתערב בתעבורת השרת בזמן אמת ומשנה את תוכן ההודעות או הנתונים.

התמודדות: RSA key exchange כלומר החלפת מפתח AES ייחודי מוצפן במפתח הציבורי של השרת. כל התקשורת אחרי החלפת המפתח מוצפנת בהצפנה סימטרית עם אימות TAG שבודק שהמסר לא שונה, במידה שיש ניסיון לשינוי כלשהו הפענוח נכשל מיד וההודעה מתויגת כשגויה.

הפעלת המערכת

אילו חולשות קיימות

האיום: קבצים או תלויות חסרים, גרסאות ספריות לא תואמות, היעדר קובץ DB או מפתחות.

התמודדות: יצירת DB אוטומטית echo_db.db אם אינו קיים, כולל יצירת טבלאות USER_INFO ו-BOOKS. ייצור (RSA Keys) בריצה עצמית: אין תלות בקבצים חיצוניים.

בדיקות ראשוניות עם ניסיון התחברות ל-TCP פתיחת החיבור ל-SQLite ובדיקת תקינות טבלאות. במקרה של כשל, תוצג הודעת שגיאה וסיום הרצה.

SQL INJECTION

האיום: הכנסת פקודות SQL בשדות קלט (שם משתמש, שם ספר) כדי לשנות או לגנוב מידע.

התמודדות: שימוש בשאילתות עם פרמטרים בכל הפניות ל-SQLite כך שהקלט מטופל כערך בלבד. וולידציה ראשונית על שדות טקסט בתחומי שמות, בשדות סיסמה

ניהול הרשאות ואימות משתמש

האיום: ניסיון force-brute השגת AES או התחזות.

התמודדות: סיסמאות נשמרות ב-HASHING בלבד.

הגבלת ניסיונות כושלים – חסימה אוטומטית אחרי 5 כישלונות למשך 30 שניות.

הזרמת קבצי אודיו

האיום: העלאת קבצים זדוניים.

התמודדות: האפליקציה מקבלת רק קבצי WAV.

מימוש הפרויקט:**חלק א'- סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם****מודולים /מחלקות מיובאים:**

שם	למה מיועד
os	ניהול קבצים ותיקיות- בדיקת קיום, יצירה, מחיקה וניווט נתיבים
socket	פתיחת חיבור TCP ושליחת וקבלת בייתים בין שרת ללקוחות
select	בדיקת מוכנות קריאה וכתיבה על גבי סוקטים ללא חסימה
threading	הפעלת מספר תהליכים במקביל
time	השהיה ושעון למדידת זמני המתנה למשל בלולאת בדיקת התמלול
datetime	עבודה עם תאריכים ושעות למשל חותמות זמן ולוגים
json	המרה בין מחרוזות JSON לאובייקטים Python ולהיפך
struct	אריזה ופירוק של ערכים בינאריים למשל שדה אורך בכותרת הודעה
rsa	הצפנה ופענוח אסימטרי של מפתח AES בעזרת מפתחות RSA
Crypto.Cipher.AES	הצפנה ופענוח סימטרי באמצעות אלגוריתם-AES GCM
Crypto.Random.get_random_bytes	יצירת מפתחות nonce אקראיים להצפנת AES
sqlite3	גישה לבסיס נתונים מקומי SQLite קריאה וכתיבה לטבלאות
cloudinary	ממשק כללי לשירות Cloudinary לניהול מדיה בענן
cloudinary.uploader	העלאת קבצי שמע ל Cloudinary לקבלת תמלול בצורה אוטומטית
cloudinary.api	בדיקה סטטוס קובץ תמלול ב Cloudinary לפני הורדה

API Cloudinary של מפתחות מאובטח של המערכת ממאגר המפתחות של המערכת	keyring
שליפת תמלול JSON מאוחסן מה Cloudinary דרך URL מאובטח	urllib.request.urlopen
הרחבה של tkinter עם GUI מודרני ורכיבים מעוצבים	customtkinter
הצגת תיבות דיאלוג כמו אזהרות, שגיאות, הודעות מידע ל-GUI	tkinter.messagebox
בחירת קבצים דרך ממשק משתמש Dialog	tkinter.filedialog
דיאלוג קלט טקסט חופשי מהמשתמש	tkinter.simpledialog
סידור אובייקטים של Python לבייטים ולהיפך	pickle
יצירת ערכים אקראיים למשל סיסמאות מוצעות	random, string
קריאה וכתיבה של קבצי שמע בפורמט WAV	wave
ממשק להקלטה והשמעת קול בזמן אמת	pyaudio
טעינה וניגון קבצי שמע	pydub.AudioSegment, pydub.playback.play
יצירת והורדת קבצי PDF	reportlab.lib.pagesizes, reportlab.pdfgen.canvas
חלוקת פסקאות לשורות באורך קבוע בעת יצירת PDF	textwrap

מודולים/מחלקות שאני פיתחתי:

שם המחלקה: AESComm

תפקיד המחלקה: בסיס לשידור הודעות מוצפנות ב AES-GCM על גבי socket

שם תכונה	תפקיד ושימוש
client_socket	ה socket שבאמצעותו נשלחים ומתקבלים הנתונים המוצפנים.
aes_key	מפתח AES המשמש להצפנה ולפענוח של ההודעות.

שם הפעולה	טענת כניסה	טענת יציאה
recv_exact	num_bytes: int	קוראת num_bytes מתבנים מה socket ומחזירה אותם כ־ bytes זורקת ConnectionError אם הסוקט נסגר
recv_header	—	קוראת header המאגד nonce, tag ואורך ciphertext, ומחזירה אותו כ־ bytes
encrypt_payload	request: str binary = False	מייצרת nonce, מצפינה טקסט ב־ AES-GCM, מייצרת payload ומחזירה אותו כ־ bytes משתנה בוליאני בשם binary עם ערך ברירת מחדל שלילי, לשימוש עבור העברת תוכן של קובץ באופן בינארי
decrypt_payload	header: bytes binary = False	מפענחת header, קוראת את ה־ ciphertext בהתאם לאורך שב־ header, בודקת תקינות tag, ומחזירה את המחרוזת המופענחת משתנה בוליאני בשם binary עם ערך ברירת מחדל שלילי, לשימוש עבור העברת תוכן של קובץ באופן בינארי
recv_message	—	קוראת header ו־ ciphertext, מפענחת, ומחזירה את הטקסט המלא כ־ str במקרה של שגיאה מחזירה JSON error כ־ str
send_message	message: str	מקבלת מחרוזת, קוראת ל encrypt_payload, ושולחת את ה payload אל ה socket במקרה של שגיאה מחזירה JSON error dict
recv_binary	—	קוראת header ו־ ciphertext, מפענחת, ומחזירה את המידע הבינארי המלא במקרה של שגיאה מחזירה JSON error כ־ str
send_binary	data	מקבלת תוכן בינארי, קוראת ל encrypt_payload, ושולחת את ה payload אל ה socket במקרה של שגיאה מחזירה JSON error dict

שם המחלקה: AESCommClient

תפקיד המחלקה: בצד הלקוח יוצר מפתח AES אקראי, מצפין ושולח אותו ב, RSA ומשתמש בו להצפנת פקודות.

שם תכונה	תפקיד ושימוש
aes_key	מפתח AES אקראי שנוצר לשימוש במפגש העבודה הנוכחי בין הלקוח לשרת- טווח הזמן שבו האפליקציה פתוחה וה-AES key פעיל (יורשת גם את client_socket ו-aes_key של AESComm)

שם הפעולה	טענת כניסה	טענת יציאה
__init__	socket	יוצר AES key אקראי, קורא ל- super עם ה- socket והמפתח
send_aes_key	server_public_key: rsa.PublicKey	מצפין את aes_key במפתח הציבורי של השרת, שולח לשרת באמצעות socket מדפיס סטטוס
send_client_command	command: str, data: Any	יוצר JSON משני פרמטרי command ו-data, קורא ל- send_message להצפנה ושליחה מחזיר JSON error במקרה של שגיאה
Send_file	file	פותח קובץ אודיו מקומי ושולח את התוכן לשרת

שם המחלקה: AESCommServer

תפקיד המחלקה: בצד השרת מקבלת את מפתח ה-AES המוצפן, מפענחת אותו, ומשתמשת בו להצפנת התגובות.

שם תכונה	תפקיד ושימוש
aes_key	המפתח הסימטרי שמפוענח ב, RSA משמש לפענוח בקשות והצפנת תגובות. (יורשת גם את client_socket של AESComm)

שם הפעולה	טענת כניסה	טענת יציאה

מקבל dict ממיר ל- JSON שולח ללקוח באמצעות exception של dict error במקרה של send_message מחזיר	response: dict	send_server_response
--	-------------------	----------------------

שם המחלקה: LoginWindow

תפקיד המחלקה: ממשק GUI למסך התחברות ויצירת חשבון, כולל חסימת ניסיונות התחברות מרובים.

שם תכונה	תפקיד ושימוש
client	מופע EchoClient לשיגור בקשות login ו create_account.
global_failed_attempts	סופר את הניסיונות כניסה הכושלים למניעת התקפות.
blocked_until	זמן timestamp עד שננעלת האפשרות לניסיון התחברות נוסף.
show_password	בוליאני לקביעת הצגת או הסתרת תווי הסיסמה בשדה.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	client: EchoClient	אתחול החלון וה attributes הצגת GUI ראשוני
create_widgets	—	בונה את כל ה widgets
toggle_password_visibility	—	מחליף בין תצוגת סיסמה מוסתרת/גלויה ומעדכן את כפתור העין
user_log_in	—	לוקח את username/password מהשדות, שולח לשרת דרך EchoClient.send_request מטפל ב- status, חוסם אחרי 5 כשלונות
create_new_account_window	—	פותח חלון חדש ליצירת חשבון עם שדות username/password, callback ו submit_new_account
Add_random_suffix	filename	מייצר תוספת רנדומלית לשם של הקובץ שנוצר בצד השרת כדי שכל קובץ יהיה בשם שונה
Recv_file	file	פותח קובץ אודיו מקומי ומקבל את התוכן מה- Client

שם המחלקה: LandingPage

תפקיד המחלקה: GUI ראשי לאחר התחברות, מאפשר ניווט ל New Book או ל Digital Library.

שם תכונה	תפקיד ושימוש
----------	--------------

client	מופע EchoClient לתקשורת עם השרת.
username	שם המשתמש המחובר להעברת בעלות על קריאות לממשק.
home_window	אובייקט החלון הראשי (CTK) להצגה על המסך.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	client: EchoClient, username: str	אתחול החלון הראשי, קביעת client/username, והצגת מסך ראשי
create_widgets	—	בונה כפתורי ניווט ל Digital Library ו New Book
open_new_book	—	מחביא את החלון הנוכחי ויוצר מופע של Homepage ומפעיל אותו
open_digital_library	—	מחביא את החלון הנוכחי ויוצר מופע של DigitalLibrary עם callback לחזרה
run	—	מפעיל את ה- mainloop() של החלון הראשי

שם המחלקה: Homepage

תפקיד המחלקה: GUI למסך "New Book" שמאפשר בחירת/הקלטת שמע ושליחתו לשרת לתמלול והפיכה לספר דיגיטלי.

שם תכונה	תפקיד ושימוש
selected_audio_file	נתיב קובץ ה WAV שנבחר, או None אם לא נבחר קובץ.
newbook_window	אובייקט הטופ לבל של CTK המציג את המסך.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	client, username: str, go_back_callback: Callable	אתחול החלון, שמירת client/username/callback, יצירת GUI
create_widgets	—	בונה את כל ה widgets
select_file	—	פותח דיאלוג בחירת קובץ WAV, שומר את הנתיב ב selected_audio_file, משנה טקסט וצבע כפתור

submit_audio_file	—	שולח פקודת create_digital_book עם הנתבי לשרת, מטפל בתשובה, פותח BookCreationWindow במידת הצורך
recording_lab	—	מחביא את החלון ומפעיל את RecordingLab עם callback
go_back	—	סוגר את החלון הנוכחי וקורא go_back_callback ל
run	—	מפעיל את הלולאה הראשית של ה Toplevel (mainloop())

שם המחלקה: RecordingLab

תפקיד המחלקה: GUI למעבדה להקלטת קול, ניגון ושליחת ההקלטה לשרת.

שם תכונה	תפקיד ושימוש
recording_window	אובייקט הטופ לבל של CTK של חלון ההקלטות.
recorder	מופע פנימי של RecordAudio לניהול הלוגיקה של ההקלטה.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	parent_window, go_back_callback, client: EchoClient, username	אתחול החלון, יצירת GUI, שמירת parent_window, callback, client, username ואתחול recorder
create_widgets	—	בונה את כל הכפתורים
start_recording	—	קורא ל recorder.start_recording(), משנה סטטוס כפתורים
stop_recording	—	קורא ל recorder.stop_recording() משנה סטטוס כפתורים
play_recording	—	קורא ל pydub להפעיל את הקובץ output.wav
submit_recording	—	שולח את הקובץ לשרת בפקודת create_digital_book מקבל תמלול, פותח BookCreationWindow על פי התשובה
go_back	—	סוגר את החלון ומפעיל את callback

שם המחלקה: RecordAudio

תפקיד המחלקה: ניהול הקלטה ברקע עם PyAudio ושמירתה כקובץ WAV

שם תכונה	תפקיד ושימוש
is_recording	בוליאני המציין אם ההקלטה פעילה.
stream	אובייקט PyAudio Stream לקריאת נתוני קול בזמן אמת.
frames	רשימת הבתים שנקלטו לאחסון בקובץ WAV בסיום ההקלטה.

שם הפעולה	טענת כניסה	טענת יציאה
start_recording	—	מגדיר PyAudio, פותח stream, מאתחל frames, ויוצר thread שקורא בלולאה נתוני אודיו ל-frames
record	—	רץ בthread קורא כל chunk נתונים מה stream ומוסיף לframes
stop_recording	—	עוצר את ה thread, סוגר את stream, קורא לsave_recording
save_recording	—	כותב את התוכן של frames אל קובץ WAV בשם output.wav

שם המחלקה: BookCreationWindow

תפקיד המחלקה: GUI להזנת שם הספר, שמירתו בדאטה בייס ופתיחת הספר.

שם תכונה	תפקיד ושימוש
book_name_entry	שדה להזנת שם הספר שישמר בטבלת BOOKS.
transcription_text	הטקסט שהתקבל מהשרת אחרי תמלול האודיו.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	parent_window, username: str, transcription_text: str	parent_window, שמירת, GUI יצירת, username, transcription_text
submit_book	—	בודקת שם ספר, מוסיפה רשומה לטבלת BOOKS עם name, text, owner, מחזירה book_id ופותחת Flipbook
run	—	מפעילה את הלולאה הראשית של ה־ Toplevel (mainloop())

שם המחלקה: DigitalLibrary

תפקיד המחלקה: GUI להצגת ספרים קיימים, עריכת השם ומחיקתם.

שם תכונה	תפקיד ושימוש
pending_action	מצב הפעולה שנבחר ("edit" או "delete").
library_window	אובייקט הטופ לבל של CTK המציג את מסך הספרייה.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	parent_window, go_back_callback, username: str	parent_window, שמירת, אתחול החלון, go_back_callback, username, GU יצירת
initialize_window	—	בונה את הכותרת, כפתורי פעולה content_frame, ומפעיל display_books
display_books	—	מנקה את ה content_frame, קורא ל fetch_books, יוצר כפתור לכל ספר
fetch_books	—	פותח חיבור DB דרך השרת, שולף רשימת (ID, NAME) של ספרים של המשתמש, מחזיר list[tuple]

create_clickable_book	parent, book_id: int, book_name: str	יוצר כפתור עם book_id וbook_name שמפעיל handle_book_click
handle_book_click	book_id: int, book_name: str	בודק pending_action ומבצע open_book/edit_book_name/confirm_delete, מאפס פעולה ומעדכן GUI
activate_edit_mode	—	משנה pending_action ל"edit", מעדכן צבע כפתור והוראות
activate_delete_mode	—	משנה pending_action ל"delete", מעדכן צבע כפתור והוראות
edit_book_name	book_id: int	פותח dialog לשינוי שם, מעדכן את BOOKS.NAME עבור book_id ב־DB דרך השרת
confirm_delete	book_id: int	מציג שני dialogs לאישור, מוחק שורה מטבלת BOOKS עבור book_id
refresh_library	—	הורס כל widget בחלון ומפעיל מחדש initialize_window
open_book	book_id: int	מחביא את החלון, פותח DigitalFlipbookApp בחלון חדש, מגדיר callback לסגירה
close_flipbook	flipbook_window	סוגר את flipbook_window ומחזיר את ההצגה של library_window
go_back	—	סוגר את library_window ומפעיל את go_back_callback

שם המחלקה: DigitalFlipbookApp

תפקיד המחלקה: GUI להצגת הספר הדיגיטלי, עריכתו, שמרת שינויו ושמידתו כ PDF

שם תכונה	תפקיד ושימוש
book_id	מזהה הספר בטבלת BOOKS לשליפת התוכן.
current_page	אינדקס העמוד המוצג. (0=cover,1=content)
current_font_size	גודל הגופן להצגת הטקסט.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	root, book_id: int	איתחול, GUI טען ספר מה DB דרך השרת אתחול, vars, יצירת widgets
create_widgets	—	בונה את כל ה- widgets: cover_page, content_page, button_frame, slider וכפתורים
update_font_size	value: float	מעדכן את current_font_size, משנה את ה-font של ה-content_label בהתאם
load_book_from_db	book_id: int	שולף את NAME, TEXT, OWNER מטבלת BOOKS, מוצא user_name, מחזיר title, author, text
save_changes	—	קורא את הטקסט מה-widget, מעדכן את BOOKS.TEXT ב-DB דרך השרת, מציג הודעת הצלחה
download_pdf	—	יוצר קובץ PDF ב-Downloads עם כותרת, מחבר וטקסט (באמצעות textwrap.wrap), מציג הודעת הצלחה
hide_all_pages	—	מסתיר את כל הדפים והרכיבים (pack_forget/place_forget) לפני הצגה של עמוד חדש
show_page	page_index: int	מציג את עמוד הכיסוי או התוכן וקורא ל-update_buttons
update_buttons	page_index: int	מסתיר/מציג כפתורי Prev/Next בהתאם לעמוד הנוכחי
next_page	—	משנה current_page ל-1 וקורא ל-show_page(1)
prev_page	—	משנה current_page ל-0 וקורא ל-show_page(0)

שם המחלקה: DigitalBookHandler

תפקיד המחלקה: בצד השרת מאפשרת את ההעלאה ל Cloudinary המתנה לתמלול והחזרת הטקסט.

תכונות המחלקה: אין תכונות קבועות- כל הלוגיקה מיושמת במתודות בלבד

שם הפעולה	טענת כניסה	טענת יציאה
create_digital_book	audio_file: str	מעלה את audio_file ל-Cloudinary, ממתין לסיום העיבוד (wait_for_transcription), מוריד transcript JSON, מחליץ את ה-transcript ומחזיר dict

public_id: str	wait_for_transcription	בודק בממשק cloudinary.api כל 10 שניות אם הקובץ transcript."public_id+" זמין או עד 30 ניסיונות
----------------	------------------------	---

שם המחלקה: EchoClient

תפקיד המחלקה: עבור לקוח התקשורת מאפשרת חיבור לשרת, החלפת מפתחות, ושליחת/קבלת פקודות.

שם תכונה	תפקיד ושימוש
client_socket	אובייקט TCP/IP socket המשמש לחיבור עם השרת.
aes_client	מופע AESCommClient לניהול הצפנה ואבטחה.

שם הפעולה	טענת כניסה	טענת יציאה
__init__	—	יוצר, socket TCP, אתחול aes_client, מגדיר connected=False
connect_to_server	—	מתחבר לשרת ב-IP/PORT, מקבל RSA pub key, קורא ל- send_aes_key, מעדכן connected=True
send_request	command: str, data: Any	שולח command+data דרך aes_client, קורא תגובה, מפענח JSON, מחזיר dict או error dict
close_connection	—	סוגר את ה-socket, מגדיר connected=False

שם המחלקה: EchoServer

תפקיד המחלקה: שרת ראשי שמאזין לחיבורים, הגנת DDOS, פענוח עיבוד בקשות הלקוח.

שם תכונה	תפקיד ושימוש
server_socket	אובייקט socket מאזין לחיבורים נכנסים.
public_key, private_key	זוג מפתחות RSA להחלפת מפתח AES עם הלקוח.
request_tracker	IP טיימסטאמפים לצורך הגבלת קצב-rate-limiting
blocked_ips	לזמן סיום החסימה במקרי חריגה בהגנה

שם הפעולה	טענת כניסה	טענת יציאה
-----------	------------	------------

בודק אם IP חורג מ־REQUEST_LIMIT בדקה, חוסם אם כן, מחזיר האם לטפל בבקשה (bool)	ip_address: str	defend_against_ddos
קורא 256 בתים מוצפנים מה־client_socket, מפענח ב־RSA עם ה־private_key, ומחזיר את הא־aes_key	client_socket	get_client_key
שולח את ה־public_key ללקוח, מקבל AES key, אתחול AESCommServer, קורא בסיבוב לבקשות, מפענח JSON, מפעיל את הפקודות ושולח תגובות	client_socket	handle_client
מעלה את קובץ ה־audio ל־Cloudinary, ממתין, מוריד transcript פשוט, מחזיר dict	data: dict	upload_audio
מפעיל לולאת accept () עם timeout, יוצר thread לכל client, מטפל ב־KeyboardInterrupt לסגירת השרת	—	start

שם המחלקה ManageDB:

תפקיד המחלקה: ניהול גישה וביצוע פעולות CRUD על בסיס הנתונים- יצירת והזדהות משתמשים, וניהול הספרים.

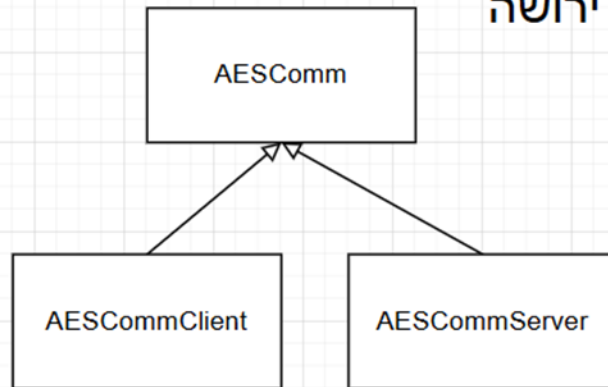
שם תכונה	תפקיד ושימוש
db_name	שם קובץ מסד הנתונים שאליו מתבצע חיבור

שם הפעולה	טענת כניסה	טענת יציאה
__init__	database: str	אתחול אובייקט ושמירת שם קובץ ה־DB-
hash_password	password: str	יצירת hash bcrypt מתוך סיסמה רגילה והחזרתו
verify_password	plain_password: str, hashed_password: str	בדיקת התאמה בין סיסמה רגילה להאש חזרת bool
login_user	data: dict{"username":str, "password":str}	אימות משתמש: בדיקת שם וסיסמה ב־USER_INFO, החזרת {"status", "message"}
create_account	data: dict{"username":str, "password":str}	יצירת חשבון חדש: בדיקת קיום, hash לסיסמה, הוספה ל־USER_INFO, חזרת {"status", "message"}
load_book_from_db	data: dict{"book_id":int}	שליפת נתוני ספר (TITLE, TEXT, author) ב־BOOKS/USER_INFO, חזרת

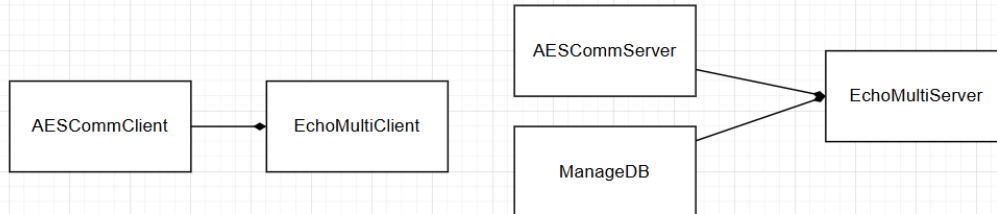
שגיאה או {"status","books_title","author","text"}		
שמירת שינויים לטקסט ספר בBOOKS חזרת {"status","message"}	data: dict{"book_id":int, "updated_text":str}	save_text_changes
שליפת השם הנוכחי של ספר ל-edit mode, חזרת {"status","book_name"} או שגיאה	data: dict{"book_id":int}	get_book_name
עדכון שדה NAME של ספר ב BOOKS-חזרת {"status","message"}	data: dict{"book_id":int, "new_name":str}	update_book_name
שמירת ספר חדש ב- BOOKS (USERNAME→OWNER), חזרת {"status","book_id"} או שגיאה	data: dict{"username":str, "book_name":str, "transcription_text":str}	submit_book
מחיקת רשומת ספר לפי ID חזרת {"status","message"}	data: dict{"book_id":int}	delete_book
קבלת user_id מתוך USER_INFO לפי username, חזרת {"status","user_id"} או שגיאה	data: dict{"username":str}	get_user_id
קבלת user_name מתוך USER_INFO לפי user-id, חזרת str או שגיאה	data: dict{"user-id":int}	get_user_name
שליפת רשימת ספרים (ID, NAME) של משתמש, חזרת {"status","books":[{"ID, NAME},...]} או שגיאה	data: dict{"user_id":int}	get_books_list

תרשימי מחלקות המתארים קשרי הכלה/ירושה:

תרשים ירושה



תרשים הכלה



חלק ב'- הבעיות האלגוריתמיות המרכזיות

העברת מפתח AES באמצעות RSA

AESCommClientב

```
def send_aes_key(self, server_public_key):
    # Encrypt the AES key with the server's public RSA key and send it
    encrypted_aes_key = rsa.encrypt(self.aes_key, server_public_key)
    self.client_socket.sendall(encrypted_aes_key)
    print("AES key encrypted and sent to server.")
```

EchoServerב

```
def get_client_key(self, client_socket):
    enc_aes_key = client_socket.recv(256) # Encrypted AES key
    aes_key = rsa.decrypt(enc_aes_key, self.private_key) # Decrypt using server's private key
    return aes_key
```

הסבר על היכולת: יצירת ערוץ תקשורת מוצפן חזק על ידי העברת מפתח ההצפנה הסימטרי AES באופן בטוח בעזרת הצפנתו במפתח הציבורי של השרת.

מניעת הצפת בקשות

EchoServerב

```
def defend_against_ddos(self, ip_address):
    """
    Prevent DDoS attacks by tracking and limiting request rates per IP.
    If an IP exceeds REQUEST_LIMIT within 60 seconds, block it temporarily.
    """
    current_time = time.time()
    # Check if IP is already blocked
    if ip_address in self.blocked_ips:
        if current_time < self.blocked_ips[ip_address]:
            print(f"[DDoS PROTECTION] Blocked request from {ip_address}.")
            return False
        else:
            del self.blocked_ips[ip_address]
    # Initialize tracker for IP if not present
    if ip_address not in self.request_tracker:
        self.request_tracker[ip_address] = []
    # Remove timestamps older than 60 seconds
    self.request_tracker[ip_address] = [timestamp for timestamp in self.request_tracker[ip_address] if current_time - timestamp < 60]
    # Add current request timestamp
    self.request_tracker[ip_address].append(current_time)
    # Check if request count exceeds limit
    if len(self.request_tracker[ip_address]) > self.REQUEST_LIMIT:
        print(f"[DDoS ALERT] Too many requests from {ip_address}. Blocking temporarily.")
        self.blocked_ips[ip_address] = current_time + self.BLOCK_TIME
        return False
    return True
```

הסבר על היכולת: הגנה נגד מתקפות DDoS על ידי הגבלת מספר הבקשות מכל כתובת IP בחלון זמן קבוע של 60 שניות, עם חסימה זמנית במידה והחריגה מהמגבלה.

הרשמה וכניסה של משתמשים

הסבר על היכולת: ניהול בטוח של יצירת חשבונות ואימות כניסה, כולל שמירת סיסמאות כ-hash והגבלת חיבור כפול באמצעות דגל. `Echo_MANAGEDB`

```
def login_user(self, data):
    # Handle user login
    username = data.get("username")
    password = data.get("password")
    # print("login_user - username:",username)
    # print("login_user - password:",password)

    if not username or not password:
        return {"status": "error", "message": "Username and password required."}

    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT user_password FROM USER_INFO WHERE user_name = ?", (username,))
        result = cursor.fetchone()

        if result:
            stored_hash = result[0]
            if self.verify_password(password, stored_hash):
                # print("Login successful")
                return {"status": "success", "message": "Login successful."}

        print("Login failed")
        return {"status": "error", "message": "Invalid username or password."}
```

```
def create_account(self, data):
    """
    Handle account creation.
    """
    username = data.get("username")
    password = data.get("password")
    if not username or not password:
        return {"status": "error", "message": "Username and password required."}
    # print("create_account - username:",username)
    # print("create_account - password:",password)
    hashed_password = self.hash_password(password)
    # print("create_account - hashed_password:",hashed_password)
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("CREATE TABLE IF NOT EXISTS USER_INFO (user_id INTEGER PRIMARY KEY, user_name TEXT, user_password TEXT)")
        cursor.execute("SELECT * FROM USER_INFO WHERE user_name = ?", (username,))
        if cursor.fetchone():
            return {"status": "error", "message": "Username already exists."}
        cursor.execute("INSERT INTO USER_INFO (user_name, user_password) VALUES (?, ?)", (username, hashed_password))
        conn.commit()
        return {"status": "success", "message": "Account created successfully."}
```

הסבר על היכולת: יצירת חשבונות ובדיקת התחברות על-פי username ו-password כפי שנשמרו ב-DB, אם כבר קיים משתמש, מחזירים שגיאה.

הצפנה ואימות של ההודעות (AES-GCM)

AESComm

```
def encrypt_payload(self,request):
    aes_nonce = get_random_bytes(12) # Generate a 12-byte random nonce (Number used once - נוצר פעם שנשלחת הודעה מוצפנת, נוצר כדי למנוע התקפות חוזרות. כל פעם שנשלחת הודעה מוצפנת, נוצר Nonce (הצפנה אחת) לבד (למשל, הצפנה אחת)
    aes_cipher = AES.new(self.aes_key, AES.MODE_GCM, aes_nonce) # Create AES-GCM cipher object with the AES key and nonce
    ciphertext, tag = aes_cipher.encrypt_and_digest(request.encode()) # Encrypt and get tag- תג אימות, תג אימות
    # Send: nonce + tag + len + ciphertext- Combine nonce, tag, length of ciphertext, and ciphertext into one payload
    payload = aes_cipher.nonce + tag + struct.pack("!I", len(ciphertext)) + ciphertext
    return payload
```

AESComm

```
def decrypt_payload(self,header):
    # Extract components from the header
    nonce = header[:12] # first 12 bytes: nonce
    tag = header[12:28] # next 16 bytes: tag
    ciphertext_length = struct.unpack("!I", header[28:32])[0] # next 4 bytes: length of ciphertext
    print("ciphertext len:", ciphertext_length)

    # Receive the encrypted message body
    ciphertext = self.recv_exact(ciphertext_length)

    # Decrypt the message using AES-GCM, AES (Advanced Encryption Standard) GCM (Galois/Counter Mode)
    aes_cipher = AES.new(self.aes_key, AES.MODE_GCM, nonce=nonce)
    decrypted_message = aes_cipher.decrypt_and_verify(ciphertext, tag)
    return (decrypted_message.decode())
```

הסבר על היכולת: שימוש במצב GCM של AES כדי לספק הצפנה סימטרית לצד אימות שלמות (TAG) בפעולה אחת

סנכרון GUI ותקשורת עם המולטי-קליינטים

EchoServer

```
def start(self):
    """
    Start the server and accept client connections.
    """
    try:
        while self.running:
            try:
                # Accept new client connection
                client_socket, client_address = self.server_socket.accept()
                # DDoS Protection: if client IP exceeds request limit, skip handling
                if not self.defend_against_ddos(client_address[0]):
                    client_socket.close()
                    continue
                # Handle client in a new thread
                threading.Thread(target=self.handle_client, args=(client_socket,)).start()
            except socket.timeout:
                continue # No connection, keep looping
    except KeyboardInterrupt:
        print("Server shutting down...")
        self.running = False
        self.server_socket.close()
        exit(0)
```

סבר על היכולת: הרצת ה-mainloop של ה-Tkinter בתהליך הראשי, ובו זמנית ניהול חיבורים מלקוחות רבים ב-thread נפרד לכל לקוח, כדי שלא יחסמו זה את זה.

תמלול קובץ שמע

DigitalBookHandler

```
def wait_for_transcription(self, public_id):
    # Wait for Cloudinary processing- Wait until the transcription file is ready
    print ("Waiting for transcription...")
    bytes = 0
    times = 0
    while(bytes == 0 and times < 30):
        time.sleep(10) # Wait for 10 seconds before checking again
        times += 1
        result = cloudinary.api.resource(public_id+".transcript", resource_type="raw", type = "authenticated")
        bytes = result['bytes'] # Check if the file size is greater than 0
        print("Bytes:", bytes)
        print("Times:", times)
```

```

def create_digital_book(self, audio_file):
    """
    Create a digital book from the provided audio file.
    Returns a dictionary with status and transcription or an error message.
    """
    try:
        # Upload the audio file to Cloudinary for processing
        #Cloudinary version
        res = cloudinary.uploader.upload(audio_file, resource_type="video", upload_preset="audio2text")

        #Google version
        #res = cloudinary.uploader.upload(audio_file, resource_type="video", upload_preset="audio2text-ggl")

        print("Cloudinary upload response:", res)
        public_id = res["public_id"]

        # Wait for Cloudinary processing- to finish transcription
        self.wait_for_transcription(public_id)

        # Construct the transcription URL- Generate a secure link to download the transcript
        # transcript_url = res["secure_url"].replace("video", "raw")[:-3] + "transcript"
        # print("Transcription URL:", transcript_url)
        signed_url, options = cloudinary.utils.cloudinary_url(public_id + ".transcript",
                                                                resource_type="raw",
                                                                type = "authenticated",
                                                                sign_url=True )

        # Request transcription JSON from Cloudinary. Download and parse the transcription JSON
        response = urlopen(signed_url)
        data_json = json.loads(response.read())
        print("Transcription JSON:", data_json)

        # Extract transcript text from each segment
        transcript = ""
        # Extract the transcript
        for item in data_json:
            print("Item: ", item)
            print("transcript of item: ", item['transcript'])
            transcript += item['transcript']

        return {"status": "success", "transcription": transcript}

```

הסבר על היכולת: העלאת קובץ WAV ל-Cloudinary ולאחר מכן בדיקה מחזורית כל 10 שניות (עד 30 ניסיונות) אם קובץ התמלול מוכן, כדי למשוך את ה-JSON עם הטקסט ולבסוף להפוך אותו לספר דיגיטלי.

חלוקה לדפים בספר הדיגיטלי

```

def show_page(self, page_index): # Show the relevant page based on the index (0: cover, 1: content)
    self.hide_all_pages() # Hide any previously shown pages before displaying a new one
    if page_index == 0: # If the page index is 0 (cover page) Display the cover page
        self.cover_page.pack(fill="both", expand=True)
    elif page_index == 1: # If the page index is 1 (content page) Display the content page
        self.content_page.pack(fill="both", expand=True) # Position buttons on the screen
        self.edit_button.place(x=10, y=10)
        self.append_button.place(x=130, y=10)
        self.download_button.place(x=250, y=10)
        self.font_slider.place(relx=0.95, y=10, anchor="ne")
    self.update_buttons(page_index) # Update button visibility and position based on the current page index

```

הסבר על היכולת: טעינת התוכן של העמוד הנוכחי והעמוד הבא מראש בזיכרון, כדי לאפשר גלילה חלקה ללא השהיות בזמני טעינה.

קטעי קוד מיוחדים

מעבר דינמי בין תצוגת Label לעריכת TextBox כשהמשתמש מעוניין לערוך את תוכן הספר

בשלב DigitalFlipBookApp EditManager

```
def edit_content(self):
    if not hasattr(self.flipbook_app, "is_editing") or not self.flipbook_app.is_editing: # Check if the app is in editing mode
        self.flipbook_app.is_editing = True # Set editing mode to True
        current_text = self.flipbook_app.content_label.cget("text") # Get the current content text
        self.flipbook_app.content_label.destroy() # Destroy the current label to replace it with a textbox
        self.flipbook_app.content_label = customtkinter.CTkTextbox(
            self.flipbook_app.content_page, width=750, height=400, font=("Times New Roman", self.flipbook_app.current_font_size)
        )
        # Create a new text box for editing
        self.flipbook_app.content_label.insert("1.0", current_text)
        self.flipbook_app.content_label.configure(state="normal")
        self.flipbook_app.content_label.pack(pady=20, padx=20)
        self.flipbook_app.edit_button.configure(text="Done Editing", fg_color="#90EE90")

    else: # If already in editing mode, save changes and return to view mode
        new_text = self.flipbook_app.content_label.get("1.0", "end").strip() # Get the edited text
        self.flipbook_app.content_label.destroy() # Remove the editable text box
        self.flipbook_app.content_label = customtkinter.CTkLabel(
            self.flipbook_app.content_page,
            text=new_text,
            font=("Times New Roman", self.flipbook_app.current_font_size),
            wraplength=750,
            justify="left",
            text_color="#343A40"
        )
        # Create a new label with the updated text
        self.flipbook_app.content_label.pack(pady=20, padx=20)
        self.flipbook_app.is_editing = False
        self.flipbook_app.edit_button.configure(text="Edit Content", fg_color="#A1C2D1")
        self.flipbook_app.save_changes()
```

הסבר על היכולת: מאפשר למשתמש ללחוץ על כפתור "Edit Content" ולהפוך את התצוגה מקריאת טקסט ל-Textbox לעריכה חופשית, ואז לחזור לחציבת תצוגת Label עם הטקסט החדש – בלי לצאת מהחלון.

יצירת PDF דינמי עם ReportLab של הספר והתוכן שלו

בבוקר DigitalFlipBookApp

```

def download_pdf(self): # Download the book as a PDF
    try:
        from reportlab.lib.pagesizes import letter
        from reportlab.pdfgen import canvas
        import os
        import textwrap

        downloads_folder = os.path.expanduser("~/Downloads")
        filename = os.path.join(downloads_folder, f"{self.book_title}.pdf")

        c = canvas.Canvas(filename, pagesize=letter)
        width, height = letter

        # Set the title font and write the book title and author
        c.setFont("Times-Bold", 24)
        c.drawCentredString(width / 2, height - 100, self.book_title)
        c.setFont("Times-Roman", 18)
        c.drawCentredString(width / 2, height - 130, f"by {self.author}")
        c.showPage()

        # Create a text object to write the content
        textobject = c.beginText(40, height - 50)
        textobject.setFont("Times-Roman", 12)

        content = self.content_label.get("1.0", "end").strip() if isinstance(self.content_label, customtkinter.CTkTextbox) else self.content_label.cget("text")

        # Wrap text and write to PDF
        for paragraph in content.split("\n"):
            wrapped_lines = textwrap.wrap(paragraph, width=80) or [""] # Wrap lines to avoid overflow
            for line in wrapped_lines:
                textobject.textline(line)

        c.drawText(textobject) # Add the text object to the canvas
        c.save() # Save the PDF

        tkmb.showinfo("Download PDF", f"PDF successfully created and saved: {filename}")
    except Exception as e:
        tkmb.showerror("Error", f"An error occurred while exporting PDF: {e}")

```

הסבר על היכולת: חיבור הטקסט מה-Flipbook ל-PDF מושקע עם כותרת, סגנון גופנים ושכירת שורות אוטומטית (textwrap), ושמירה אוטומטית בתיקיית Downloads. אם התוכן של הספר שונה על ידי המשתמש כשערך אותו, והמשתמש שמר את הספר ועשה לו הורדה, תוכן קובץ הפידיאף משתנה בהתאם.

חלק ג' מסמך בדיקות מלא

בדיקה 1: התחברות תקינה

מטרה הבדיקה: לוודא שמשתמש קיים מצליח להיכנס עם שם וסיסמה תקינים.

מה בוצע בפועל: במסך ההתחברות הוזנו "username="Fiona" ו-"password="Password123! לחצתי על כפתור הLogin.

תוצאות הבדיקה: נרשמת הצלחה, החלון נסגר ונפתח מסך ה LandingPage עם שם המשתמש בראש.

בעיות ופתרון: לא נמצאו בעיות.

בדיקה 2: התחברות עם פרטי משתמש שגויים

מטרה הבדיקה: לוודא שהמערכת חוסמת כניסה עם שם משתמש או סיסמה לא נכונים.

מה בוצע בפועל: במסך ההתחברות הוזנו "username="Fiona" ו-"password="WrongPass" ולחצתי Login.

תוצאות הבדיקה: מוצגת תיבת שגיאה "Invalid username or password." והחלון נשאר פתוח.

בעיות ופתרון: לא נמצאו בעיות.

בדיקה 3: יצירת חשבון חדש

מטרה הבדיקה: לוודא שניתן לרשום משתמש חדש העומד בקריטריוני סיסמה חזקה.

מה בוצע בפועל: במסך Create New Account הזנתי "username="bob" ו-"password="Strong@2025", לחצתי Submit.

תוצאות הבדיקה: הוצגה הודעת "Account created successfully." ו-DB מכיל שורה חדשה עם הפרטים.

בעיות ופתרון: לא נמצאו בעיות.

בדיקה 4: יצירת חשבון עם שם קיים

מטרה הבדיקה: לוודא שהמערכת מונעת רישום שם משתמש שכבר קיים.

מה בוצע בפועל: ניסיתי ליצור שוב את "username="bob" עם סיסמה אחרת.

תוצאות הבדיקה: התקבלה תיבת שגיאה "Username already exists." וללא שורה נוספת ב-DB.

בעיות ופתרון: לא נמצאו בעיות.

בדיקה 5: החלפת מפתח AES באמצעות RSA

מטרה הבדיקה: לוודא שהלקוח מקבל את המפתח הציבורי של השרת, שולח את מפתח ה-AES המוצפן, והשרת מפענח אותו נכון.

מה בוצע בפועל: הרצתי את השרת והלקוח במקביל, ובדקתי בלוג:

השרת הדפיס: "Connected to the server and received public key!"

הלקוח הדפיס: "AES key encrypted and sent to server."

השרת הדפיס: "AES key received from client"

תוצאות הבדיקה: הפענוח עבר בהצלחה ללא שגיאות.

בעיות ופתרון: בהפעלה ראשונית גיליתי המתנה ארוכה מדי עד לקבלת ה-public key אז התמודדתי עם זה על ידי כך שהוספתי timeout קצר יותר על הקריאה ל-recv כדי לטפל בזה.

בדיקה 6: תקשורת AES-GCM מוצפנת

מטרה הבדיקה: לוודא שהודעה מוצפנת ונשלחת בשלמותה, מפוענחת מבלי לפגוע בתוכן.

מה בוצע בפועל: באמצעות לקוח שלחתי מחרוזת ארוכה ורעננתי את לוג השרת והלקוח.

תוצאות הבדיקה: הטקסט התקבל במלואו ללא חריגות Invalid Tag או שגיאות.

בעיות ופתרון: בהתחלה נשלחו רק 1024 בתים ולא כל ההודעה פתרתי זאת בעזרת recv_exact שקוראת בדיוק את כל הבתים המוגדרים ב-header.

בדיקה 7: העלאת קובץ אודיו לתמלול

מטרה הבדיקה: לוודא שהקובץ מתקבל אצל Cloudinary תמלול מבוצע והטקסט מוחזר בהצלחה.

מה בוצע בפועל: שלחתי פקודה create_digital_book עם קובץ WAV קטן של ממש כמה שניות.

תוצאות הבדיקה:

Cloudinary קיבל את הקובץ.

לאחר מספר Polling כל 10 שניות התמלול הופיע ב-JSON

הפונקציית create_digital_book החזירה את הספר הדיגיטלי שתומלל.

בעיות ופתרון: בקריאה הראשונה ל-API התקבלה בעיה עד שהקובץ התפרסם, כדי לפתור את זה הוספתי לולאה שכל אינטרציה ממתינה 10 שניות ואז מבצעת שוב את הקריאה, עד שהקובץ יהיה מוכן.

בדיקה 8: יצירת ספר דיגיטלי מהתמלול

מטרה הבדיקה: לוודא שהתמלול נשמר בטבלת BOOKS עם שם הספר וטקסט התמלול.

מה בוצע בפועל: לאחר קבלת התמלול, במסך BookCreationWindow הזנתי שם "MyBook" ולחצתי Submit.

תוצאות הבדיקה: טבלת BOOKS מכילה שורה עם TEXT, NAME="MyBook", ו-OWNER נכון.

בעיות ופתרון: בהתחלה ראיתי שהטבלה לא נוצרת אוטומטית אז הוספתי קריאה ל-
CREATE TABLE IF NOT EXISTS BOOKS INSERT לפני ה-

בדיקה 9: פתיחת ספר דיגיטלי

מטרה הבדיקה: לוודא שהספר נטען במלואו וניתן לדפדף קדימה ואחורה.

מה בוצע בפועל: במסך הספרייה הדיגיטלית לחצתי על "MyBook". ואחרי שהוא נפתח לחצתי Next ואז Previous מספר פעמים.

תוצאות הבדיקה: מעבר הדפים חלק, הקאבר והעמוד התוכן נטענו כפי שרציתי.

בעיות ופתרון: לא נמצאו בעיות.

בדיקה 10: יצוא כקובץ PDF

מטרה הבדיקה: לוודא שנוצר קובץ PDF בתיקיית Downloads עם כותרת, מחבר וטקסט נכון.

מה בוצע בפועל: פתחתי ספר דיגיטלי מהאפליקציה, לחצתי Download PDF, פתחתי את הקובץ שתאם את השם של הספר בדאונלואודס שיש אצלי על המחשב וגללתי.

תוצאות הבדיקה: PDF הוצג כהלכה, כל השורות נשברו בהתאם, והכותרת והמחבר במקום הנכון.

בעיות ופתרון: בהתחלה לא השתמשתי ב-textwrap אז כל הטקסט נכתב בשורה אחת ארוכה, דבר שגרם לחריגה מגבולות העמוד ולחיתוך חלקי של מילים בפסקאות הארוכות. הפתרון היה לחפש באינטרנט מה אני יכולה לעשות כדי לתקן את זה עד שמצאתי את textwrap והוספתי לפרויקט.

פירוט בדיקות נוספות אותן בצעתי למערכת:

מניעת יצירת ספר עם שם ריק

מטרה הבדיקה: לוודא שהמערכת לא תאפשר יצירת ספר ללא שם ותמנע קריסה.

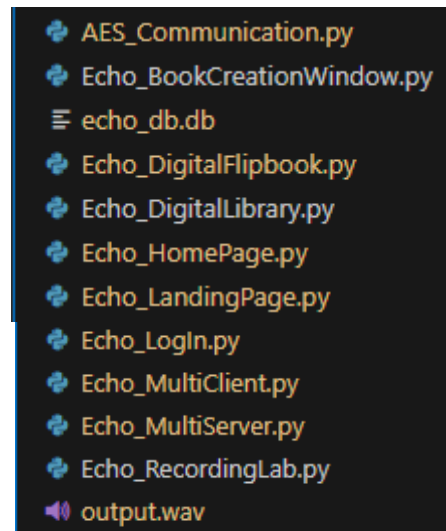
מה בוצע בפועל: במסך Create New Book השארתי את שדה שם הספר ריק ולחצתי על Submit and Open Digital Flipbook.

תוצאות הבדיקה: הופיעה תיבת שגיאה "Book name cannot be empty". והחלון נשאר פתוח, ללא הכנסת ערך ל-DB.

בעיות ופתרון: בהתחלה לא היה קיים ב-submit_book בלוק בדיקה שמונע קריסה ומוודא שהשם לא ריק. אבל אחרי שהוספתי אותו לא היו בעיות נוספות.

מדריך למשתמש:

עץ קבצי המערכת:



התקנת המערכת:

פירוט הסביבה הנדרשת:

Python 3.13.3 כולל מודול tkinter ו-sqlite3 מובנים.

PortAudio עבור PyAudio בשביל להקליט

גישה לרשת כדי לתקשר עם השרת.

פירוט הכלים הנדרשים:

להתקין PyCharm או VisualStudioCode עם פייתון, בכדי להריץ בצורה נוחה את הקוד

DB Browser for SQLite

להתקין PIP

מיקומי קבצים: להניח את כל הקבצים בתיקייה ראשית, למשל echo_project

נתונים התחלתיים: אין צורך בשמות משתמש או סיסמא, משתמש שנכנס רשאי להכין לעצמו אחד מבלי שיהיה קיים לפני.

רשת: הלקוח והשרת חייבים להיות באותה רשת LAN או שהפורט פתוח וניתן לגישה.

ארכיטקטורה מינימלית נדרשת: נדרש רק שרת שהקליינט יתחבר אליו

משתמשי המערכת:**מדריך למשתמש Echo Client:**

על המשתמש לשנות את IP השרת לIP של המחשב שבו מופעל השרת אם מפעיל אותו, לפני שמריץ את האפליקציה. לאחר מכן, כשהשרת פעיל, על המשתמש להריץ את קוד ה-Echo_MultiClient.py בסביבת העבודה שבחר להדביק בה את הקוד.

לאחר שהמשתמש מתחיל להריץ את קוד הקליינט מוצג לו ראשית המסך הבא:

זהו מסך ה-LOGIN של Echo, פה הוא יכול להתחבר למשתמש קיים עם שם המשתמש והסיסמא שלו ולחיצה על כפתור ה-Login- אם יש לו כזה, ואם אין לו הוא יכול ללחוץ על כפתור ה-Create New Account.

אם הוא בוחר להכין משתמש חדש, ולוחץ על הכפתור. המשתמש מגיע לחלון הבא:

כאן הוא רשאי להכניס את שם המשתמש שירצה שיקראו לאקאונט החדש שיווצר, ואת הסיסמא. על הסיסמא לעמוד בדרישות של Echo ל"סיסמא חזקה", אם היא לא עומדת בתקן, או שהשם משתמש שהמשתמש החדש רוצה לקרוא לאקאונט שלו לקוח, או שהוא לא הכניס בשדות פרטים ראויים (למשל השאיר אותם ריקים), יוצגו למשתמש ההודעות שגיאנה הבאות, כאשר כל אחת תואמת את הבעיה שבגללה הוצגה:

המשתמש רשאי ללחוץ על כפתור הsuggest password אם מעוניין שהמערכת תציע לו סיסמא שעומדת בתקנים, ואם מעוניין לצפות בה, הוא יכול ללחוץ על האייקון של העין שאחראי על להסתיר את הסיסמא.

לבסוף, אם המשתמש מכניס שם שלא קיים במערכת וסיסמא חזקה תוצג לו הודעת ההצלחה הבאה:

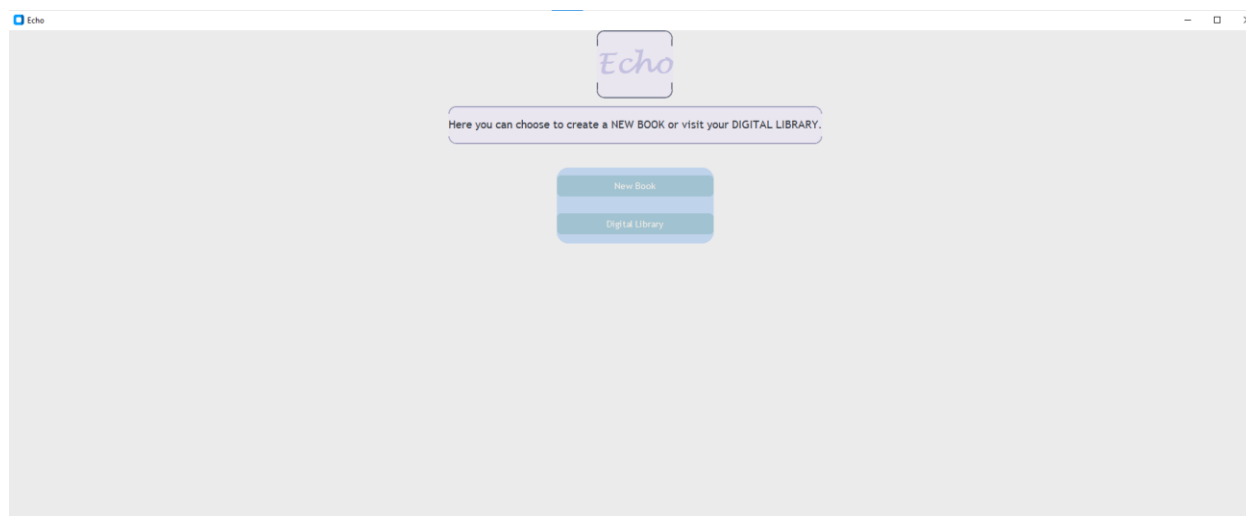
Account Created



Your account has been created successfully.

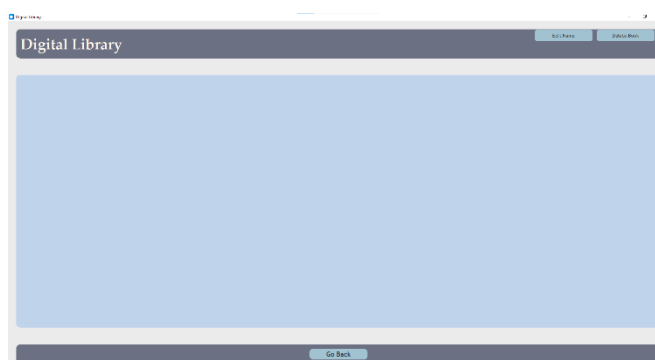
אישור

וכשילחץ על "אישור" חלון הכנת משתמש חדש יסגר אוטומטית ויחזיר את המשתמש לחלון הלוגאין, שם הוא יוכל להכניס את הפרטים של האקאונט שהכין ולהתחבר לEcho ויוצג לו מסך הכניסה הבא:



פה המשתמש רשאי לקרוא את ההוראות של מה ניתן לעשות בעמוד, ולהחליט אם מעוניין להכין ספר חדש או לגלוש בספרייה הדיגיטלית שלו. *הספרייה הזאת תהיה ריקה אם למשתמש אין ספרים, אך ברגע שיכין אחד, היא תכיל אותו וכאלו עתידיים.

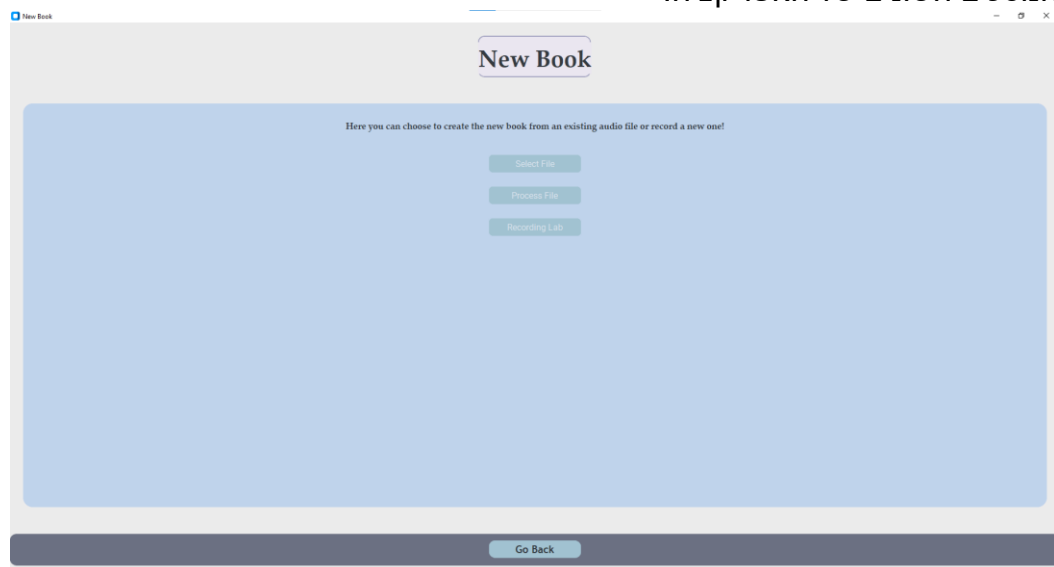
דוגמא לחלון ספרייה של משתמש ללא ספרים:



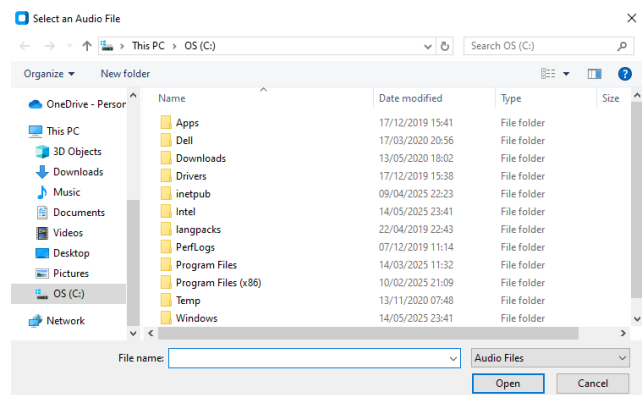
דוגמא לחלון ספרייה של משתמש עם ספרים:



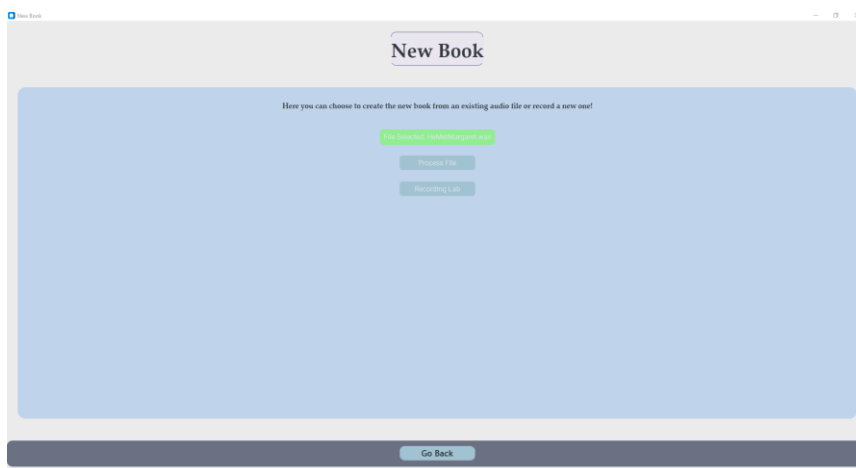
כאשר המשתמש לוחץ על כפתור הכנת ספר חדש, מוצג לו המסך הבא, ונסגר הקודם. המשתמש רשאי בכל רגע לחזור למסך הקודם בלחיצה על כפתור החזרה בתחתית העמוד, איתו הוא יכול לדפדף בין המסכים השונים של האפליקציה.



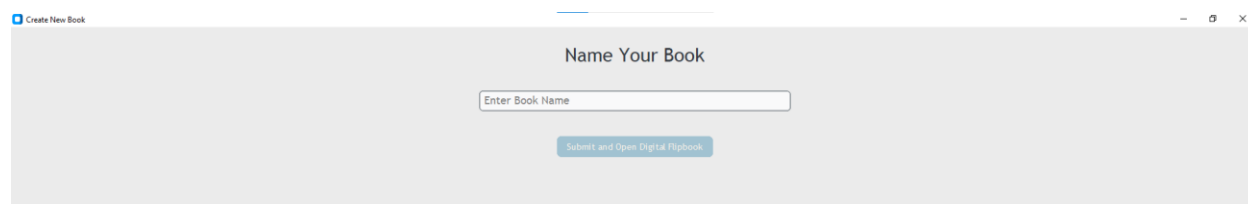
במסך ההכנה של ספר חדש מוצגות למשתמש הוראות מה הוא יכול לעשות. הוא רשאי להעלות קובץ אודיו קיים, (שיש לו על המחשב לדוגמא) לתמלול והפיכה לספר דיגיטלי- אם לוחץ על כפתור בחירת הקובץ. ואז עולה לו המסך הבא לבחירת קבצים.



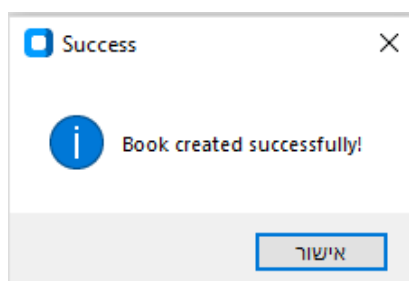
כאשר המשתמש בוחר את הקובץ הרצוי, הכפתור משנה את צבעו לירוק והטקסט שלו משתנה לשם הקובץ הנבחר. עכשיו המשתמש יכול ללחוץ על Process File כדי להפוך את הקובץ לספר דיגיטלי.



לחיצה זו תתחיל את פעולת הכנת הספר ותוביל את המשתמש לחלון הבא.



החלון הזה מוצג למשתמש בכל ספר חדש שהוא מכין, לאחר שהספר עצמו תומלל כבר, והוא השלב האחרון לפני הפתיחה שלו. כאן המשתמש רשאי לתת שם שהוא בוחר לספר החדש ולפתוח אותו. אחרי שהוא לוחץ על כפתור השליחה ופתיחה של הספר החדש מופיעה לו הודעת ההצלחה הזאת



וכאשר ילחץ אישור יפתח לו הספר החדש ויוצג לו בצורה הזאת בחלון הבא:



כאשר שם הספר שנתן לו, נמצא בטייטל, ושם המשתמש כתוב אחרי נכתב עלי ידי.

זהו עמוד הקאבר של הספר, הוא מכיל בנוסף שני כפתורים. כפתור שמירה ששומר את הספר לדאטה בייס ובכך גם לספרייה הדיגיטלית של המשתמש, כך שיוכל מעכשיו לדפדף בו מתי שרוצה, וכפתור "הבא" שיוביל את המשתמש לעמוד התוכן של הספר שמכיל את המילים שאמר בהקלטה שממנה יצר את הספר. אם המשתמש רוצה לסגור את הספר הוא יכול ללחוץ על כפתור האיקס בפינה למעלה, וזה יחזיר אותו למסך ממנו התחיל את תהליך הכנת הספר.

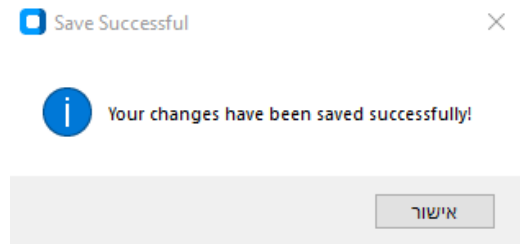
כאשר המשתמש לוחץ על כפתור "הבא" מוצג לו המסך הזה, שמכיל את תוכן הספר:



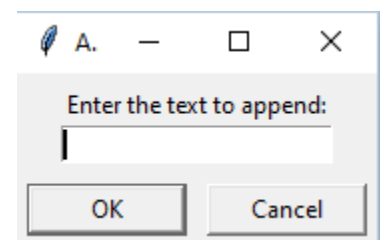
כאן המשתמש רשאי לעשות מספר דברים, הוא יכול לגלול מטה כדי לראות את שאר תוכן הספר (אם תוכן הספר ארוך מדי כדי להיות מוצג במלואו במסך). הוא יכול ללחוץ עך כפתור השמירה ממקודם, או כפור ה"חזור" שיחזיר אותו לעמוד הקאבר של הספר. אם יש בעיה בתמלול האודיו, המשתמש יכול ללחוץ על כפתור עריכת הטקסט ויתעדכן לו המסך בצורה הזאת:



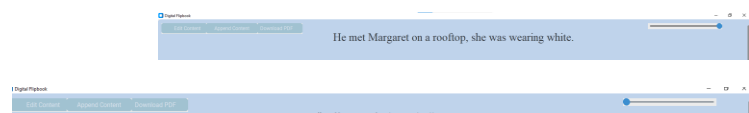
כפתור העריכה משנה את צבעו ואת מה שכתוב בוא וכך, המשתמש יכול להקליד בעצמו את השינויים, ולמחוק דברים מתוכן הספר. כאשר הוא סיים, הוא יכול ללחוץ על הכפתור המעודכן שעכשיו אומר "סיימתי לערוך" ובכך לכבל הודעת צליחה, תוכן הספר מתעדכן לפי מה שכתב ושינויי הספר נשמרים אוטומטית.



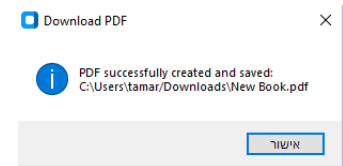
אם המשתמש לוחץ על הכפתור של הוסף טקסט, מופיעה לו החלונית הזאת שם הוא רשאי לכתוב את השורה שרוצה להוסיף, והיא תתווסף כשילחץ על אישור מתחת לשורה האחרונה בתוכן.



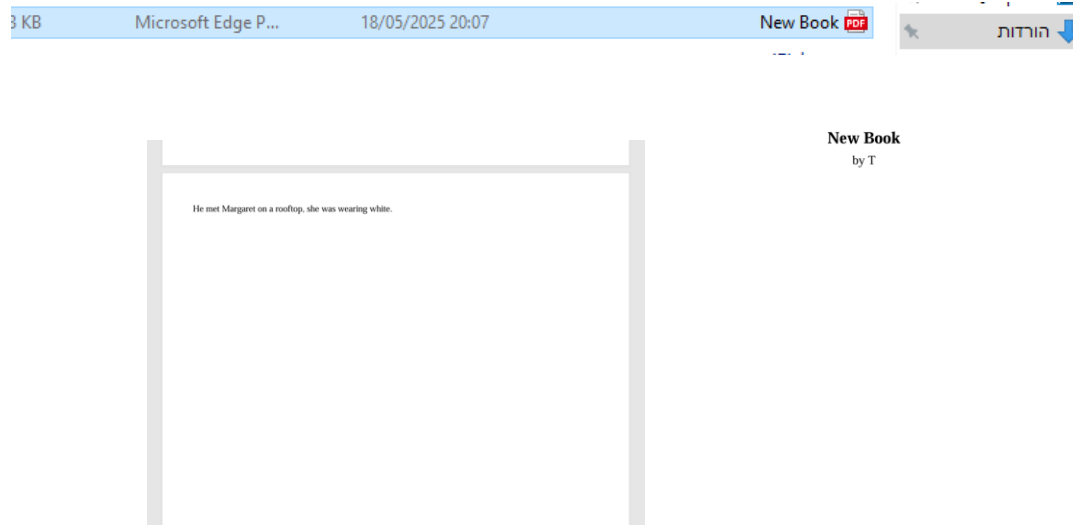
בנוסף, המשתמש יכול לשנות את גודל התצוגה של הטקסט לפי נוחותו בצורה דינמית, ככל שמזיז את scrollbar שנמצא בצד ימין למעלה.



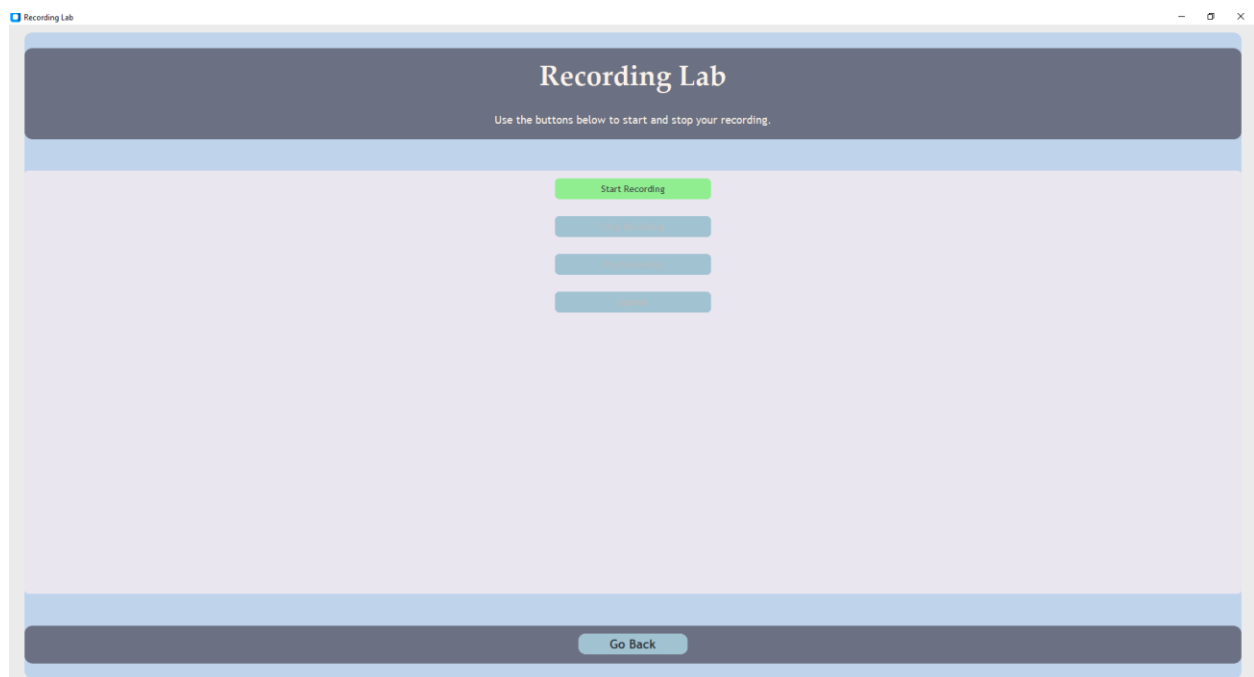
אם המשתמש מעוניין להוריד את הספר שלו שישמר כקובץ PDF למחשב שלו, הוא יכול ללחוץ על כפתור ההורדה ולקבל את הודעת ההצלחה הבאה:



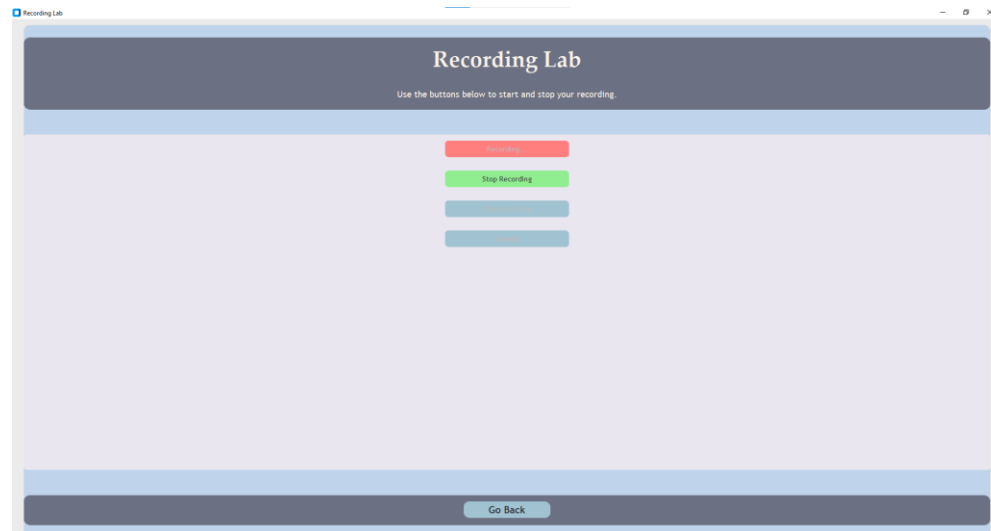
לאחר מכן הספר ימצא אצלו בתיקיית ההורדות על המחשב והוא רשאי לפתוח אותו ולקרוא בו



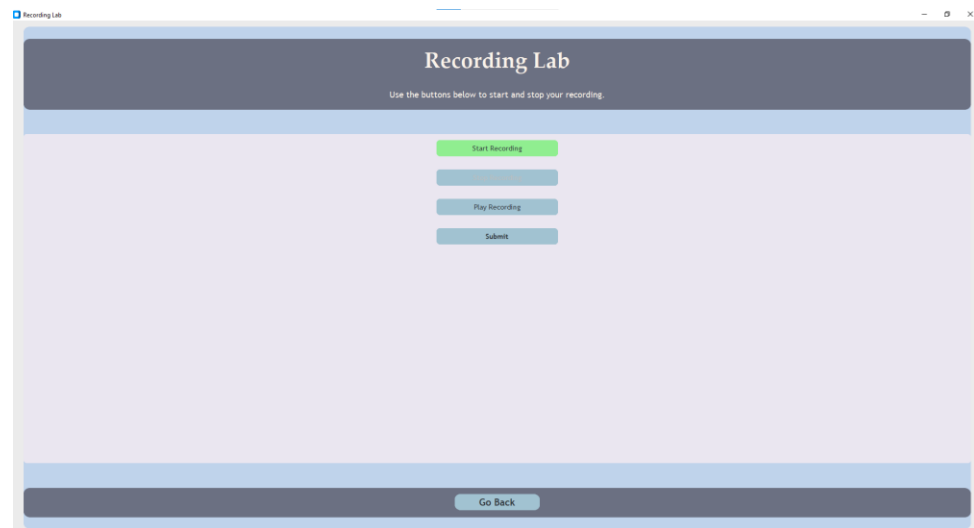
לעומת זאת, אם המשתמש מחליט שהוא רוצה להכין את ספרו החדש בהקלטת קול שנעשית במקום ולא מוכנה מראש כקובץ. הוא יכול לעבור למעבדת ההקלטות שנראת כך:



שם הוא יכול לחזור לעמוד הקודם בלחיצה על כפתור החזרה, או להתחיל הקלטה קולית בלחיצה על כפתור ההתחלת הקלטה שזוהר בירוק. כאשר הוא מתחיל הקלטה, הכפתורים משתנים דינמית ומתאימים את עצמם לשלב בו נמצא המשתמש בתהליך ההקלטה. כפתור התחלת ההקלטה משתנה לאדום ומקרין טקסט מקליט..., כפתור סיום ההקלטה משתנה לירוק וכעת ניתן ללחוץ עליו.



כשהמשתמש מסיים לדבר ולחוץ על כפתור סיום ההקלטה, הכפתורים האחרים "מתעוררים" והוא רשאי להקשיב להקלטה שביצע, להקליט הקלטה חדשה אם ילחץ שוב על כפתור התחלת ההקלטה, או לשלוח את ההקלטה ושתהפך לספר דיגיטלי.



כשהמשתמש לוחץ על שליחה, החלון שמאפשר לו לתת שם לספר מופיע והתהליך שהסברתי עליו מקודם קורה עד שלבסוף מוצג למשתמש הספר כשהתוכן שלו זה ההקלטה שהקליט במעבדת ההקלטות.

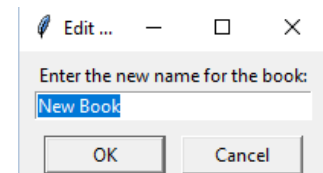
עכשיו אם המשתמש רוצה לפתוח אחד מהספרים שהוא יצר הוא יכול ללחוץ על כפתור הספרייה הדיגיטלית ויוצג לו המסך הבא:



כאן הוא יכול או לשנות שם לספר ספציפי שרוצה, בלחיצה על כפתור העריכת שם שמשתנה לירוק, ומופיע מתחתיו הסבר לאיך לשנות את השם של הספר.



המשתמש ילחץ על הספר שרוצה לשנות את שמו. ותופיע לו החלונית הבאה:

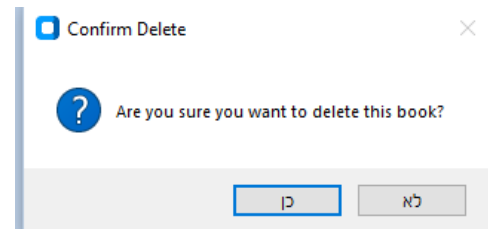


כאן הוא מכניס את השם החדש הרצוי, לוחץ על אישור והשם מתעדכן בספרייה הדיגיטלית. אם הוא מתחרט הוא יכול ללחוץ שנית על כפתור השנה שם, והוא יוציא את המשתמש ממוד שינוי שם של ספר.

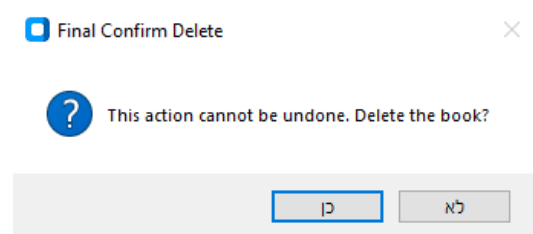
אם המשתמש רוצה למחוק ספר באופן סופי, הוא יכול ללחוץ על כפתור המחיקת ספר, שישנה את צבעו ויציג הסבר מתחתיו בצורה הזאת:



עכשיו המשתמש במוד מחיקת ספר, והוא יכול ללחוץ על הספר שרוצה למחוק ויוצגו לו הודעות הווידוי הבאות:



אם ילחץ כן תוצג לו ההודעה הנוספת:



אם ילחץ כן שנית, הספר ימחק סופית ולא יוצג בספרייה הדיגיטלית של המשתמש.

אם המשתמש רוצה לפתוח ספר ספציפי, הוא רשאי ללחוץ על שמו של הספר שרוצה לפתוח והספר יפתח עם התוכן שלו.



סיכום אישי / רפלקציה:**תיאור תהליך העבודה על הפרויקט:****ההצלחות, האתגרים, הקשיים ודרכי הפתרון**

במהלך השנה של עבודה על הפרויקט, אני נתקלתי ביותר אתגרים ממה שציפיתי. אני מכיתה י' כבר חושבת על רעיונות שונים לפרויקט, חלקם מוצלחים חלקם פחות. אבל בכל זאת, עד המסע לפולין לא הייתי סגורה בכלל מה אני אעשה. חשבתי שלבחור רעיון יהיה החלק הקשה, אבל ברגע שהתחלתי לכתוב את הקוד הבנתי שאני לא יודעת כיצד לגשת לזה אפילו. מלבד תקלות מפתיעות שצצו מדי פעם בקוד, להתחיל מאפס היה אחד מהאתגרים הקשים ביותר עבורי, והכנת סביבת העבודה שתעבוד אצלי בלפטופ בבית היה מאתגר לא פחות. אני זוכרת שהיה יום אחד שעבדתי שש שעות ברצף על הפרויקט ואז סביבת העבודה שלי קרסה ומחקה את כל העבודה הארוכה שעשיתי. להתאושש מזה היה ממש קשה, גם בגלל שכל ההתקדמות שלי נמחקה וגם בגלל שלא ידעתי מה הייתה הבעיה. אבל חקרתי הרבה בגוגל והחלטתי לחבר את הפרויקט שלי לגיטהאב שישמור את כל הגרסאות וזה פתר לי את בעיית סביבת העבודה. היה מאגר לעבוד על הפרויקט בזמן הלחץ של כיתה י"ב אבל הכנתי לי טבלה מסודרת של מה עליי להספיק בכל יום בכדי שיהיה מוכן בזמן, וזה מאוד עזר לי להיות מפוקסת ולהתקדם אתו. אני יכולה בביטחון להעיד שכל האתגרים הללו היו שווים לעבור מכיוון שעכשיו אני יודעת כיצד לפתור אותם אם אעשה פרויקטים עתידיים, ולראות את הקוד שיצרתי מאפס רץ ועובד בצורה שרציתי, זה חוויה של להפוך את הרעיון שלי למציאות וזה הצלחה שקשה להסביר והרגשה מצוינת ששווה את העבודה הקשה.

תיאור תהליך הלמידה:**תיאור הלמידה שהתקיימה, תכולות חדשות שנלמדו באופן עצמאי**

לא תיארתי לעצמי שאלמד כל כך הרבה מהפרויקט. הוא באמת העשיר לי את הידע במחשבים בכל תחום אפשרי, אם זה המושגים החדשים שלמדתי, הקיצורים המיוחדים שמשתמשים כדי לתאר אותם, איך לפתור בעיה שקוראת לי עם הקוד, איפה לחפש פתרונות, איך לסדר סביבת עבודה נוחה וכל הדברים הללו הם לא קשורים אפילו ללמידה שלי בפיתוח. העשרתי את הידע שלי בשפת קוד הזאת בצורה שהייתה יותר אפקטיבית ממה שציפיתי, ועם החופש לעבוד על הרעיון שלי לפרויקט ולהוסיף מה שבא לי אליו, למדתי דברים שבאמת סקרנו אותי. למדתי כמה סוגי הצפנות ואת החשיבות שלהן, עבודה עם API **ולמידה עצמית של AI**, כשהשתמשתי בממשק המתמלל אודיו של קלאודיניר שעושה זאת באמצעות בינה מלאכותית, כך שכל פעם שמשתמש מקליט אודיו או מכניס אודיו לאפליקציה שלי, הוא נהפך לטקסט באמצעות הבינה. הלמידה שלי הייתה בעיקר עצמאית, אך אם היו דברים שבכל זאת לא הצלחתי להבין לבד אחרי מחקר, לא היססתי לבקש עזרה מהמורים של המגמה והם עזרו לי מאוד.

כלים להמשך:**אילו כלים נלקחים להמשך**

כלי עיקרי שאני ייקח איתי לכל פרויקט שאעשה זה סנכרון הקוד שלי עם גיטהאב, אותו לא הייתי מכירה ומשתמשת בו אם לא היה הפרויקט, בנוסף עבודה עם API פתוח, באמצעות העבודה עם Cloudinary יכולתי להוסיף לפרויקט שלי AI שיהפוך את ההקלטות למלל אבל בעיקר אני חושבת שכל דבר חדש שלמדתי במשך עבודה על הפרויקט ילך איתי להמשך.

תובנות מהתהליך:

התובנה הכי גדולה שלי מהתהליך היא ללא ספק לא להירתע מבעיות שקורות לי בדרך. במהלך העבודה על הפרויקט, על הקוד, על הספר. נתקלתי בכל כך הרבה בעיות שלא ציפיתי אליהן וזה היה ממש מייאש בהתחלה. אבל ככל שצצו יותר בעיות ראיתי שאני יכולה לעבור אותן אחת אחת ולא צריכה לתת להן

להוריד לי את המוטיבציה. בנוסף, העבודה בסביבה בכיתה לצד חברות שלי הייתה ממש מרוממת, וכיפית. הרגשתי שגם כשכל אחת עבדה על הפרויקט שלה אנחנו מן צוות ביחד שמנסות להשלים את המשימה הגדולה הזאת. הן תמיד עזרו לי שביקשתי מהן עזרה ובעיקר תמכו בי כשלא היה לי כוח לעבוד על הפרויקט להתקדם אתו בכל זאת. עבודה קבוצתית היא תובנה משמעותית שהצלחתי לעכל לעומק בזכות הפרויקט.

בראייה לאחור:

האם הייתי מיישמת אחרת חלקים בפרויקט

בראייה לאחור, הייתי קודם כל מוודאת שסביבת העבודה שלי על הפרויקט מושלמת, לפני שאני כותבת קוד. לאחר מכן הייתי מציירת לי תכנון וויזואלי של איך אני רוצה שהתוצר הסופי יראה ולפיו עובדת על כל חלק בתורו. בעיקר, הייתי עובדת מסודר יותר מההתחלה של הפרויקט ושומרת על הסדר ככל שאני מתקדמת אתו.

משאבים נוספים:

במידה והיו ברשותי משאבים נוספים

אם היו ברשותי משאבים נוספים הייתי מוסיפה וובהוק שיושב חיצונית שאומר לי מתי התמלול לטקסט מוכן, והייתי משתמשת במשאבים לקנות מודל תמלול AI יותר חכם או אפילו מכינה אחד בעצמי מאפס, שיעשה את המטרה הספציפית של הפרויקט שלי.

שאלות חקר עצמי:

שאלות חקר עצמי לשיקול התלמידים:

כיצד אוכל לשפר את איכות התמלול והדיוק של הטקסט?
איך לשלב וובהוק?
איזו שיטת הצפנה נוספת יכולה לשפר את האבטחה?
כיצד להתמודד עם עומס גבוה של משתמשים בשרת?
איך
איך אני יכולה להפוך את הקוד לאפליקציה להורדה לטלפון או מחשב?
איך להפוך את הפרויקט לאתר?

תודות:

בראש ובראשונה תודה למורי המגמה שלוו אותי בתהליך הקשה של הכנת הפרויקט מאפס. זכיתי לעזרה עם הפרויקט גם מהמורים שלא לימדו את כיתתי במהלך השנים. אבל בעיקר, אני רוצה להגיד תודה שהמורים תמיד היו שם בשביל לעזור ולנתב את הפרויקט לתוצר הסופי, והתחשבו בתאריכים ובעומס. תודה גם לחברות שאיתי בכיתה, שעזרו לי לא פחות, והפכו את כל החוויה של עבודה על הפרויקט לכיפית, הן חלק אדיר בכך שיש לי תוצר סופי של פרויקט וספר שמוגשים ובלעדיהן לא הייתי מסוגלת לעשות את זה.

ביבליוגרפיה:

Cloudinary, from Image & Video APIs overview

https://cloudinary.com/documentation/programmable_media_overview

Unified Modeling Language (UML) Diagrams

<https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/>

המרכז הלאומי לסייבר חינוכי רשתות תקשורת

<https://data.cyber.org.il/networks/networks.pdf>

method for obtaining digital signatures and public-key

<https://dl.acm.org/doi/10.1145/359340.359342>

Python Software Foundation Python 3.12.1 documentation.

<https://docs.python.org/3/>

Hipp, D. R. (2000) SQLite documentation. SQLite Consortium

<https://www.sqlite.org/docs.html>

Tkinter Team. tkinter — Python interface to Tk. In Python Standard Library

<https://docs.python.org/3/library/tkinter.html>

Pickle module documentation. Python Standard Library.

<https://docs.python.org/3/library/pickle.html>

PyCryptodome 3.17 Documentation

<https://pycryptodome.readthedocs.io/en/latest/>

ReportLab User Guide

<https://www.reportlab.com/docs/reportlab-userguide.pdf>

Cloudinary

<https://cloudinary.com/>

RSA Algorithm in Cryptography geeksforgeeks

<https://www.geeksforgeeks.org/rsa-algorithm-cryptography/>

Advanced Encryption Standard (AES) geeksforgeeks

<https://www.geeksforgeeks.org/advanced-encryption-standard-aes/>

Create a Voice Recorder using Python geeksforgeeks

<https://www.geeksforgeeks.org/create-a-voice-recorder-using-python/>

```

import json
import struct
import rsa
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

import string
import random

BUFFER_SIZE = 4096 # 4KB buffer size for file transfer

# Client                                Server
# |                                     |
# |----- Connect to server ---->|
# |                                     |
# |<--- Send RSA public key -----|
# |                                     |
# |-- Encrypt & send AES key --->|  client locks (Encrypt) AES key with RSA
public key and sends it to the Server
# |                                     |
# |<-- Decrypt AES key -----|  server unlocks (Decrypt) AES key with its
RSA private key
# |                                     |
# |                                     |  Now both Client and Server have the same
AES key
# |==== Encrypted messages =====>|  Secure communication begins: Client sends
messages encrypted with AES key, Server replies with encrypted responses
# |<=== Encrypted responses =====|

class AESComm:
    def __init__(self,socket,key):
        self.client_socket = socket # Store the socket for communication
        self.aes_key = key # Store the AES encryption key

    def recv_exact(self, num_bytes):
        # פונקציה לקבלת כמות מדויקת של בתים מהסוקט
        data = b''
        while len(data) < num_bytes:
            packet = self.client_socket.recv(num_bytes - len(data))

```

```

        if not packet:
            raise ConnectionError("Socket closed while receiving data.")
        data += packet
    return data

def recv_header(self):
    # Receive the fixed-size header - מקבל את כותרת ההודעה (nonce + tag + length)
    header = self.recv_exact(12 + 16 + 4) # 12 bytes for nonce, 16 bytes for
tag, 4 bytes for length
    return header

def encrypt_payload(self, request, binary=False):
    if not binary:
        request = request.encode()
        aes_nonce = get_random_bytes(12) # Generate a 12-byte random
nonce (Number used once ערך Nonce כדי למנוע התקפות חוזרות. כל פעם שנשלחת הודעה מוצפנת, נוצר
חדש. for AES - (אקראי שנוצר במיוחד עבור פעולה אחת בלבד (למשל, הצפנה אחת)
        aes_cipher = AES.new(self.aes_key, AES.MODE_GCM, aes_nonce) # Create AES-
GCM cipher object with the AES key and nonce
        ciphertext, tag = aes_cipher.encrypt_and_digest(request) # Encrypt and
get tag- ערך קצר שנוצר בזמן ההצפנה ומתווסף להודעה, תג אימות
        # Send: nonce + tag + len + ciphertext- Combine nonce, tag, length of
ciphertext, and ciphertext into one payload
        payload = aes_cipher.nonce + tag + struct.pack("!I", len(ciphertext)) +
ciphertext
    return payload

def decrypt_payload(self, header, binary=False):
    # Extract components from the header
    nonce = header[:12] # first 12 bytes: nonce
    tag = header[12:28] # next 16 bytes: tag
    ciphertext_length = struct.unpack("!I", header[28:32])[0] # next 4 bytes:
length of ciphertext
    # print("ciphertext len:", ciphertext_length)

    # Receive the encrypted message body
    ciphertext = self.recv_exact(ciphertext_length)

    # Decrypt the message using AES-GCM, AES (Advanced Encryption Standard)
GCM (Galois/Counter Mode)
    aes_cipher = AES.new(self.aes_key, AES.MODE_GCM, nonce=nonce)
    decrypted_message = aes_cipher.decrypt_and_verify(ciphertext, tag)
    if not binary:
        decrypted_message = decrypted_message.decode()
    return (decrypted_message)

```



```

def recv_message(self):
    # Receive and decrypt a full AES-encrypted message
    try:
        header = self.recv_header() # Get the header חלק של המידע שנשלח שמכיל מידע
        # תכניי חשוב לגבי הדרך שבה יש לפענח את שאר המידע או לארגן אותו
        message = self.decrypt_payload(header) # Decrypt the rest of the
        # message using the header
        # print("Got message ",message)
        return message

    except Exception as e:
        print(f"Error recieving : {e}")
        return json.dumps({"status": "error", "message": str(e)})

def recv_binary(self):

    try:
        binary = True
        header = self.recv_header()
        data = self.decrypt_payload(header,binary)
        # print("Got binary data")
        return data

    except Exception as e:
        print(f"Error recieving binary: {e}")
        return json.dumps({"status": "error", "message": str(e)})

def send_message(self, message):
    # Send a request to the client, encrypting the request with the AES
    try:
        payload = self.encrypt_payload(message) # Encrypt message into bytes
        self.client_socket.sendall(payload) # Send the encrypted data
    except Exception as e:
        print(f"Error sending: {e}")
        return json.dumps({"status": "error", "message": str(e)})

def send_binary(self, data):

    try:
        binary = True
        payload = self.encrypt_payload(data,binary)
        self.client_socket.sendall(payload)
    except Exception as e:
        print(f"Error sending binary: {e}")

```

```

        return json.dumps({"status": "error", "message": str(e)})

class AESCommServer(AESComm):

    def send_server_response(self, response):
        # Send a response to the client as a JSON string, encrypted with AES
        try:
            # print("sending to client...",response)
            message = json.dumps(response)
            self.send_message(message)
        except Exception as e:
            print(f"Error preparing server message: {e}")
            return {"status": "error", "message": str(e)}

    def add_random_suffix(self, filename, length=6):
        # print("add random suffix to file name",filename)
        random_suffix = ''.join(random.choices(string.ascii_letters +
string.digits, k=length))
        base, ext = filename.rsplit('.', 1)
        return f"{base}_{random_suffix}.{ext}"

    def recv_file(self, file):

        try:
            file_name = self.add_random_suffix(file)
            with open(file_name, 'wb') as f:
                while True:
                    # print("calling recv_binary")
                    data = self.recv_binary()
                    if not data:
                        break
                    f.write(data)
            print("file ended")
            return file_name

        except Exception as e:
            print(f"Error recieving file content: {e}")
            return {"status": "error", "message": str(e)}

class AESCommClient(AESComm):

    def __init__(self,socket):
        aes_key = get_random_bytes(32) # Generate a secure 32-byte AES key

```

```

    super().__init__(socket,aes_key) # Initialize base class with socket and
key

def send_aes_key(self,server_public_key):
    # Encrypt the AES key with the server's public RSA key and send it
    encrypted_aes_key = rsa.encrypt(self.aes_key, server_public_key)
    self.client_socket.sendall(encrypted_aes_key)
    print("AES key encrypted and sent to server.")

def send_client_command(self, command, data):
    # Send a command and data as JSON, encrypted with AES
    try:
        # print("sending to server...")
        message = json.dumps({"command": command, "data": data})
        self.send_message(message)
    except Exception as e:
        print(f"Error preparing client message: {e}")
        return {"status": "error", "message": str(e)}

def send_file(self, file):

    try:
        "start to send file server"
        with open(file, 'rb') as f:
            while True:
                bytes_read = f.read(BUFFER_SIZE)
                if len(bytes_read) > 0:
                    self.send_binary(bytes_read)
                else:
                    break
        # print("Sending file completed")
    except Exception as e:
        print(f"Error sending file content: {e}")
        return {"status": "error", "message": str(e)}

```

```

import sqlite3
import bcrypt

class ManageDB :
    def __init__(self, database):
        self.db_name = database

    def hash_password(self, password):
        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
        return hashed.decode('utf-8')

    def verify_password(self, plain_password, hashed_password):
        # print("verify_password - plain_password:",plain_password)
        # print("verify_password - hashed_password:",hashed_password)
        try:
            login_ok = bcrypt.checkpw(plain_password.encode(),
hashed_password.encode())
        except Exception as e:
            print("Error in verify_password:", e)
            return False
        return login_ok

    def login_user(self, data):
        # Handle user login
        username = data.get("username")
        password = data.get("password")
        # print("login_user - username:",username)
        # print("login_user - password:",password)

        if not username or not password:
            return {"status": "error", "message": "Username and password
required."}

        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT user_password FROM USER_INFO WHERE user_name =
?", (username,))
            result = cursor.fetchone()

            if result:
                stored_hash = result[0]
                if self.verify_password(password, stored_hash):
                    # print("Login successful")

```

```

        return {"status": "success", "message": "Login successful."}

    print("Login failed")
    return {"status": "error", "message": "Invalid username or
password."}

def create_account(self, data):
    """
    Handle account creation.
    """
    username = data.get("username")
    password = data.get("password")
    if not username or not password:
        return {"status": "error", "message": "Username and password
required."}
    # print("create_account - username:",username)
    # print("create_account - password:",password)
    hashed_password = self.hash_password(password)
    # print("create_account - hashed_password:",hashed_password)
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("CREATE TABLE IF NOT EXISTS USER_INFO (user_id INTEGER
PRIMARY KEY, user_name TEXT, user_password TEXT)")
        cursor.execute("SELECT * FROM USER_INFO WHERE user_name = ?",
(username,))
        if cursor.fetchone():
            return {"status": "error", "message": "Username already exists."}
        cursor.execute("INSERT INTO USER_INFO (user_name, user_password)
VALUES (?, ?)", (username, hashed_password))
        conn.commit()
        return {"status": "success", "message": "Account created
successfully."}

def load_book_from_db(self, data):
    """
    Load book data from the database using the provided book_id.
    Args:
        data: book_id: The ID of the book to load.
    Returns:
        data: (book_title, author, pages)
    """

    book_id = data.get("book_id")
    if not book_id:

```

```

        return {"status": "error", "message": "Missing data for fetching
book data."}

    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT NAME, TEXT, OWNER FROM BOOKS WHERE ID = ?",
(book_id,))
        result = cursor.fetchone()
        if not result:
            return {"status": "error", "message": "Book not found."}
        book_title, text, owner_id = result
        cursor.execute("SELECT user_name FROM USER_INFO WHERE user_ID = ?",
(owner_id,))
        author_result = cursor.fetchone()
        if not author_result:
            return {"status": "error", "message": "Owner not found."}

        author = author_result[0]
        return {"status": "success", "books_title": book_title, "author":
author, "text": text}

def save_text_changes(self, data):
    """
    Save changes to the book text in the database.
    Args:
        data: book_id: The ID of the book to update.
            new_text: The new text content for the book.
    Returns:
        status: A message indicating success or failure.
    """
    book_id = data.get("book_id")
    updated_text = data.get("updated_text")

    # print("save_text_changes - book_id:",book_id)

    if not book_id or not updated_text:
        return {"status": "error", "message": "Missing data to save
changes."}

    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("UPDATE BOOKS SET TEXT = ? WHERE ID = ?",
(updated_text, book_id))
        conn.commit()

```

```

        return {"status": "success", "message": "Changes saved
successfully."}

def get_book_name(self, data):
    """
    Allow the user to change the book's name.
    After updating the database, refresh the digital library.
    """
    # Fetch the current name as a default
    current_name = None
    book_id = data.get("book_id")
    try:
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT NAME FROM BOOKS WHERE ID = ?", (book_id,))
            result = cursor.fetchone()
            if result:
                current_name = result[0]
                return {"status": "success", "book_name": current_name}
            else:
                return {"status": "error", "message": "Book not found."}

    except Exception as e:
        return {"status": "Error", "message": "An error occurred while fetching
the current book name"}

def update_book_name(self, data):
    book_id = data.get("book_id")
    new_name = data.get("new_name")
    try:
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("UPDATE BOOKS SET NAME = ? WHERE ID = ?",
(new_name, book_id))
            conn.commit()
            return {"status": "success", "message": "Book name updated
successfully."}
    except Exception as e:
        return {"status": "Error", "message": "An error occurred while updating
the book name"}

def submit_book(self, data):
    username = data.get("username")

```

```

book_name = data.get("book_name")
transcription_text = data.get("transcription_text")
try:
    # Save the book to the database
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()

        # Check if the username exists in the database
        cursor.execute("SELECT user_ID FROM USER_INFO WHERE user_name =
?", (username,))
        user_result = cursor.fetchone()
        user_id = user_result[0] # Extract the user ID

        if not book_name or not transcription_text or not user_id:
            return {"status": "error", "message": "Missing data to submit
book data."}

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS BOOKS (
                ID INTEGER PRIMARY KEY AUTOINCREMENT,
                NAME TEXT NOT NULL,
                TEXT TEXT NOT NULL,
                OWNER INTEGER NOT NULL,
                FOREIGN KEY (OWNER) REFERENCES USER_INFO(user_ID)
            )
        """)
        cursor.execute("INSERT INTO BOOKS (NAME, TEXT, OWNER) VALUES (?,
?, ?)",
                        (book_name, transcription_text, user_id))
        conn.commit()

        # Fetch the last inserted book ID
        book_id = cursor.lastrowid

        return {"status": "success", "book_id": book_id}

except Exception as e:
    return {"status": "Error", "message": "An error occurred while
submitting the book data"}

def delete_book(self, data):

    book_id = data.get("book_id")

```



```

    if not book_id:
        return {"status": "error", "message": "Missing book ID to delete."}
    try:
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("DELETE FROM BOOKS WHERE ID = ?", (book_id,))
            conn.commit()
    except Exception as e:
        return {"status": "error", "message": "An error occurred while deleting the book"}

    return {"status": "success", "message": "Book deleted."}

def get_user_id(self, data):

    user_name = data.get("username")
    if not user_name:
        return {"status": "error", "message": "Missing user name."}

    # Connect to the database
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()

        # Fetch the user ID using the username
        cursor.execute("SELECT user_ID FROM USER_INFO WHERE user_name = ?",
            (user_name,))
        user_result = cursor.fetchone()

        if not user_result:
            return (-1)

        user_id = user_result[0]

        return {"status": "success", "user_id": user_id}

def get_user_name(self, data):

    user_id = data.get("user-id")
    if not user_id:
        return {"status": "error", "message": "Missing user name."}

    # Connect to the database
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()

```

```

        # Fetch the user ID using the username
        cursor.execute("SELECT user_name FROM USER_INFO WHERE user_ID = ?",
(user_id,))
        user_result = cursor.fetchone()

        if not user_result:
            return (-1)

        user_name = user_result[0]

        return user_name

    def get_books_list(self, data):

        user_id = data.get("user_id")
        if not user_id:
            return {"status": "error", "message": "Missing user data to get
books list."}
        # Connect to the database
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            # Fetch all books associated with the user ID
            cursor.execute("SELECT ID, NAME FROM BOOKS WHERE OWNER = ?",
(user_id,))
            books = cursor.fetchall()

            if not books:
                return (0)

            return {"status": "success", "books": books}

```

```

import socket
import threading
import json
import clouinary
import clouinary.uploader
import clouinary.api
import time
import keyring
from urllib.request import urlopen
import rsa # Added for RSA encryption
from AES_Communication import AESCommServer
from Echo_ManageDB import ManageDB

# Server connection details
SERVER_IP = '127.0.0.1' # Localhost (can be changed to a real IP)
SERVER_PORT = 1729 # Port number for the server to listen on
MAX_MESSAGE_LENGTH = 4096 # Max message size

#Set up Clouinary credentials securely using keyring
clouinary.config(
    cloud_name=keyring.get_password("Echo", "cloud_name"),
    api_key=keyring.get_password("Echo", "api_key"),
    api_secret=keyring.get_password("Echo", "api_secret")
)

# Class to handle digital book creation from audio files
class DigitalBookHandler:
    def __init__(self):
        pass

    def create_digital_book(self, audio_file):
        """
        Create a digital book from the provided audio file.
        Returns a dictionary with status and transcription or an error message.
        """
        try:
            # Upload the audio file to Clouinary for processing
            #Clouinary version
            res = clouinary.uploader.upload(audio_file, resource_type="video",
            upload_preset="audio2text")

            #Google version

```

```

        #res = cloudinary.uploader.upload(audio_file, resource_type="video",
upload_preset="audio2text-ggl")

        #print("Cloudinary upload response:", res)
        public_id = res["public_id"]

        # Wait for Cloudinary processing- to finish transcription
        self.wait_for_transcription(public_id)

        # Construct the transcription URL- Generate a secure link to download
the transcript
        # transcript_url = res["secure_url"].replace("video", "raw")[:-3] +
"transcript"
        # print("Transcription URL:", transcript_url)
        signed_url, options = cloudinary.utils.cloudinary_url(public_id +
".transcript",
                                                                resource_type="
raw",
                                                                type =
"authenticated",
                                                                sign_url=True )

        # Request transcription JSON from Cloudinary. Download and parse the
transcription JSON
        response = urlopen(signed_url)
        data_json = json.loads(response.read())
        # print("Transcription JSON:", data_json)

        # Extract transcript text from each segment
        transcript = ""
        # Extract the transcript
        for item in data_json:
            # print("Item: ",item)
            # print("transcript of item: ",item['transcript'])
            transcript += item['transcript']

        return {"status": "success", "transcription": transcript}

    except Exception as e:
        return {"status": "error", "message": str(e)}

    def wait_for_transcription(self, public_id):

        # Wait for Cloudinary processing- Wait until the transcription file
is ready

```

```

        print("Waiting for transcription...")
        bytes = 0
        times = 0
        while(bytes == 0 and times < 30):
            time.sleep(10) # Wait for 10 seconds before checking again
            times += 1
            result = clouddinary.api.resource(public_id+".transcript",
resource_type="raw", type = "authenticated")
            bytes = result['bytes'] # Check if the file size is greater than
0

            # print("Bytes:", bytes)
            # print("Times:", times)

# EchoServer class handles client connections and requests
class EchoServer:
    def __init__(self):
        # Set up server socket and listen for connections
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.server_socket.settimeout(1) # Timeout to avoid blocking
        self.server_socket.bind((SERVER_IP, SERVER_PORT))
        self.server_socket.listen(5)
        print(f"Server started on {SERVER_IP}:{SERVER_PORT}")
        self.digital_book_handler = DigitalBookHandler()
        self.running = True

        # Generate RSA key pair (1024 bits) for secure AES key exchange
        self.public_key, self.private_key = rsa.newkeys(1024)
        self.db = ManageDB("echo_db.db") # Initialize database manager

        # DDoS Protection variables
        self.request_tracker = {} # Dictionary to track request timestamps per
IP
        self.blocked_ips = {} # Dictionary to track blocked IPs with
expiration timestamp
        self.REQUEST_LIMIT = 50 # Max requests per IP per minute
        self.BLOCK_TIME = 60 # Block time in seconds

    def defend_against_ddos(self, ip_address):
        """
        Prevent DDoS attacks by tracking and limiting request rates per IP.
        If an IP exceeds REQUEST_LIMIT within 60 seconds, block it temporarily.
        """
        current_time = time.time()
        # Check if IP is already blocked
        if ip_address in self.blocked_ips:

```

```

        if current_time < self.blocked_ips[ip_address]:
            print(f"[DDoS PROTECTION] Blocked request from {ip_address}.")
            return False
        else:
            del self.blocked_ips[ip_address]
            # Initialize tracker for IP if not present
            if ip_address not in self.request_tracker:
                self.request_tracker[ip_address] = []
            # Remove timestamps older than 60 seconds
            self.request_tracker[ip_address] = [timestamp for timestamp in
self.request_tracker[ip_address] if current_time - timestamp < 60]
            # Add current request timestamp
            self.request_tracker[ip_address].append(current_time)
            # Check if request count exceeds limit
            if len(self.request_tracker[ip_address]) > self.REQUEST_LIMIT:
                print(f"[DDoS ALERT] Too many requests from {ip_address}. Blocking
temporarily.")
                self.blocked_ips[ip_address] = current_time + self.BLOCK_TIME
                return False
            return True

def get_client_key(self, client_socket):
    enc_aes_key = client_socket.recv(256) # Encrypted AES key
    aes_key = rsa.decrypt(enc_aes_key, self.private_key) # Decrypt using
server's private key
    return aes_key

def handle_client(self, client_socket):
    """
    Handle client requests in a separate thread.
    First, send the server's public key. Then, for each incoming message,
    decrypt it using RSA.
    """
    try:
        # Send the server's public key to the client (in PEM format)- Send
RSA public key to client
        client_socket.send(self.public_key.save_pkcs1())
    except Exception as e:
        print("Error sending public key:", e)
        client_socket.close()
        return

    # Receive AES key from client and initialize AES communication
    aes_key = self.get_client_key(client_socket)

```

```

print("AES key received from client")

aes_comm_server = AESCommServer(client_socket, aes_key)

while True:
    # try:
    #     # Receive the encrypted message from the client
    #     encrypted_message = client_socket.recv(MAX_MESSAGE_LENGTH)
    #     if not encrypted_message:
    #         break
    #
    #     # Decrypt the message using the server's private key
    #     try:
    #         decrypted_message = rsa.decrypt(encrypted_message,
self.private_key)
    #         message = decrypted_message.decode()
    #     except Exception as e:
    #         print("Decryption error:", e)
    #         continue
    #
    #     request = json.loads(message)
    #     command = request.get("command")
    #     data = request.get("data")
    #
    #     # Receive and decrypt the message using AES
    #
    # try:
    #     # Receive and decrypt message from client
    #     message = aes_comm_server.recv_message()
    #     if not message:
    #         # print("Client disconnected")
    #         break # No message means client disconnected
    #
    #     # Decode JSON message
    #     request = json.loads(message)
    #     if "command" not in request:
    #         print("Invalid request format", request)
    #         break
    #
    #     # Extract command and data
    #     command = request.get("command")
    #     data = request.get("data")
    #
    #     # Process client command
    #     if command == "login":

```

```

        response = self.db.login_user(data)
    elif command == "create_account":
        response = self.db.create_account(data)
    elif command == "upload_audio":
        response = self.upload_audio(data)
    elif command == "get_user_id":
        response = self.db.get_user_id(data)
    elif command == "get_books_list":
        response = self.db.get_books_list(data)
    elif command == "load_book_from_db":
        response = self.db.load_book_from_db(data)
    elif command == "get_book_name":
        response = self.db.get_book_name(data)
    elif command == "update_book_name":
        response = self.db.update_book_name(data)
    elif command == "save_text_changes":
        response = self.db.save_text_changes(data)
    elif command == "delete_book":
        response = self.db.delete_book(data)
    elif command == "submit_book":
        response = self.db.submit_book(data)
    elif command == "create_digital_book":
        audio_file = data.get("audio_file")
        response =
self.digital_book_handler.create_digital_book(audio_file)
    else:
        response = {"status": "error", "message": "Unknown command.
"+command}

    # Send back response to client
    aes_comm_server.send_server_response(response)
# except ConnectionResetError:
#     print("Client disconnected unexpectedly.")
#     break
except Exception as e:
    print(f"Error handling client: {e}")
    break
# finally:
#     client_socket.close()
#     print("Connection closed.")

# def login_user_old(self, data):
#     username = data.get("username")
#     password = data.get("password")

```



```

# with sqlite3.connect("echo_db.db") as conn:
#     cursor = conn.cursor()
#     cursor.execute("SELECT * FROM USER_INFO WHERE user_name = ? AND
user_password = ?", (username, password))
#     result = cursor.fetchone()
#     if result:
#         return {"status": "success", "message": "Login successful."}
#     else:
#         return {"status": "error", "message": "Invalid username or
password."}

# def create_account_old(self, data):

#     username = data.get("username")
#     password = data.get("password")
#     with sqlite3.connect("echo_db.db") as conn:
#         cursor = conn.cursor()
#         cursor.execute("CREATE TABLE IF NOT EXISTS USER_INFO (user_id
INTEGER PRIMARY KEY, user_name TEXT, user_password TEXT)")
#         cursor.execute("SELECT * FROM USER_INFO WHERE user_name = ?",
(username,))
#         if cursor.fetchone():
#             return {"status": "error", "message": "Username already
exists."}
#         cursor.execute("INSERT INTO USER_INFO (user_name, user_password)
VALUES (?, ?)", (username, password))
#         conn.commit()
#         return {"status": "success", "message": "Account created
successfully."}

def upload_audio(self, data):
    """
    Handle audio file upload.
    """
    filename = data.get("filename")
    try:
        res = cloudinary.uploader.upload(filename, resource_type="video",
upload_preset="audio2text")
        time.sleep(10) # Wait for processing
        transcript_url = res["secure_url"].replace("video", "raw")[:-3] +
"transcript"
        response = urlopen(transcript_url)
        transcript_data = json.loads(response.read())
        transcript = transcript_data[0]["transcript"]
        return {"status": "success", "transcript": transcript}

```

```

except Exception as e:
    return {"status": "error", "message": str(e)}

def start(self):
    """
    Start the server and accept client connections.
    """
    try:
        while self.running:
            try:
                # Accept new client connection
                client_socket, client_address = self.server_socket.accept()
                # DDoS Protection: if client IP exceeds request limit, skip
                handling
                if not self.defend_against_ddos(client_address[0]):
                    client_socket.close()
                    continue
                # Handle client in a new thread
                threading.Thread(target=self.handle_client,
                                args=(client_socket,)).start()
            except socket.timeout:
                continue # No connection, keep looping
        except KeyboardInterrupt:
            print("Server shutting down...")
            self.running = False
            self.server_socket.close()
            exit(0)

# Start the server
if __name__ == "__main__":
    server = EchoServer()
    server.start()

```

```

from Echo_LogIn import LoginWindow
from Echo_HomePage import Homepage
from Echo_DigitalFlipbook import DigitalFlipbookApp
from Echo_RecordingLab import RecordingLab
import socket
import json
import tkinter.messagebox as tkmb
from urllib.request import urlopen
import rsa # Added for RSA encryption
from AES_Communication import AESCommClient

# Server connection details
SERVER_IP = '127.0.0.1'
SERVER_PORT = 1729
MAX_MESSAGE_LENGTH = 4096

# #Network availability check
# def is_server_reachable(ip, port, timeout=2):
#     try:
#         with socket.create_connection((ip, port), timeout=timeout):
#             return True
#     except Exception:
#         return False

# EchoClient class handles communication with the server
class EchoClient:
    def __init__(self):
        # Create a socket for TCP (Transmission Control Protocol) communication
        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.connected = False # Connection status
        self.server_public_key = None # Will store server's RSA public key after
        connection
        self.aes_client = AESCommClient(self.client_socket) # Setup AES
        encrypted communication

    def connect_to_server(self):
        # Attempt to connect to the server
        try:
            self.client_socket.connect((SERVER_IP, SERVER_PORT)) # Connect using
            IP and port
            self.connected = True

```

```

        # Receive the server's public RSA key (PEM(Privacy-Enhanced Mail)
format)
        public_key_data = self.client_socket.recv(4096)
        self.server_public_key = rsa.PublicKey.load_pkcs1(public_key_data)
        print("Connected to the server and received public key!")

        # Send the AES key securely using RSA encryption
        self.aes_client.send_aes_key(self.server_public_key)
        print("AES key sent to server.")

    except ConnectionRefusedError:
        # If connection fails, show an error popup
        print("Failed to connect to the server.")
        tkmb.showerror("Connection Error", "Failed to connect to the
server.")

        self.connected = False

def send_request(self, command, data):
    # Send a command and data to the server securely using AES encryption
    if not self.connected:
        return {"status": "error", "message": "Not connected to the server."}
    try:
        # Send command using AES encryption
        # self.client_socket.settimeout(5) # communication timeout- Set a
timeout for the socket operations
        self.aes_client.send_client_command(command,data)

        # Wait and receive response from server
        response = self.aes_client.recv_message()
        # print("Received response from server:", response)
        return json.loads(response)
    except Exception as e:
        # print(f"Error sending request: {e}")
        return {"status": "error", "message": str(e)}

def close_connection(self):
    # Close the connection to the server
    # try:
    #     self.client_socket.shutdown(socket.SHUT_RDWR) # safe disconnect
    # except Exception:
    #     pass
    self.client_socket.close() # Close the socket
    self.connected = False

# Main function that starts the client application

```

```
def main():
    # Check network before attempting connection
    # if not is_server_reachable(SERVER_IP, SERVER_PORT):
    #     tkmb.showerror("Network Error", "Server is unreachable. Check your
network connection.")
    #     return

    # Create the client instance
    client = EchoClient()
    client.connect_to_server() # Try to connect to server

    # Start the login GUI window with the connected client
    login_window = LoginWindow(client)
    login_window.run() # Start the GUI event loop

# Entry point for the script
if __name__ == "__main__":
    main()
```

```

import customtkinter
import tkinter.messagebox as tkmb
import random
import string
import time

from Echo_HomePage import Homepage # Import the Homepage class
from Echo_LandingPage import LandingPage # Import the landing page class

def is_strong_password(password):
    """
    Check if the given password is strong.
    A strong password should be at least 8 characters long,
    contain at least one uppercase letter, one lowercase letter,
    one digit, and one special character.
    """
    # Must be at least 8 characters and contain upper, lower, digit, and special
    character
    if len(password) < 8:
        return False
    if not any(c.isupper() for c in password):
        return False
    if not any(c.islower() for c in password):
        return False
    if not any(c.isdigit() for c in password):
        return False
    if not any(c in "!@#$%^&*()-_+= " for c in password):
        return False
    return True

# Function to generate a strong random password
def suggest_strong_password(min_length=12):
    """
    Suggest a strong password of at least min_length characters.
    Guarantees at least one uppercase letter, one lowercase letter,
    one digit, and one special character.
    """
    uppercase = random.choice(string.ascii_uppercase)
    lowercase = random.choice(string.ascii_lowercase)
    digit = random.choice(string.digits)
    special = random.choice("!@#$%^&*()-_+= ")
    # Generate the remaining characters from all allowed characters.
    remaining = ''.join(random.choices(string.ascii_letters + string.digits +
"!@#$%^&*()-_+= ", k=min_length - 4))

```

```

password = uppercase + lowercase + digit + special + remaining
password_list = list(password)
random.shuffle(password_list) # Shuffle to ensure randomness
return ''.join(password_list)

class LoginWindow:
    def __init__(self, client):
        """Initialize the LoginWindow."""
        self.client = client # Store client connection object
        self.app = customtkinter.CTk()
        self.app.geometry("800x600")
        self.app.title("Echo")
        self.app.configure(bg='#343A40')
        self.create_widgets()

        # Global counter for failed attempts and a timestamp to block further
        attempts
        self.global_failed_attempts = 0
        self.blocked_until = None # Timestamp until which login is blocked
        self.show_password = False # Password visibility toggle

    def create_widgets(self):
        """Create widgets for the login window."""
        # Header Frame
        l_frame = customtkinter.CTkFrame(
            master=self.app,
            border_color="#6B7082",
            border_width=2,
            fg_color="#EAE6F0",
            corner_radius=20
        )
        l_frame.pack(side="top", padx=30, pady=20, fill="both")
        label = customtkinter.CTkLabel(
            master=l_frame,
            text="Echo",
            font=("Palatino Linotype", 50, "bold"),
            text_color="#C4C1E0"
        )
        label.pack(pady=10)

        # Login Form Frame
        frame = customtkinter.CTkFrame(
            master=self.app,
            border_color="#A1A1C0",
            border_width=2,

```

```

        fg_color="#BFD3EB",
        corner_radius=20
    )
    frame.pack(side="top", pady=20, padx=20)

    # Title Label Frame
    t_frame = customtkinter.CTkFrame(
        master=frame,
        border_color="#A1C2D1",
        border_width=2,
        fg_color="#BFD3EB",
        corner_radius=20
    )
    t_frame.pack(pady=20, padx=20, side="top")
    title_label = customtkinter.CTkLabel(
        master=t_frame,
        text="Log in with your username and password",
        font=("Trebuchet MS", 24),
        text_color="#343A40"
    )
    title_label.pack(pady=10, padx=10)

    # Username input field
    self.user_n = customtkinter.CTkEntry(
        master=frame,
        width=295,
        placeholder_text="Username",
        font=("Trebuchet MS", 18),
        text_color="#343A40",
        border_color="#A1A1C0",
        placeholder_text_color="#C8C8C8",
        fg_color="#EAE6F0"
    )
    self.user_n.pack(pady=12, padx=10)

    # Password entry frame (for eye icon + field)
    password_frame = customtkinter.CTkFrame(
        master=frame,
        fg_color="#BFD3EB",
        border_width=0
    )
    password_frame.pack(pady=12, padx=10)

    # Password field (hidden by default)
    self.user_p = customtkinter.CTkEntry(

```



```

        master=password_frame,
        width=250,
        placeholder_text="Password",
        show="*",
        font=("Trebuchet MS", 18),
        text_color="#343A40",
        border_color="#A1A1C0",
        placeholder_text_color="#C8C8C8",
        fg_color="#EAE6F0"
    )
    self.user_p.pack(side="left") # Pack side-by-side

    # Eye-toggle button to show/hide the password
    self.toggle_btn = customtkinter.CTkButton(
        master=password_frame,
        text="👁", # Eye icon (using an emoji)
        width=40,
        command=self.toggle_password_visibility
    )
    self.toggle_btn.pack(side="left", padx=5)

    # Create Account Button
    new_account_button = customtkinter.CTkButton(
        master=frame,
        text="Create New Account",
        command=self.create_new_account_window,
        hover_color="#EAE6F0",
        text_color="#343A40",
        font=("Trebuchet MS", 12, "underline")
    )
    new_account_button.pack(pady=12, padx=10)

    # Login Button
    login_button = customtkinter.CTkButton(
        master=frame,
        text="Login",
        command=self.user_log_in,
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        font=("Trebuchet MS", 12)
    )
    login_button.pack(pady=12, padx=10)

    # Toggle password visibility

```

```

def toggle_password_visibility(self):
    """Toggle the password entry's visibility without changing layout
    dimensions."""
    if self.show_password:
        # Mask the password and set icon to the open eye
        self.user_p.configure(show="*") # Hide password
        self.toggle_btn.configure(text="👁")
        self.show_password = False
    else:
        # Unmask the password and set icon to a closed eye
        self.user_p.configure(show="") # Show password
        self.toggle_btn.configure(text="🙈")
        self.show_password = True

# Handle user login
def user_log_in(self):
    """
    Use the server to log in the user.
    Limits wrong attempts to prevent bots. If more than 5 wrong attempts
    occur,
    block login for 30 seconds.
    """
    # Check if login is blocked due to too many failed attempts
    if self.blocked_until is not None and time.time() < self.blocked_until:
        remaining = int(self.blocked_until - time.time())
        tkmb.showerror("Blocked", f"Too many failed attempts. Please try
again in {remaining} seconds.")
        return

    try:
        # Retrieve the username and password
        username = self.user_n.get().strip()
        password = self.user_p.get().strip()

        if not username or not password:
            tkmb.showerror("Login Failed", "Username and Password cannot be
empty.")
            return

        # Prepare the login request for the server.
        command = "login"
        data = {"username": username, "password": password}

        # Send the request using the EchoClient.
        response = self.client.send_request(command, data)

```

```

        if response.get("status") == "success":
            self.app.destroy() # Close the Login Window.
            landingpage = LandingPage(self.client, username=username)
            landingpage.run()
            self.global_failed_attempts = 0 # Reset counter on success.
        else:
            self.global_failed_attempts += 1
            tkmb.showerror("Login Failed", response.get("message", "Invalid
username or password. "))
            # If more than 5 failed attempts, block login for 30 seconds.
            if self.global_failed_attempts >= 5:
                self.blocked_until = time.time() + 30
    except Exception as e:
        tkmb.showerror("Error", f"An error occurred during login: {e}")

# Handle new account creation
def create_new_account_window(self):
    """Open the create new account window."""
    def submit_new_account():
        username = new_user_name_entry.get().strip()
        password = new_user_password_entry.get().strip()

        # Clear previous messages
        error_label.configure(text="")
        success_label.configure(text="")

        if not username or not password:
            error_label.configure(text="Username and Password cannot be
empty", text_color="#FF6B6B")
            return

        # Enforce strong password criteria.
        if not is_strong_password(password):
            error_label.configure(
                text="Password too weak. It must be at least 8 characters
long and include uppercase, lowercase, a digit, and a special character.",
                text_color="#FF6B6B"
            )
            return

        try:
            response = self.client.send_request("create_account",
{"username": username, "password": password})
            if response.get("status") == "error":

```

```

        tkmb.showerror("Account Creation Failed",
response.get("message", "Failed to create account.))
        error_label.configure(text=response.get("message", "Failed
to create account."), text_color="#FF6B6B")
    else:
        success_label.configure(text="Account created successfully!",
text_color="green")
        tkmb.showinfo("Account Created", "Your account has been
created successfully.")
        new_account_window.destroy()

    except Exception as db_e:
        error_label.configure(text=f"Server Error: {db_e}",
text_color="#FF6B6B")

    # Create the new account window
    new_account_window = customtkinter.CTk()
    new_account_window.title("Create New Account")
    new_account_window.geometry("300x400")

    # Username Entry
    #customtkinter.CTkLabel(new_account_window, text="Username",
font=("Trebuchet MS", 14)).pack(pady=5)
    new_user_name_entry = customtkinter.CTkEntry(new_account_window,
width=250,placeholder_text="Username", font=("Trebuchet MS", 14))
    new_user_name_entry.pack(pady=5)

    # Password Entry with Preview Toggle
    pass_frame = customtkinter.CTkFrame(new_account_window,
fg_color="#EAE6F0", border_width=0)
    pass_frame.pack(pady=5)
    new_user_password_entry = customtkinter.CTkEntry(
        master=pass_frame,
        placeholder_text="Password",
        show="*",
        width=200,
        font=("Trebuchet MS", 14)
    )
    new_user_password_entry.pack(side="left", padx=(0,5))

    def toggle_new_acct_password():
        if new_user_password_entry.cget("show") == "*":
            new_user_password_entry.configure(show="")
            new_toggle_btn.configure(text="👁️")
        else:

```

```

        new_user_password_entry.configure(show="*")
        new_toggle_btn.configure(text="👁")
new_toggle_btn = customtkinter.CTkButton(
    master=pass_frame,
    text="👁",
    width=40,
    command=toggle_new_acct_password
)
new_toggle_btn.pack(side="left")

# Button to Suggest a Strong Password
suggest_pass_button = customtkinter.CTkButton(
    new_account_window,
    text="Suggest Password",
    width=150,
    command=lambda: new_user_password_entry.delete(0, "end") or
new_user_password_entry.insert(0, suggest_strong_password())
)
suggest_pass_button.pack(pady=5)

# Error and Success Labels
error_label = customtkinter.CTkLabel(new_account_window, text="",
text_color="#FF6B6B", wraplength=280, justify="left", font=("Arial", 10))
error_label.pack(pady=5)
success_label = customtkinter.CTkLabel(new_account_window, text="",
text_color="green", wraplength=280, justify="left", font=("Arial", 10))
success_label.pack(pady=5)

# Submit Button
submit_button = customtkinter.CTkButton(new_account_window,
text="Submit", command=submit_new_account, width=150)
submit_button.pack(pady=5)

new_account_window.mainloop()

def run(self):
    """Run the login window."""
    self.app.mainloop()

```

```

import customtkinter
import tkinter.messagebox as tkmb
import tkinter.filedialog as fd

from Echo_HomePage import Homepage
from Echo_DigitalLibrary import DigitalLibrary

class LandingPage:
    def __init__(self, client, username):
        """
        Initialize the homepage window.
        Args:
            client: The client object handling server communication.
            username: The logged-in user's username.
        """
        self.client = client
        self.username = username
        # Create the main window with Echo's signature dark background
        self.home_window = customtkinter.CTk()
        self.home_window.title("Echo")
        self.h_w, self.h_h = self.home_window.winfo_screenwidth(),
self.home_window.winfo_screenheight()
        self.home_window.geometry(f"{self.h_w}x{self.h_h}+0+0")
        self.home_window.configure(bg="#343A40") # Dark Gray background
        customtkinter.set_widget_scaling(1.2)
        self.create_widgets()

    def create_widgets(self):
        """Design the homepage like the old design with an explanation frame and
        two main buttons."""
        # App Name Frame
        app_name_frame = customtkinter.CTkFrame(
            master=self.home_window,
            width=self.h_w,
            height=80,
            corner_radius=10,
            fg_color="#EAE6F0",
            border_color="#6B7082",
            border_width=2
        )
        app_name_frame.pack(pady=0)
        app_name_label = customtkinter.CTkLabel(
            master=app_name_frame,

```

```

        text="Echo",
        font=("Lucida Handwriting", 36, "bold"),
        text_color="#C4C1E0"
    )
    app_name_label.pack(pady=20)

    # Explanation Frame
    explanation_frame = customtkinter.CTkFrame(
        master=self.home_window,
        width=self.h_w,
        height=80,
        corner_radius=10,
        fg_color="#EAE6F0",
        border_color="#A1A1C0",
        border_width=2
    )
    explanation_frame.pack(pady=10)
    explanation_label = customtkinter.CTkLabel(
        master=explanation_frame,
        text="Here you can choose to create a NEW BOOK or visit your DIGITAL
LIBRARY.",
        font=("Trebuchet MS", 14),
        text_color="#343A40"
    )
    explanation_label.pack(pady=10)

    # Navigation Buttons
    button_frame = customtkinter.CTkFrame(
        master=self.home_window,
        fg_color="#BFD3EB",
        corner_radius=15
    )
    button_frame.pack(pady=20, padx=20)

    new_book_button = customtkinter.CTkButton(
        master=button_frame,
        text="New Book",
        command=self.open_new_book,
        font=("Trebuchet MS", 12),
        fg_color="#A1C2D1",
        hover_color="#BFD3EB",
        text_color="#FBF3EA",
        width=200
    )
    new_book_button.pack(pady=10)

```

```

digital_library_button = customtkinter.CTkButton(
    master=button_frame,
    text="Digital Library",
    command=self.open_digital_library,
    font=("Trebuchet MS", 12),
    fg_color="#A1C2D1",
    hover_color="#BFD3EB",
    text_color="#FBF3EA",
    width=200
)
digital_library_button.pack(pady=10)
def open_new_book(self):
    """
    Open the New Book window by hiding the Homepage and starting the NewBook
    window.
    """
    try:
        # Hide the Homepage window
        self.home_window.withdraw()

        from Echo_HomePage import Homepage
        homepage_window = Homepage(
            client=self.client,
            username=self.username,
            go_back_callback=self.home_window.deiconify # When NewBook
            closes, this will show the Homepage again
        )

        homepage_window.run()

    except Exception as e:
        tkmb.showerror("Error", f"An error occurred while opening the New
        Book window: {e}")

def open_digital_library(self):
    try:
        # Hide the Homepage window
        self.home_window.withdraw()

        # Open the Digital Library window
        DigitalLibrary(
            parent_window=self.home_window,

```



```
        go_back_callback=self.home_window.deiconify, # Show the Homepage
when going back
        username=self.username,
        client=self.client # Pass the client to the Digital Library
    )
    except Exception as e:
        tkmb.showerror("Error", f"An error occurred while opening the Digital
Library: {e}")

def run(self):
    """Run the homepage main loop."""
    self.home_window.mainloop()
```

```

import customtkinter
from tkinter import filedialog as fd
import tkinter.messagebox as tkmb
from urllib.request import urlopen
from Echo_BookCreationWindow import BookCreationWindow
from tkinter import simpledialog
from Echo_DigitalFlipbook import DigitalFlipbookApp
from Echo_RecordingLab import RecordingLab

class Homepage:
    def __init__(self, client, username, go_back_callback):
        """
        Initialize the HomePage window.

        Args:
            client: The client object handling server communication.
            username: The logged-in user's username.
            go_back_callback: Callback function to return to the previous
(Landing) window.
        """
        self.client = client
        self.username = username
        self.go_back_callback = go_back_callback
        self.selected_audio_file = None

        # Create the HomePage window as a Toplevel window
        self.newbook_window = customtkinter.CTkToplevel()
        self.newbook_window.title("New Book")
        self.h_w = self.newbook_window.winfo_screenwidth()
        self.h_h = self.newbook_window.winfo_screenheight()
        self.newbook_window.geometry(f"{self.h_w}x{self.h_h}+0+0")
        self.newbook_window.state("zoomed")
        self.newbook_window.configure(bg="#343A40")

        self.create_widgets()

    def create_widgets(self):
        """
        Create and arrange all widgets in the HomePage window.
        """
        # Explanation Frame- Title & Description
        explanation_frame = customtkinter.CTkFrame(
            master=self.newbook_window,

```

```

        width=self.h_w,
        height=150,
        corner_radius=10,
        fg_color="#EAE6F0",
        border_color="#A1A1C0",
        border_width=2
    )
    explanation_frame.pack(pady=20)

    # Title label
    title_label = customtkinter.CTkLabel(
        master=explanation_frame,
        text="New Book",
        font=("Palatino Linotype", 36, "bold"),
        text_color="#343A40"
    )
    title_label.pack(pady=(10, 5))

    # Main Content Frame
    main_frame = customtkinter.CTkFrame(
        master=self.newbook_window,
        fg_color="#BFD3EB",
        corner_radius=15
    )
    main_frame.pack(pady=20, padx=20, fill="both", expand=True)

    # Welcome Label
    welcome_label = customtkinter.CTkLabel(
        master=main_frame,
        text=f"Here you can choose to create the new book from an existing
audio file or record a new one!",
        font=("Palatino Linotype", 14, "bold"),
        text_color="#343A40"
    )
    welcome_label.pack(pady=20)

    # Select File Button
    self.submit_button = customtkinter.CTkButton(
        master=main_frame,
        text="Select File",
        command=self.select_file,
        fg_color="#A1C2D1"
    )
    self.submit_button.pack(pady=10)

```

```

# Process File Button for Book Creation
process_button = customtkinter.CTkButton(
    master=main_frame,
    text="Process File",
    command=self.submit_audio_file,
    fg_color="#A1C2D1"
)
process_button.pack(pady=10)

# Recording Lab Access Button
record_lab_button = customtkinter.CTkButton(
    master=main_frame,
    text="Recording Lab",
    command=self.recording_lab,
    fg_color="#A1C2D1"
)
record_lab_button.pack(pady=10)

# Go Back Button
footer_frame = customtkinter.CTkFrame(master=self.newbook_window,
fg_color="#6B7082", corner_radius=10)
footer_frame.pack(fill="x", pady=20)

go_back_button = customtkinter.CTkButton(
    master=footer_frame,
    text="Go Back",
    font=("Trebuchet MS", 16, "bold"),
    fg_color="#A1C2D1",
    hover_color="#EAE6F0",
    text_color="#343A40",
    corner_radius=10,
    command=self.go_back,
)
go_back_button.pack(pady=10)

def select_file(self):
    """Allow the user to select an audio file."""
    filetypes = (('Audio Files', '*.wav'), ('All Files', '*.*')) # Define the
file types allowed
    filename = fd.askopenfilename(
        title="Select an Audio File", # Title for the file selection dialog
        initialdir="/", # Initial directory to open
        filetypes=filetypes # File types to filter
    )
    if filename: # If a file is selected

```

```

        self.selected_audio_file = filename # Store the selected audio file
        self.submit_button.configure(
            text=f"File Selected: {filename.split('/')[-1]}", # Update the
button text to show the selected file
            fg_color="light green"
        )

    def submit_audio_file(self):
        """
        Submit the audio file for transcription using the client's server
request,
        and then open the Book Creation Window with the transcribed text.
        """
        try:
            if not self.selected_audio_file: # If no audio file is selected
                tkmb.showerror("Error", "No audio file selected. Please select a
file before submitting.")
                return

            command = "create_digital_book" # Command to create a digital book
            data = {"audio_file": self.selected_audio_file} # Prepare data with
the selected audio file
            response = self.client.send_request(command, data) # Send request to
server for transcription

            if response.get("status") == "success": # If the response is
successful
                transcription_text = response.get("transcription") # Get the
transcription text from the server
                if not transcription_text: # If no transcription text is received
                    tkmb.showerror("Error", "No transcription data received from
the server.")
                    return

                self.newbook_window.withdraw() # Hide the HomePage window
                book_creation_window = BookCreationWindow(
                    self.newbook_window, # Pass the current window as parent
                    client=self.client, # Pass the client for server
communication
                    username=self.username, # Pass the username
                    transcription_text=transcription_text # Pass the
transcription text
                )
                book_creation_window.run() # Open the BookCreationWindow
            else: # If the server response indicates failure

```

```

        tkmb.showerror("Error", response.get("message", "An unknown error
occurred.))
    except Exception as e: # If an exception occurs
        tkmb.showerror("Error", f"An error occurred while submitting the
audio file: {e}")

def recording_lab(self):
    """Access the Recording Lab window."""
    try:
        def go_back_to_homepage(): # Show the HomePage window again
            self.newbook_window.deiconify()
            self.newbook_window.state("zoomed")

        self.newbook_window.withdraw() # Hide the current window

        RecordingLab(
            parent_window=self.newbook_window, # Pass the current window as
parent
            go_back_callback=go_back_to_homepage, # Pass the callback
function to return to HomePage
            client=self.client, # Pass the client for server communication
            username=self.username # Pass the username
        )
    except Exception as e: # If an error occurs while opening the Recording
Lab
        tkmb.showerror("Error", f"An error occurred while opening the
Recording Lab: {e}")

def go_back(self):
    """
    Close the HomePage window and trigger the go_back_callback to return to
the previous (Landing) window.
    """
    self.newbook_window.destroy() # Destroy the current window
    self.go_back_callback() # Call the callback function to go back to the
previous window

def run(self):
    """Start the HomePage window's main loop."""
    self.newbook_window.mainloop() # Start the Tkinter main event loop to
display the window

```

```

import customtkinter
import tkinter.messagebox as tkmb
from Echo_DigitalFlipbook import DigitalFlipbookApp
from tkinter import simpledialog

class DigitalLibrary:
    def __init__(self, parent_window, go_back_callback, username, client):
        """
        Initialize the Digital Library window.
        Args:
            parent_window: The Homepage window to return to.
            go_back_callback: A callback function to return to the Homepage.
            username: The logged-in user's username.
            client: The client object for server communication.
        """
        self.parent_window = parent_window
        self.go_back_callback = go_back_callback
        self.username = username # Used for database queries
        self.pending_action = None # Will be set to "edit" or "delete" when
        activated
        self.client = client # Client object for server communication

        # Create the Digital Library window
        self.library_window = customtkinter.CTkToplevel(self.parent_window)
        self.library_window.title("Digital Library")
        self.library_window.state("zoomed")
        self.library_window.protocol("WM_DELETE_WINDOW", self.go_back) # Handle
        window close event
        self.library_window.configure(bg="#343A40") # Dark gray background

        # Styling and fonts
        self.title_font = ("Palatino Linotype", 36, "bold")
        self.link_font = ("Trebuchet MS", 16, "bold")
        self.text_color = "#FBF3EA"

        # Initialize the interface
        self.initialize_window()

    def initialize_window(self):
        """
        Set up the Digital Library window, including fetching and displaying
        books.
        """

```

```

    # Header frame for the title and action buttons
    header_frame = customtkinter.CTkFrame(master=self.library_window,
fg_color="#6B7082", corner_radius=10)
    header_frame.pack(fill="x", pady=20, padx=20)

    # Left side: Title label
    title_label = customtkinter.CTkLabel(
        master=header_frame,
        text="Digital Library",
        font=self.title_font,
        text_color=self.text_color,
    )
    title_label.pack(side="left", padx=10, pady=10)

    # Right side: Action buttons and instruction label
    action_frame = customtkinter.CTkFrame(master=header_frame,
fg_color="#6B7082", corner_radius=10)
    action_frame.pack(side="right", padx=10)

    # Edit Name Button
    self.edit_action_button = customtkinter.CTkButton(
        master=action_frame,
        text="Edit Name",
        font=("Trebuchet MS", 12),
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        command=self.activate_edit_mode
    )
    self.edit_action_button.grid(row=0, column=0, padx=5, pady=2)

    # Delete Book Button
    self.delete_action_button = customtkinter.CTkButton(
        master=action_frame,
        text="Delete Book",
        font=("Trebuchet MS", 12),
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        command=self.activate_delete_mode
    )
    self.delete_action_button.grid(row=0, column=1, padx=5, pady=2)

    # Instruction label to guide the user after selecting an action
    self.action_instruction_label = customtkinter.CTkLabel(

```



```

        master=action_frame,
        text="",
        font=("Trebuchet MS", 12, "bold"),
        text_color=self.text_color
    )
    self.action_instruction_label.grid(row=1, column=0, columnspan=2, pady=5)

    # Content frame for the list of books
    self.content_frame = customtkinter.CTkScrollableFrame(
        master=self.library_window,
        fg_color="#BFD3EB",
        corner_radius=10,
        width=700,
        height=400
    )
    self.content_frame.pack(pady=20, padx=20, fill="both", expand=True)

    # Fetch and display books
    self.display_books()

    # Footer frame for the "Go Back" button
    footer_frame = customtkinter.CTkFrame(master=self.library_window,
        fg_color="#6B7082", corner_radius=10)
    footer_frame.pack(fill="x", pady=20, padx=20)

    go_back_button = customtkinter.CTkButton(
        master=footer_frame,
        text="Go Back",
        font=self.link_font,
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        corner_radius=10,
        command=self.go_back,
    )
    go_back_button.pack(pady=10)

    def display_books(self):
        """
        Clear any existing content and fetch books from the database to create
        clickable book buttons.
        """
        # Clear current content
        for widget in self.content_frame.winfo_children():
            widget.destroy()

```

```

        books = self.fetch_books()
        for book_id, book_name in books:
            self.create_clickable_book(self.content_frame, book_id, book_name)

    def fetch_books(self):
        """
        Fetch the books associated with the logged-in user.
        Returns:
            list: A list of tuples (book_id, book_name).
        """
        try:
            response = self.client.send_request("get_user_id", {"username":
self.username})
            if response.get("status") == "error":
                tkmb.showerror("Error", f"No user found with username
'{self.username}'")
                return []
            user_id = response.get("user_id")
            # Fetch books owned by the user
            response = self.client.send_request("get_books_list", {"user_id":
user_id})
            books = response.get("books")
            if books:
                return books
            else:
                return []

        except Exception as e:
            tkmb.showerror("Error", f"An error occurred while fetching books:
{e}")
            return []

    def create_clickable_book(self, parent, book_id, book_name):
        """
        Create a clickable book title. In normal mode, clicking opens the book.
        If a pending action (edit or delete) is active, then the clicked book is
targeted for that action.
        Args:
            parent: The parent frame where the clickable title will be added.
            book_id (int): The book's ID.
            book_name (str): The name of the book.
        """
        book_button = customtkinter.CTkButton(
            master=parent,

```

```

        text=book_name,
        font=self.link_font,
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        corner_radius=5,
        command=lambda: self.handle_book_click(book_id, book_name)
    )
    book_button.pack(pady=5, fill="x", padx=20)

def handle_book_click(self, book_id, book_name):
    """
    Handle a click on a book title. If no pending action exists, open the
    book.

    If an action is pending, perform the action on this book.
    """
    if self.pending_action is None:
        self.open_book(book_id)
    elif self.pending_action == "edit":
        self.edit_book_name(book_id)
        self.pending_action = None
        self.action_instruction_label.configure(text="")
        # Reset the Edit button color
        self.edit_action_button.configure(fg_color="#A1C2D1",
        hover_color="#EAE6F0")
        self.refresh_library()
    elif self.pending_action == "delete":
        self.confirm_delete(book_id)
        self.pending_action = None
        self.action_instruction_label.configure(text="")
        # Reset the Delete button color
        self.delete_action_button.configure(fg_color="#A1C2D1",
        hover_color="#EAE6F0")
        self.refresh_library()

def activate_edit_mode(self):
    """
    Toggle Edit Name mode.
    If already active, turn it off. Otherwise, activate it.
    """
    if self.pending_action == "edit":
        # Turn off edit mode
        self.pending_action = None
        self.edit_action_button.configure(fg_color="#A1C2D1",
        hover_color="#EAE6F0")

```

```

        self.action_instruction_label.configure(text="")
    else:
        # If delete mode is active, cancel it
        if self.pending_action == "delete":
            self.delete_action_button.configure(fg_color="#A1C2D1",
hover_color="#EAE6F0")
            self.pending_action = "edit"
            self.edit_action_button.configure(fg_color="light green",
hover_color="light green")
            self.action_instruction_label.configure(text="Edit Mode: Click on the
book you wish to rename.")

    def activate_delete_mode(self):
        """
        Toggle Delete mode.
        If already active, turn it off. Otherwise, activate it.
        """
        if self.pending_action == "delete":
            # Turn off delete mode
            self.pending_action = None
            self.delete_action_button.configure(fg_color="#A1C2D1",
hover_color="#EAE6F0")
            self.action_instruction_label.configure(text="")
        else:
            if self.pending_action == "edit":
                self.edit_action_button.configure(fg_color="#A1C2D1",
hover_color="#EAE6F0")
                self.pending_action = "delete"
                self.delete_action_button.configure(fg_color="light green",
hover_color="light green")
                self.action_instruction_label.configure(text="Delete Mode: Click on
the book you wish to delete.")

    def edit_book_name(self, book_id):
        """
        Allow the user to change the book's name.
        After updating the database, refresh the digital library.
        """
        # Fetch the current name as a default
        current_name = None
        try:
            response = self.client.send_request("get_book_name", {"book_id":
book_id})
            if response.get("status") == "error":
                tkmb.showerror("Error", "Could not fetch book name")

```

```

        return
        current_name = response.get("book_name")
        new_name = simpledialog.askstring("Edit Name", "Enter the new name
for the book:", initialValue=current_name)
        if new_name and new_name != current_name:
            response = self.client.send_request("update_book_name",
{"book_id": book_id, "new_name": new_name})
            if response.get("status") == "error":
                tkmb.showerror("Error", "Could not update book name")

        except Exception as e:
            tkmb.showerror("Error", f"An error occurred while updating the
book name: {e}")

    def confirm_delete(self, book_id):
        """
        Use a two-step confirmation process before deleting the book.
        After deletion, refresh the library.
        """
        confirm = tkmb.askyesno("Confirm Delete", "Are you sure you want to
delete this book?")
        if confirm:
            final_confirm = tkmb.askyesno("Final Confirm Delete", "This action
cannot be undone. Delete the book?")
            if final_confirm:
                try:
                    response = self.client.send_request("delete_book",
{"book_id": book_id})
                    if response.get("status") == "error":
                        tkmb.showerror("Error", "Could not delete book")
                except Exception as e:
                    tkmb.showerror("Error", f"An error occurred while deleting
the book: {e}")

    def refresh_library(self):
        """
        Refresh the Digital Library display by clearing all widgets
        and reinitializing the window.
        """
        for widget in self.library_window.winfo_children():
            widget.destroy()
        self.initialize_window()
    def open_book(self, book_id):
        """
        Open the book in the DigitalFlipbookApp.

```

```

    Args:
        book_id (int): The ID of the book to open.
    """
    try:
        # Hide the Digital Library window before opening the flipbook.
        self.library_window.withdraw()

        # Open the DigitalFlipbook in a new, independent Toplevel window.
        flipbook_window = customtkinter.CTkToplevel()
        DigitalFlipbookApp(flipbook_window,
book_id=book_id,client=self.client)

        # Set a protocol so that when the flipbook is closed, the Digital
        Library is restored.
        flipbook_window.protocol("WM_DELETE_WINDOW", lambda:
self.close_flipbook(flipbook_window))
    except Exception as e:
        tkmb.showerror("Error", f"An error occurred while opening the book:
{e}")

    def close_flipbook(self, flipbook_window):
        """
        Handle the closure of the digital flipbook and restore the Digital
        Library window.
        """
        # Destroy the flipbook window.
        flipbook_window.destroy()

        # Restore the Digital Library window.
        self.library_window.deiconify()
        # Set the state back to zoomed (full screen) so it looks the same as
        before.
        self.library_window.state("zoomed")

    def go_back(self):
        """
        Close the Digital Library window and execute the go-back callback.
        """
        self.library_window.destroy()
        self.go_back_callback()

```

```

import customtkinter
import tkinter.messagebox as tkmb
from tkinter import simplifiedialog

class DigitalFlipbookApp:
    def __init__(self, root, book_id, client):
        # Initialize the application with the root window and a book ID
        self.root = root
        self.root.state("zoomed")
        self.root.title("Digital Flipbook")
        self.book_id = book_id # Store the book ID for future reference
        self.client = client # Store the client object for server communication

        # Load the book's title, author, and content from the database
        self.book_title, self.author, self.book_text =
self.load_book_from_db(book_id)

        # Initialize the edit manager and set the current page and font size
        self.edit_manager = self.EditManager(self)
        self.current_page = 0
        self.current_font_size = 18

        # Create the widgets
        self.create_widgets()

    def create_widgets(self):
        self.cover_page = customtkinter.CTkFrame(self.root, corner_radius=10,
border_width=2, border_color="gray") # Create a frame for the cover page with
styling

        title_label = customtkinter.CTkLabel( # Add a label for the book title in
the cover page
            self.cover_page,
            text=self.book_title,
            text_color="#343A40",
            font=("Times New Roman", 24, "bold"),
            wraplength=500,
            justify="center",
        )
        title_label.pack(pady=100)

        author_label = customtkinter.CTkLabel( # Add a label for the author name
under the title

```

```

        self.cover_page,
        text=f"by {self.author}",
        font=("Times New Roman", 18),
        text_color="#343A40",
        wraplength=500,
        justify="center",
    )
    author_label.pack(pady=10)

    self.content_page = customtkinter.CTkScrollableFrame( # Create a
scrollable frame for the content page
        self.root, fg_color="#BFD3EB", corner_radius=10, width=800,
height=600
    )

    self.content_label = customtkinter.CTkLabel( # Add a label for the
content of the book
        self.content_page,
        text=self.book_text,
        font=("Times New Roman", self.current_font_size),
        wraplength=750, # Wrap text after 750px
        justify="left", # Align text to the left
        text_color="#343A40"
    )
    self.content_label.pack(pady=20, padx=20)

    self.button_frame = customtkinter.CTkFrame(self.root, fg_color="#6B7082",
corner_radius=10) # Create a frame for buttons
    self.button_frame.pack(side="bottom", fill="x", pady=30, padx=30)

    self.download_button = customtkinter.CTkButton( # Create a button to
download the PDF of the book
        self.root,
        text="Download PDF",
        fg_color="#A1C2D1",
        command=self.download_pdf
    )

    self.prev_button = customtkinter.CTkButton(
        self.button_frame, text="<< Previous", fg_color="#A1C2D1",
command=self.prev_page
    )
    self.next_button = customtkinter.CTkButton(
        self.button_frame, text="Next >>", fg_color="#A1C2D1",
command=self.next_page

```



```

    )
    self.prev_button.pack(side="left", padx=20) # Add left padding to the
previous button
    self.next_button.pack(side="right", padx=20) # Add right padding to the
next button

    self.edit_button = customtkinter.CTkButton(
        self.root, text="Edit Content", fg_color="#A1C2D1",
command=self.edit_manager.edit_content
    )
    self.append_button = customtkinter.CTkButton(
        self.root, text="Append Content", fg_color="#A1C2D1",
command=self.edit_manager.append_content
    )

    self.save_button = customtkinter.CTkButton( # Save button to save the
changes
        self.button_frame,
        text="Save",
        font=("Arial", 14, "bold"),
        fg_color="#A1C2D1",
        hover_color="#EAE6F0",
        text_color="#343A40",
        command=self.save_changes,
    )
    self.save_button.pack(side="bottom", pady=20)

    self.font_slider = customtkinter.CTkSlider( # Create a slider to adjust
the font size of the content
        self.root,
        from_=12, # Minimum font size
        to=30, # Maximum font size
        number_of_steps=18, # Number of steps in the slider
        command=self.update_font_size
    )
    self.font_slider.set(self.current_font_size)

    self.show_page(self.current_page) # Display the current page (starts at
page 0)

    def update_font_size(self, value): # Update the font size based on the slider
value
        self.current_font_size = int(value)
        if isinstance(self.content_label, customtkinter.CTkTextbox): # Adjust the
font of the content label to the new font size

```

```

        self.content_label.configure(font=("Times New Roman",
self.current_font_size))
    else:
        self.content_label.configure(font=("Times New Roman",
self.current_font_size))

    def load_book_from_db(self, book_id): # Load book details (title, content,
author) from the database via server based on book ID
        try:
            response = self.client.send_request("load_book_from_db", {"book_id":
book_id}) # Send request to server to load book data
            if response.get("status") == "error":
                raise ValueError(f"No book found with ID {book_id}")
            book_title = response.get("books_title") # Get book title from the
response

            author = response.get("author") # Get author name from the response
            text = response.get("text") # Get book content from the response
            return book_title, author, text

        except Exception as e: # If any error occurs, display an error message
            tkmb.showerror("Error", f"An error occurred while loading the book:
{e}")

            return "Error", "Unknown", "No content available."

    def save_changes(self): # Save changes to the database
        try:
            updated_text = self.content_label.cget("text") if
isinstance(self.content_label, customtkinter.CTkLabel) else
self.content_label.get("1.0", "end").strip()
            if not hasattr(self, "book_id") or not self.book_id:
                raise ValueError("No valid book_id provided to update the
database.")
            response = self.client.send_request("save_text_changes", {"book_id":
self.book_id, "updated_text": updated_text}) # Send request to server
            if response.get("status") == "success":
                tkmb.showinfo("Save Successful", "Your changes have been saved
successfully!")
            else: # If the response indicates an error, raise a ValueError with
the error message
                raise ValueError(f"An error occurred while saving changes:
{response.get('message')}")

        except Exception as e:
            tkmb.showerror("Error", f"An error occurred while saving changes:
{e}")

```

```

def download_pdf(self): # Download the book as a PDF
    try:
        from reportlab.lib.pagesizes import letter
        from reportlab.pdfgen import canvas
        import os
        import textwrap

        downloads_folder = os.path.expanduser("~/Downloads")
        filename = os.path.join(downloads_folder, f"{self.book_title}.pdf")

        c = canvas.Canvas(filename, pagesize=letter)
        width, height = letter

        # Set the title font and write the book title and author
        c.setFont("Times-Bold", 24)
        c.drawCentredString(width / 2, height - 100, self.book_title)
        c.setFont("Times-Roman", 18)
        c.drawCentredString(width / 2, height - 130, f"by {self.author}")
        c.showPage()

        # Create a text object to write the content
        textobject = c.beginText(40, height - 50)
        textobject.setFont("Times-Roman", 12)

        content = self.content_label.get("1.0", "end").strip() if
isinstance(self.content_label, customtkinter.CTkTextbox) else
self.content_label.cget("text")

        # Wrap text and write to PDF
        for paragraph in content.split("\n"):
            wrapped_lines = textwrap.wrap(paragraph, width=80) or [""] # Wrap
lines to avoid overflow
            for line in wrapped_lines:
                textobject.textLine(line)

        c.drawText(textobject) # Add the text object to the canvas
        c.save() # Save the PDF

        tkmb.showinfo("Download PDF", f"PDF successfully created and saved:
{filename}")
    except Exception as e:
        tkmb.showerror("Error", f"An error occurred while exporting PDF:
{e}")

```

```

def hide_all_pages(self): # Hide all pages and controls
    self.cover_page.pack_forget()
    self.content_page.pack_forget()
    self.edit_button.place_forget()
    self.append_button.place_forget()
    self.download_button.place_forget()
    self.font_slider.place_forget()

def show_page(self, page_index): # Show the relevant page based on the index
    (0: cover, 1: content)
    self.hide_all_pages() # Hide any previously shown pages before displaying
a new one
    if page_index == 0: # If the page index is 0 (cover page) Display the
cover page
        self.cover_page.pack(fill="both", expand=True)
    elif page_index == 1: # If the page index is 1 (content page) Display the
content page
        self.content_page.pack(fill="both", expand=True) # Position buttons
on the screen
        self.edit_button.place(x=10, y=10)
        self.append_button.place(x=130, y=10)
        self.download_button.place(x=250, y=10)
        self.font_slider.place(relx=0.95, y=10, anchor="ne")
    self.update_buttons(page_index) # Update button visibility and position
based on the current page index

def update_buttons(self, page_index):
    self.prev_button.pack_forget() # Hide the previous page button
    self.next_button.pack_forget() # Hide the next page button
    if page_index == 0: # If on the cover page show the next button to go to
the content page
        self.next_button.pack(side="right", padx=20)
    elif page_index == 1: # If on the cover page show the prev button to go
back to the cover page
        self.prev_button.pack(side="left", padx=20)

def next_page(self):
    self.current_page = 1 # Set the current page to content page (1)
    self.show_page(self.current_page)

def prev_page(self):
    self.current_page = 0 # Set the current page to cover page (0)
    self.show_page(self.current_page)

```

```

class EditManager:
    def __init__(self, flipbook_app):
        self.flipbook_app = flipbook_app # Store the reference to the main
flipbook application

    def edit_content(self):
        if not hasattr(self.flipbook_app, "is_editing") or not
self.flipbook_app.is_editing: # Check if the app is in editing mode
            self.flipbook_app.is_editing = True # Set editing mode to True
            current_text = self.flipbook_app.content_label.cget("text") #
Get the current content text
            self.flipbook_app.content_label.destroy() # Destroy the current
label to replace it with a textbox
            self.flipbook_app.content_label = customtkinter.CTkTextbox(
                self.flipbook_app.content_page, width=750, height=400,
font=("Times New Roman", self.flipbook_app.current_font_size)
            )
            # Create a new text box for editing
            self.flipbook_app.content_label.insert("1.0", current_text)
            self.flipbook_app.content_label.configure(state="normal")
            self.flipbook_app.content_label.pack(pady=20, padx=20)
            self.flipbook_app.edit_button.configure(text="Done Editing",
fg_color="#90EE90")

        else: # If already in editing mode, save changes and return to view
mode
            new_text = self.flipbook_app.content_label.get("1.0",
"end").strip() # Get the edited text
            self.flipbook_app.content_label.destroy() # Remove the editable
text box
            self.flipbook_app.content_label = customtkinter.CTkLabel(
                self.flipbook_app.content_page,
                text=new_text,
                font=("Times New Roman",
self.flipbook_app.current_font_size),
                wraplength=750,
                justify="left",
                text_color="#343A40"
            )
            # Create a new label with the updated text
            self.flipbook_app.content_label.pack(pady=20, padx=20)
            self.flipbook_app.is_editing = False
            self.flipbook_app.edit_button.configure(text="Edit Content",
fg_color="#A1C2D1")
            self.flipbook_app.save_changes()

```

```

def append_content(self):
    additional_text = simpledialog.askstring("Append Content", "Enter the
text to append:")
    if additional_text: # If additional text is provided, append it to
the current content
        current_text = self.flipbook_app.content_label.cget("text") if
isinstance(self.flipbook_app.content_label, customtkinter.CTkLabel) else
self.flipbook_app.content_label.get("1.0", "end").strip() # Get current content
        new_text = current_text + "\n" + additional_text # Append the new
text
        if isinstance(self.flipbook_app.content_label,
customtkinter.CTkTextbox): # If content is in a text box
            self.flipbook_app.content_label.delete("1.0", "end") # Clear
the existing text
            self.flipbook_app.content_label.insert("1.0", new_text) #
Insert the updated text
        else: # Otherwise, update the label text with new content
            self.flipbook_app.content_label.configure(text=new_text)

```

```

import customtkinter
import pyaudio
import wave
import threading
from pydub import AudioSegment
from pydub.playback import play
import tkinter.messagebox as tkmb

import cloundinary
from Echo_BookCreationWindow import BookCreationWindow
import time
import json
from urllib.request import urlopen

class RecordingLab:
    def __init__(self, parent_window, go_back_callback, client, username):
        """
        Initialize the recording lab window.
        Args:
            parent_window: The Homepage window to return to.
            go_back_callback: Callback function to return to the Homepage.
            client: Instance of EchoClient for server communication.
            username: Username of the logged-in user.
        """
        # Initialize attributes
        self.parent_window = parent_window
        self.go_back_callback = go_back_callback
        self.client = client # EchoClient instance for server communication
        self.username = username # Store the logged-in username
        self.recorder = self.RecordAudio()

        # Create the Recording Lab window
        self.recording_window = customtkinter.CTkToplevel(self.parent_window)
        self.recording_window.title("Recording Lab")
        self.recording_window.state("zoomed") # Make the window fullscreen
        (maximized)
        self.recording_window.protocol("WM_DELETE_WINDOW", self.go_back) #
        Handle window close event
        self.recording_window.configure(bg="#343A40") # Match dark gray
        background

        # Initialize interface
        self.create_widgets()

```

```

def create_widgets(self):
    """
    Create and style the Recording Lab interface.
    """
    # Main frame for the recording lab
    self.recording_lab_frame =
customtkinter.CTkFrame(master=self.recording_window, fg_color="#BFD3EB",
corner_radius=10)
    self.recording_lab_frame.pack(fill="both", expand=True, pady=10, padx=20)

    # Top frame for the title
    top_frame = customtkinter.CTkFrame(master=self.recording_lab_frame,
fg_color="#6B7082", corner_radius=10)
    top_frame.pack(fill="x", pady=20)

    title_label = customtkinter.CTkLabel(
        master=top_frame,
        text="Recording Lab",
        font=("Palatino Linotype", 36, "bold"), # Match DigitalLibrary title
style
        text_color="#FBF3EA",
    )
    title_label.pack(pady=10)

    # Explanation label
    explanation_label = customtkinter.CTkLabel(
        master=top_frame,
        text="Use the buttons below to start and stop your recording.",
        font=("Trebuchet MS", 14),
        text_color="#FBF3EA",
    )
    explanation_label.pack(pady=10)

    # Middle frame for control buttons
    middle_frame = customtkinter.CTkFrame(master=self.recording_lab_frame,
fg_color="#EAE6F0")
    middle_frame.pack(fill="both", expand=True, pady=20)

    # Buttons for recording actions
    self.start_recording_button = customtkinter.CTkButton(
        master=middle_frame,
        text="Start Recording",
        font=("Trebuchet MS", 12),
        command=self.start_recording,

```



```

        fg_color="light green",
        hover_color="#BFD3EB",
        text_color="#343A40",
        width=200,
    )
    self.start_recording_button.pack(pady=10)

    self.stop_recording_button = customtkinter.CTkButton(
        master=middle_frame,
        text="Stop Recording",
        state="disabled",
        font=("Trebuchet MS", 12),
        command=self.stop_recording,
        fg_color="#A1C2D1",
        hover_color="#BFD3EB",
        text_color="#343A40",
        width=200,
    )
    self.stop_recording_button.pack(pady=10)

    self.play_recording_button = customtkinter.CTkButton(
        master=middle_frame,
        text="Play Recording",
        state="disabled",
        font=("Trebuchet MS", 12),
        command=self.play_recording,
        fg_color="#A1C2D1",
        hover_color="#BFD3EB",
        text_color="#343A40",
        width=200,
    )
    self.play_recording_button.pack(pady=10)

    self.submit_recording_button = customtkinter.CTkButton(
        master=middle_frame,
        text="Submit",
        state="disabled",
        font=("Trebuchet MS", 12, "bold"),
        command=self.submit_recording,
        fg_color="#A1C2D1",
        hover_color="#BFD3EB",
        text_color="#343A40",
        width=200,
    )
    self.submit_recording_button.pack(pady=10)

```

```

        # Footer frame for the "Go Back" button
        footer_frame = customtkinter.CTkFrame(master=self.recording_lab_frame,
        fg_color="#6B7082", corner_radius=10)
        footer_frame.pack(fill="x", pady=20)

        # Go Back button matching DigitalLibrary style and functionality
        go_back_button = customtkinter.CTkButton(
            master=footer_frame,
            text="Go Back",
            font=("Trebuchet MS", 16, "bold"),
            fg_color="#A1C2D1",
            hover_color="#EAE6F0",
            text_color="#343A40",
            corner_radius=10,
            command=self.go_back,
        )
        go_back_button.pack(pady=10)

    def start_recording(self):
        # Start recording audio
        self.recorder.start_recording()
        self.start_recording_button.configure(text="Recording...",
        fg_color="#FF7F7F", state="disabled")
        self.stop_recording_button.configure(state="normal", fg_color="light
        green")

    def stop_recording(self):
        # Stop recording audio
        self.recorder.stop_recording()
        self.start_recording_button.configure(text="Start Recording",
        fg_color="light green", state="normal")
        self.stop_recording_button.configure(state="disabled",
        fg_color="#A1C2D1")
        self.submit_recording_button.configure(state="normal")
        self.play_recording_button.configure(state="normal")

    def play_recording(self):
        # Play the recorded audio
        output_recorded = "output.wav"
        audio = AudioSegment.from_wav(output_recorded)
        play(audio)

    def submit_recording(self):
        tkmb.showinfo("Submit", "Recording submitted successfully!")
        """

```

```

        Submit the recorded audio file (output.wav) for transcription using the
EchoClient
        and handle the server response.
        """
        try:
            # Path to the recorded audio file
            audio_file_path = "output.wav"

            # Ensure the recorded audio file exists
            if not audio_file_path:
                tkmb.showerror("Error", "No recorded audio found. Please record
and save an audio file before submitting.")
                return

            # Prepare the request for the server
            command = "create_digital_book"
            data = {"audio_file": audio_file_path}

            # Use EchoClient to send the request
            response = self.client.send_request(command, data)

            # Handle the server response
            if response.get("status") == "success":
                # Extract transcription from the response
                transcription_text = response.get("transcription")
                if not transcription_text:
                    tkmb.showerror("Error", "No transcription data received from
the server.")
                    return

                # Open the Book Creation Window with the transcription text
                self.recording_window.withdraw() # Hide the Recording Lab window
                book_creation_window = BookCreationWindow(
                    parent_window=self.recording_window,
                    client = self.client, # Pass the EchoClient instance
                    username=self.username, # Pass the logged-in user's username
                    transcription_text=transcription_text # Pass the transcribed
text
                )
                book_creation_window.run()
            else:
                # Display the error message from the server
                tkmb.showerror("Error", response.get("message", "An unknown error
occurred."))

```

```

except Exception as e:
    # Handle exceptions during submission
    tkmb.showerror("Error", f"An error occurred while submitting the
recorded audio: {e}")

def go_back(self):
    # Close the Recording Lab window and return to the parent window
    self.recording_window.destroy()
    self.go_back_callback()

class RecordAudio:
    def __init__(self):
        # Initialize the audio recorder
        self.is_recording = False
        self.recording_thread = None
        self.stream = None
        self.frames = []

    def start_recording(self):
        # Start recording audio
        chunk = 1024
        format = pyaudio.paInt16
        channels = 1
        rate = 44100
        self.frames = []
        self.p = pyaudio.PyAudio()

        # print('Recording')

        self.stream = self.p.open(format=format,
                                   channels=channels,
                                   rate=rate,
                                   frames_per_buffer=chunk,
                                   input=True)

        self.is_recording = True
        self.recording_thread = threading.Thread(target=self.record)
        self.recording_thread.start()

    def record(self):
        # Record audio in chunks
        chunk = 1024
        while self.is_recording:
            data = self.stream.read(chunk)
            self.frames.append(data)

```

```
def stop_recording(self):
    # Stop recording audio
    self.is_recording = False
    self.recording_thread.join()
    self.stream.stop_stream()
    self.stream.close()
    self.save_recording()
    self.p.terminate()

def save_recording(self):
    # Save the recorded audio to a file
    filename = "output.wav"
    wf = wave.open(filename, 'wb')
    wf.setnchannels(1)
    wf.setsampwidth(self.p.get_sample_size(pyaudio.paInt16))
    wf.setframerate(44100)
    wf.writeframes(b''.join(self.frames))
    wf.close()
    # print("Recording saved as output.wav")
```

```
import customtkinter
import tkinter.messagebox as tkmb
from Echo_DigitalFlipbook import DigitalFlipbookApp

class BookCreationWindow:
    def __init__(self, parent_window, client, username, transcription_text):
        """
        Initialize the book creation window.
        Args:
            parent_window: The parent window to interact with.
            username: The logged-in user's username (book owner).
            transcription_text: The transcribed text to save in the book.
        """
        self.parent_window = parent_window
        self.client = client # Client for server communication
        self.username = username
        self.transcription_text = transcription_text

        # Create book creation window
        self.book_creation_window = customtkinter.CTkToplevel(self.parent_window)
        self.book_creation_window.title("Create New Book")
        self.book_creation_window.geometry("600x400")
        self.book_creation_window.state("zoomed")

        # UI Elements
        title_label = customtkinter.CTkLabel(
            master=self.book_creation_window,
            text="Name Your Book",
            font=("Trebuchet MS", 24),
            text_color="#343A40"
        )
        title_label.pack(pady=20)

        # Book Name Entry
        self.book_name_entry = customtkinter.CTkEntry(
            master=self.book_creation_window,
            placeholder_text="Enter Book Name",
            font=("Trebuchet MS", 14),
            width=400
        )
        self.book_name_entry.pack(pady=10)

        # Submit Button
```

```

submit_button = customtkinter.CTkButton(
    master=self.book_creation_window,
    text="Submit and Open Digital Flipbook",
    font=("Trebuchet MS", 12),
    command=self.submit_book,
    fg_color="#A1C2D1",
    hover_color="#BFD3EB",
    text_color="#FBF3EA",
    width=200
)
submit_button.pack(pady=20)

def submit_book(self):
    """
    Save the book details to the database and open the Digital Flipbook.
    """
    book_name = self.book_name_entry.get().strip()

    if not book_name:
        tkmb.showerror("Error", "Book name cannot be empty.")
        return

    try:
        response = self.client.send_request("submit_book", {"username":
self.username,
                                                                    "book_name":
book_name,
                                                                    "transcription_t
ext": self.transcription_text})
        book_id = response.get("book_id")# Fetch the last inserted book ID
        if response.get("status") == "success" and book_id:
            # Notify the user
            tkmb.showinfo("Success", "Book created successfully!")
        else:
            tkmb.showerror("Error", "Book submit failed.")
            return

        # Open the digital flipbook with the book ID
        digital_book_window =
customtkinter.CTkToplevel(self.book_creation_window)
        DigitalFlipbookApp(digital_book_window, book_id,self.client)

```

```
# Close the book creation window and open the Home page window
self.parent_window.deiconify()
self.parent_window.state("zoomed")
#self.book_creation_window.destroy()

except Exception as e:
    tkmb.showerror("Error", f"An error occurred: {e}")

def run(self):
    """
    Run the book creation window.
    """
    self.book_creation_window.mainloop()
```